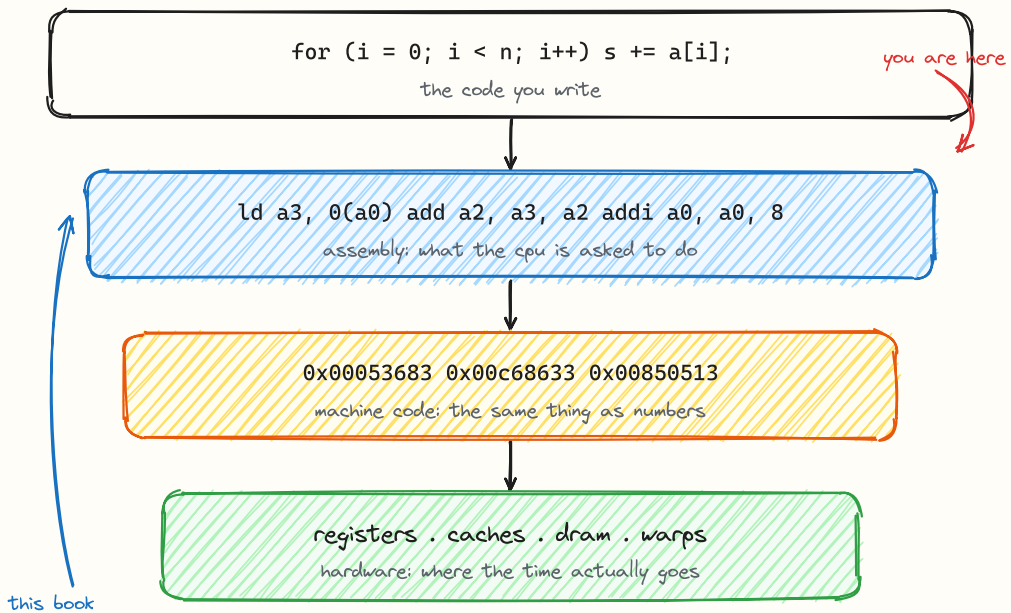


A working engineer's guide to assembly

Below the code

From bits to GPU kernels: RISC-V, x86-64, SIMD, PTX and SASS



For anyone who wants to know what their code really does

Every assembly program in this book was assembled and executed, and every compiler listing and benchmark number was produced by real tools, before it was printed here. Where that was not possible (one SASS listing in chapter 15), the text says so.

Contents

Part 1: the machine

0	How to use this book	5
1	Bits, bytes and numbers	7
2	What a CPU actually does	13

Part 2: RISC-V, the clean instruction set

3	Registers and arithmetic	18
4	Memory: loads and stores	22
5	Control flow	28
6	Functions and the stack	34
7	Whole programs: directives, syscalls, linking, encoding	40
8	Reading compiler output	44

Part 3: x86-64 and SIMD

9	x86-64 for someone who knows RISC-V	52
10	SIMD: one instruction, many numbers	60

Part 4: why code is fast or slow

11	The cost model	67
12	Case study: matrix multiply on a CPU	75

Part 5: GPUs

13	How a GPU runs code	83
14	PTX	90
15	SASS	103
16	Assembly and ML compilers	107

Appendices

A	Exercise solutions	112
B	Cheat sheets	122
C	Tools and commands	126

Part 1

The machine

Chapter 0

How to use this book

This book teaches you to read and write assembly well enough to answer one question about any piece of code: what does the hardware actually do when this runs, and what does it cost?

It goes in a fixed order. First the raw material: bits, numbers, memory, and the loop a CPU runs forever. Then RISC-V, a small and clean instruction set, where you learn every idea (registers, loads, branches, the stack, calling conventions) without historical junk in the way. Then x86-64, which is what most servers and laptops run, and SIMD, where one instruction works on 8 or 16 numbers at once. Then the cost model: why the same math can run 50 times faster or slower depending on how it touches memory. Finally GPUs: how they run code, and how to read PTX and SASS, the two levels of NVIDIA GPU assembly.

Each chapter builds on the previous one. Do not skip part 2 even if you already know some x86. RISC-V is where the ideas are clean.

How the examples were made

Nothing here is typed from memory and hoped to be right. The RISC-V programs were assembled with LLVM and executed instruction by instruction in an emulator (Unicorn), and each one was checked against a Python reference on random inputs, 245 checks in total. The x86-64 programs were assembled with gcc and run natively against a C test program. Every listing marked “compiler output” is exactly what clang or rustc printed, with only directive lines and auto-generated comments removed. The benchmark numbers are real runs on the machine this book was built on, an Intel Xeon in a shared cloud VM. Your numbers will differ. The ratios between them are the lesson, not the exact values.

The one exception is SASS in chapter 15. The NVIDIA tools that print SASS were not available where this book was built, so that listing shows the standard shape nvcc produces for that kernel on an A100 (sm_80). The chapter says so again where it matters.

What you need

A Linux machine (or WSL on Windows) and a browser. That is enough.

Tool	What it is for	How to get it
<code>godbolt.org</code>	See the assembly any compiler produces for your code, live	Open it in a browser
<code>clang</code> , <code>gcc</code>	Compile C, produce assembly with <code>-S</code>	<code>sudo apt install clang gcc</code>
<code>riscv64-linux-gnu-gcc</code> , <code>qemu-riscv64</code>	Build and run RISC-V programs on an x86 machine	<code>sudo apt install gcc-riscv64-linux-gnu qemu-user</code>
<code>objdump</code>	Disassemble a compiled file back into assembly	Comes with binutils
<code>perf</code>	Count cycles, cache misses, branch misses	<code>sudo apt install linux-tools-generic</code>
<code>nvcc</code> , <code>cuobjdump</code>	The CUDA toolkit: PTX, and SASS from compiled kernels. Installs and works without a GPU	NVIDIA's website

Appendix C lists every command used in this book.

How to study it

Read a section, then type the code yourself. Do not copy and paste. Typing forces you to look at every register name. Then change something and predict what happens before you run it. The exercises at the end of each chapter matter more than the reading. Solutions are in appendix A, and every solution there was run and checked.

When you read compiler output, read every line. If one instruction makes no sense, look it up before moving on. One skipped line becomes ten confusing ones later.

Chapter 1

Bits, bytes and numbers

Everything a computer stores is a pattern of bits. A bit is a single value that is either 0 or 1. Assembly is the level where you stop pretending otherwise, so this chapter makes sure the patterns are second nature.

Binary

A group of bits is read as a number using place values that double from right to left. The rightmost bit is worth 1, the next 2, then 4, 8, 16, and so on.

```
bit pattern:  1  0  1  1
place value:  8  4  2  1
value:        8 + 0 + 2 + 1 = 11
```

8 bits make a **byte**. A byte can hold $2^8 = 256$ different patterns, so as an unsigned number it goes from 0 to 255.

Hexadecimal

Writing 64 bits as 0s and 1s is unreadable, so people write them in base 16, called hex. Each hex digit is exactly 4 bits, which is why hex is used instead of base 10: you can convert digit by digit without doing arithmetic.

```
hex digit:  0   1   2   3   4   5   6   7
bits:       0000 0001 0010 0011 0100 0101 0110 0111

hex digit:  8   9   a   b   c   d   e   f
bits:       1000 1001 1010 1011 1100 1101 1110 1111
```

A hex number is written with `0x` in front. `0x2f` is `0010 1111` in binary, which is $32 + 15 = 47$. One byte is always exactly two hex digits. A 64-bit value is 16 hex digits.

Memorize the table above. You will read hex constantly: addresses, instruction encodings, float bit patterns, offsets.

Sizes and their names

Bits	Bytes	RISC-V name	x86 name	C type (on Linux 64-bit)
8	1	Byte (b)	Byte	char , int8_t
16	2	Half (h)	Word	short , int16_t
32	4	Word (w)	Dword	int , float , int32_t
64	8	Double (d)	Qword	long , double , pointers

Notice that “word” means 32 bits on RISC-V and 16 bits on x86. x86 kept the name from the 1970s, when it was a 16-bit chip. The instruction names carry these letters: RISC-V `lw` loads a word (4 bytes), `ld` loads a double (8 bytes). x86 writes `qword ptr` for 8 bytes.

Negative numbers: two's complement

The hardware has no minus sign. Negative integers are stored with a rule called **two's complement**: the top bit's place value is negative.

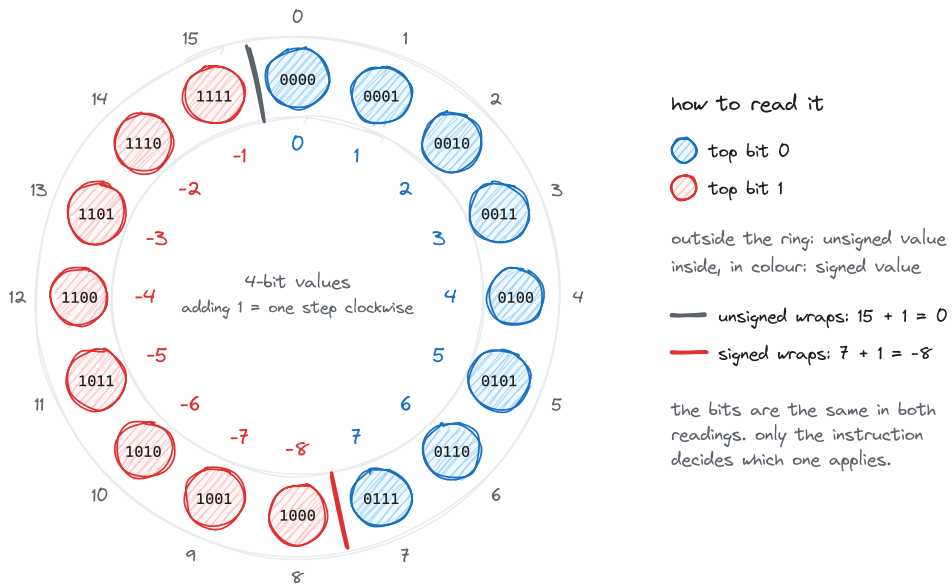
For 8 bits, the place values are:

```
-128  64  32  16  8  4  2  1
```

So `1000 0000` is -128. `1111 1111` is $-128 + 64 + 32 + 16 + 8 + 4 + 2 + 1 = -1$. `0111 1111` is 127. The signed range for 8 bits is -128 to 127. For 64 bits it is -2^{63} to $2^{63} - 1$.

Three facts fall out of this rule and you will use all three:

1. **-1 is all ones.** In 64 bits, -1 is `0xffffffffffffffff`.
2. **To negate a number, flip every bit and add 1.** 5 is `0000 0101`. Flip: `1111 1010`. Add 1: `1111 1011`. Check: $-128 + 64 + 32 + 16 + 8 + 2 + 1 = -5$.
3. **Addition and subtraction are the same circuit for signed and unsigned numbers.** Adding the bit patterns gives the right answer either way. This is why RISC-V has one `add` instruction, not a signed one and an unsigned one.

two readings of the same four bits

the point:

adding 1 is one step clockwise. overflow is stepping past the place where the reading wraps around. the bits never change meaning by themselves.

Same bits, different meaning

A register holds bits. It does not know whether they are signed, unsigned, a pointer, a float, or four characters. **The instruction decides how to read them.** The byte `0xff` is 255 if you treat it as unsigned and -1 if you treat it as signed.

So wherever the meaning matters, the instruction set has two versions:

Operation	Signed version	Unsigned version
Compare and branch (RISC-V)	<code>blt</code> , <code>bge</code>	<code>bltu</code> , <code>bgeu</code>
Load one byte into a 64-bit register	<code>lb</code> (copies the sign bit into the upper 56 bits)	<code>lbu</code> (fills the upper bits with 0)
Shift right	<code>sra</code> (copies the sign bit in from the left)	<code>srl</code> (shifts in zeros)
Divide	<code>div</code>	<code>divu</code>
Conditional jump (x86)	<code>jl</code> , <code>jg</code> (less, greater)	<code>jb</code> , <code>ja</code> (below, above)

This is the source of many bugs. If `i` is unsigned and you compare it with a signed branch, a huge value like 2^{63} looks negative and the loop does the wrong thing.

Sign extension and zero extension

When a small value moves into a bigger register, the extra upper bits must be filled with something.

- **Zero extension** fills them with 0. Correct for unsigned values.
- **Sign extension** fills them with copies of the top bit. Correct for signed values, because it keeps the number the same. The 8-bit pattern for -5 is `1111 1011`. Sign-extended to 16 bits it becomes `1111 1111 1111 1011`, which is still -5.

RISC-V `lb` sign-extends, `lbu` zero-extends. On RV64, `lw` sign-extends a 32-bit value to 64 bits, and `lwu` zero-extends it.

Overflow wraps around

Registers have a fixed width. If a result needs more bits, the extra bits are thrown away. In 8 bits, $255 + 1 = 256$ needs 9 bits (`1 0000 0000`), so the stored result is `0000 0000` = 0. In 64 bits, adding 1 to the largest signed value `0x7fffffffffffffff` gives `0x8000000000000000`, the most negative value. The CPU does not stop or complain. RISC-V does not even record that it happened. x86 sets a flag bit, which you can test if you care.

Shifts are multiplication and division by powers of 2

Shifting left by k bits multiplies by 2^k . `x << 3` is `x * 8`. Compilers use this everywhere, because a shift costs 1 cycle and is simpler than a multiply.

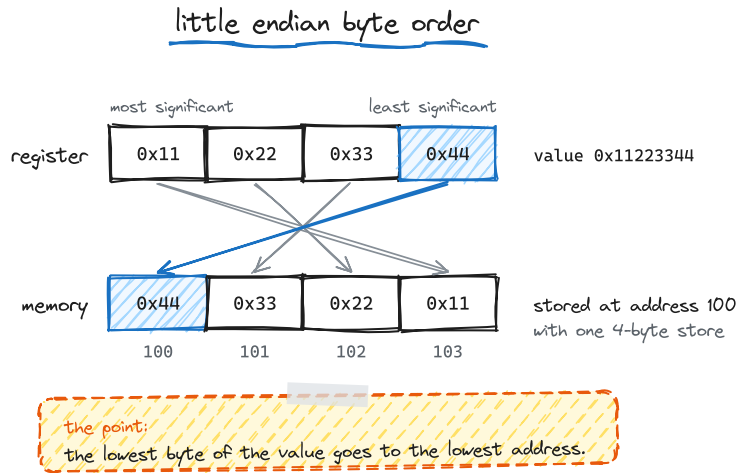
Shifting right by k bits divides by 2^k , but with a catch for negative numbers. An arithmetic right shift rounds toward negative infinity: `-7 >> 1` is -4. C's division rounds toward zero: `-7 / 2` is -3. So they disagree whenever a negative number is not an exact multiple of 2^k . You will see the compiler insert extra instructions to fix this when you divide a signed value by a constant (chapter 8). Dividing an unsigned value by 8 is just one shift.

Byte order: little endian

Memory is a row of bytes, and a 4-byte value occupies 4 of them. Which byte goes first? RISC-V, x86 and Arm (in normal mode) are all **little endian**: the lowest byte of the value goes at the lowest address.

Storing the 32-bit value `0x11223344` at address 100 puts these bytes in memory:

address:	100	101	102	103
byte:	0x44	0x33	0x22	0x11



This looks backwards when you dump memory in hex, and it will confuse you exactly once. The instruction encodings printed by tools in chapter 7 show bytes in memory order for this reason.

Floating point, briefly

Floats are bit patterns with three fields. A 32-bit float (`f32`) has 1 sign bit, 8 exponent bits and 23 fraction bits. The value is roughly $(1 + \text{fraction}) \times 2^{(\text{exponent} - 127)}$, with the sign bit choosing + or -. 1.0 is stored as `0x3f800000`.

how 1.0 is stored as an f32



1.0 as an f32 = `0x3f800000`

sign 0 means positive

value = $(1 + \text{fraction}) \times 2^{(\text{exponent} - 127)} = (1 + 0) \times 2^0 = 1.0$

The facts that matter for this book:

- **Floats are approximations.** 0.1 cannot be stored exactly. A float has 24 bits of precision, so between 2^{26} and 2^{27} (about 67 million to 134 million), the gap between neighboring floats is 8. So in f32, $100000000 + 1 = 100000000$.
- **Float addition is not associative.** $(1e8 + 1) - 1e8 = 0$ in f32, but $(1e8 - 1e8) + 1 = 1$. The order matters. **This is why a compiler will not reorder a float sum to make it faster unless you allow it** (with `-ffast-math` or by writing the reordering yourself). Chapters 10 and 11 show the speed cost of this with real numbers.
- ML uses smaller formats too. `f16` has 1 sign, 5 exponent and 10 fraction bits. `bfp16` has 1 sign, 8 exponent and 7 fraction bits: the same range as f32 with much less precision. GPU tensor cores (chapter 14) mostly multiply these small types and add into f32.

Exercises

1. Convert `0xb7` to binary and to decimal (as unsigned, and as a signed 8-bit value).
2. What is -1 as a 32-bit hex value? What is -2?
3. The byte `0x80` is loaded with `1b` into a 64-bit register. What hex value is in the register? And with `1bu`?
4. The 32-bit value `0xdeadbeef` is stored at address `0x1000` on a little endian machine. Which byte is at address `0x1001`?
5. Why does `(x >> 3)` give a different answer from `x / 8` when `x = -9`? Compute both.

Answers are in appendix A.

Chapter 2

What a CPU actually does

Memory

Memory is a long row of bytes. Each byte has a number, its **address**, starting at 0. On a 64-bit machine an address is a 64-bit number. A pointer in C is nothing more than an address stored in a variable.

Your program does not see physical RAM directly. The operating system gives each program its own **virtual** address space and the hardware translates every address to a physical one, in chunks of 4096 bytes called **pages**. For this book that detail mostly does not matter: you see a big row of bytes, and some addresses are valid (mapped) and some crash your program if touched.

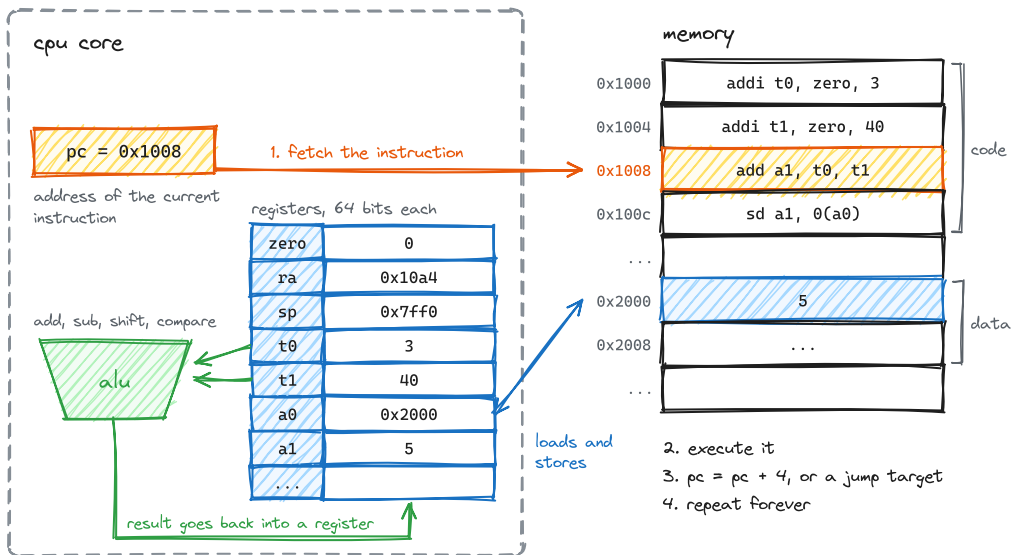
The CPU loop

Inside the CPU are a small number of storage slots called **registers**. On RISC-V there are 32 general registers, each 64 bits wide. Registers are where all computation happens. Memory is only read and written by special load and store instructions.

One more register is special: the **program counter**, **pc**. It holds the address of the instruction being run. The CPU repeats one loop forever:

1. read the instruction stored in memory at address pc
2. do what it says (add two registers, load from memory, ...)
3. set pc to the next instruction:
 - normally $pc + 4$ (risc-v instructions are 4 bytes)
 - for a jump or a taken branch, the target address instead

That is the entire model. There is no “if”, no “for”, no “function” in hardware. An if-statement is a branch instruction that sometimes changes pc. A loop is a branch that jumps backwards. A function call is a jump that also remembers where to come back to. Everything in the next five chapters is this loop plus details.

the whole machine, as the cpu sees it

Instructions are numbers in memory

An instruction is just a number. On RISC-V every instruction is exactly 32 bits. The instruction `add t0, t1, t2` (put `t1 + t2` into `t0`) is the number `0x007302b3`. Chapter 7 takes this number apart field by field. The important point now: code is stored in memory the same way as data. The CPU only knows which bytes are instructions because `pc` points at them.

An **assembler** turns the text `add t0, t1, t2` into that number. A **disassembler** turns the number back into text. Assembly language is a one-to-one text form of machine instructions, plus some conveniences (labels, names for registers, shortcut instructions that expand into one or two real ones).

Why registers exist

Registers are tiny (32 of them on RISC-V, 16 general ones on x86-64) because they must be extremely fast: the CPU can read several of them and write one in every cycle. Memory is huge and slow. Here are numbers measured on the machine that built this book (chapter 11 shows how):

Where the data is	Time to load one value
A register	Effectively 0, it is already there
L1 cache (small, closest to the core)	About 1.7 ns
Main memory, random address	About 170 ns

170 ns is roughly 500 clock cycles. In that time the CPU could have done hundreds of additions. This one number explains most of performance engineering: **the hard part is getting data to the registers, not computing on it.**

ISA and microarchitecture

The **ISA** (instruction set architecture) is the contract: which instructions exist, which registers exist, what each instruction does. RISC-V, x86-64 and Arm64 are ISAs. A program written for an ISA runs on any chip that implements it.

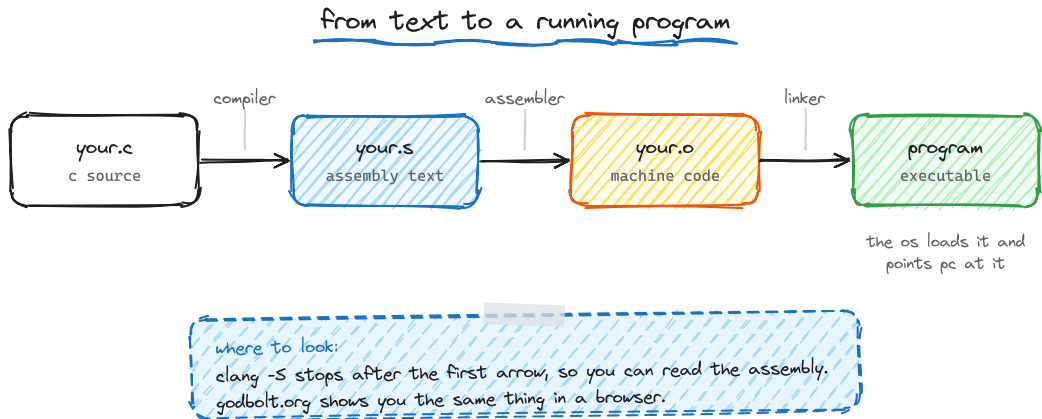
The **microarchitecture** is how one particular chip implements the ISA: how many instructions it works on at once, how big its caches are, how it guesses branches. Two chips with the same ISA can run the same program at very different speeds. Assembly tells you exactly what work the program asks for. The microarchitecture decides how long that work takes. You need both views, and part 4 is about the second one.

Load/store versus memory operands

RISC-V and Arm are **load/store** ISAs: arithmetic only works on registers. To add a value from memory you must first load it into a register with one instruction, then add with another. x86 lets many instructions take one operand directly from memory:

`add rax, qword ptr [rdi]` loads and adds in one instruction. Inside the chip, x86 still splits it into a load step and an add step. The difference is mostly in how the text looks, not in what the hardware does.

From text to running program



The compiler turns C into assembly text. The assembler turns text into machine code in an object file. The linker combines object files, fills in the final addresses of functions and data, and writes an executable. The operating system loads it into memory and sets pc to its first instruction. `clang -S` stops after the first step so you can read the assembly. godbolt.org shows you the same thing in a browser.

Part 2

RISC-V, the clean instruction set

Chapter 3

Registers and arithmetic

RISC-V was designed in 2010 at Berkeley, with no backward compatibility to carry. That makes it the best ISA to learn on: almost every instruction does one obvious thing. This book uses **RV64**, the 64-bit version, plus the **m** extension for multiply and divide. That combination is written `rv64im`.

The 32 registers

Registers are numbered `x0` to `x31`. You will almost never write those numbers. Instead you use names that say what each register is for by convention:

Name	Number	Used for	Survives a function call?
<code>zero</code>	X0	Always reads as 0. Writes to it are thrown away	(Constant)
<code>ra</code>	X1	Return address	No
<code>sp</code>	X2	Stack pointer	Yes
<code>gp</code> , <code>tp</code>	X3, x4	Global and thread pointers. Leave them alone	(Reserved)
<code>t0</code> .. <code>t6</code>	X5-7, x28-31	Temporaries, scratch values	No
<code>s0</code> .. <code>s11</code>	X8-9, x18-27	Saved values (<code>s0</code> is also <code>fp</code> , the frame pointer)	Yes
<code>a0</code> .. <code>a7</code>	X10-17	Function arguments. A0 (and a1) also hold the return value	No

The last column will matter a lot in chapter 6. For now, use **t** registers for scratch work and **a** registers for inputs and outputs.

`x0` being hard-wired to zero is a deliberate design trick. It lets one instruction do many jobs. “copy `t1` into `t0`” is just `add t0, t1, zero`. “throw this result away” is writing to `zero`.

The form of an instruction

Almost every RISC-V arithmetic instruction looks like this:

```
op  rd, rs1, rs2    # rd = rs1 op rs2
```

`rd` is the destination register, `rs1` and `rs2` are the sources. The destination always comes first. Everything after `#` is a comment.

Arithmetic and logic

```
add  t0, t1, t2      # t0 = t1 + t2
sub  t0, t1, t2      # t0 = t1 - t2
and  t0, t1, t2      # t0 = t1 & t2    (bit by bit)
or   t0, t1, t2      # t0 = t1 | t2
xor  t0, t1, t2      # t0 = t1 ^ t2
sll  t0, t1, t2      # t0 = t1 << t2    shift left
srl  t0, t1, t2      # t0 = t1 >> t2    shift right, fill with zeros (unsigned)
sra  t0, t1, t2      # t0 = t1 >> t2    shift right, fill with sign bit (signed)
slt  t0, t1, t2      # t0 = (t1 < t2) ? 1 : 0    signed compare
sltu t0, t1, t2      # t0 = (t1 < t2) ? 1 : 0    unsigned compare
mul  t0, t1, t2      # t0 = low 64 bits of t1 * t2
mulh t0, t1, t2      # t0 = high 64 bits of the 128-bit signed product
div  t0, t1, t2      # t0 = t1 / t2    signed, rounds toward zero
divu t0, t1, t2      # unsigned divide
rem  t0, t1, t2      # t0 = t1 % t2    signed remainder
```

Immediates: constants inside the instruction

Often one operand is a constant. Instructions ending in `i` take a constant, called an **immediate**, instead of `rs2`:

```
addi t0, t1, 5        # t0 = t1 + 5
addi t0, t1, -1       # t0 = t1 - 1    (there is no subi: add a negative number)
andi t0, t1, 0xff     # keep only the low 8 bits
slli t0, t1, 3        # t0 = t1 << 3 = t1 * 8
srai t0, t1, 1        # t0 = t1 >> 1, signed
slti t0, t1, 10       # t0 = (t1 < 10) ? 1 : 0
```

The immediate is stored inside the 32-bit instruction, and only 12 bits are available for it. A 12-bit signed number ranges from **-2048 to 2047**. This limit is real and you will hit it.

Loading big constants

To put a bigger constant in a register, RISC-V uses two instructions. `lui` (load upper immediate) puts a 20-bit value into bits 12 to 31 of a register. `addi` then adds the low 12 bits. You write `li t1, 5000` and the assembler does the splitting. This is the real output of the assembler for two constants:

<code>li t1, 5000</code>	becomes	<code>lui t1, 1</code>	# t1 = 1 << 12 = 4096
		<code>addiw t1, t1, 904</code>	# t1 = 4096 + 904 = 5000
 <code>li t0, 0x12345678</code>	 becomes	 <code>lui t0, 0x12345</code>	 # t0 = 0x12345000
		<code>addiw t0, t0, 0x678</code>	# t0 = 0x12345678

(`addiw` is the 32-bit version of `addi`, explained below.) a full 64-bit constant can take up to 8 instructions. Compilers often store such constants in memory and load them instead.

Pseudo-instructions

Some instructions you write are not real. The assembler turns them into real ones. You will see them constantly in compiler output, so learn what they really are:

You write	The real instruction	Meaning
<code>li rd, imm</code>	<code>addi rd, zero, imm</code> (or <code>lui + addi</code>)	Load a constant
<code>mv rd, rs</code>	<code>addi rd, rs, 0</code>	Copy a register
<code>not rd, rs</code>	<code>xori rd, rs, -1</code>	Flip every bit
<code>neg rd, rs</code>	<code>sub rd, zero, rs</code>	Negate
<code>nop</code>	<code>addi zero, zero, 0</code>	Do nothing
<code>seqz rd, rs</code>	<code>sltiu rd, rs, 1</code>	Rd = (rs == 0)
<code>snez rd, rs</code>	<code>sltu rd, zero, rs</code>	Rd = (rs != 0)
<code>j label</code>	<code>jal zero, label</code>	Jump
<code>ret</code>	<code>jalr zero, 0(ra)</code>	Return from function

`seqz` is worth a second look. “is rs equal to zero” is computed as “is rs, read as unsigned, less than 1”. The only unsigned value below 1 is 0. Tricks like this are why RISC-V needs so few real instructions.

32-bit operations on a 64-bit machine

C's `int` is 32 bits. RV64 has a `w` version of most arithmetic: `addw`, `subw`, `addiw`, `sllw`, `mulw`, `divw`. They compute using the low 32 bits only, then **sign-extend** the 32-bit result into all 64 bits. When you see `addw` in compiler output, the source code was doing `int` math.

A first example

Compute $(a + b) * (a - b)$ where `a` is in `a0` and `b` is in `a1`, leaving the result in `a0`:

```
add  t0, a0, a1    # t0 = a + b
sub  t1, a0, a1    # t1 = a - b
mul  a0, t0, t1    # a0 = (a + b) * (a - b)
```

Read it the way the CPU does: three instructions, three registers written, in order. Nothing else happens.

Exercises

1. Write instructions that compute `a0 = 3 * a0 + 7` without using `mul`. (Hint: $3x = x + 2x$, and $2x$ is a shift.)
2. `a0` holds a value. Write one instruction that sets `a0` to 1 if it is negative and 0 otherwise. (There are two good answers: one uses a compare, one uses a shift.)
3. What does `li t0, 2048` turn into? Why can't it be one `addi`?
4. What is in `t0` after `addi t0, zero, -1` followed by `srlt t0, t0, 60`?

Chapter 4

Memory: loads and stores

Registers hold a few values. Everything else lives in memory. RISC-V moves data between the two with exactly two kinds of instruction: **loads** copy bytes from memory into a register, **stores** copy bytes from a register into memory.

The instructions

```
ld    t0, 8(a0)    # t0 = the 8 bytes at address (a0 + 8)
sd    t0, 8(a0)    # write the 8 bytes of t0 to address (a0 + 8)
```

The form `offset(register)` means “the address is the value in the register plus the offset”. The offset is a 12-bit signed immediate, so -2048 to 2047. There is no other addressing mode. Any address calculation more complicated than `register + constant` must be done with separate arithmetic instructions first.

For each size there is a load and a store. Smaller loads come in signed and unsigned versions because the upper bits of the register must be filled (chapter 1):

Size	Load (sign-extend)	Load (zero-extend)	Store
1 byte	<code>lb</code>	<code>lbu</code>	<code>sb</code>
2 bytes	<code>lh</code>	<code>lhu</code>	<code>sh</code>
4 bytes	<code>lw</code>	<code>lwu</code>	<code>sw</code>
8 bytes	<code>ld</code>	(Not needed)	<code>sd</code>

Stores do not need two versions. A store of 1 byte just writes the low 8 bits of the register and ignores the rest.

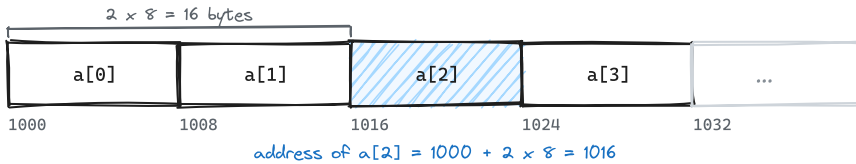
Arrays

An array is a run of same-sized elements stored back to back. If the array starts at address `base` and each element is `size` bytes, element `i` is at:

```
address of a[i] = base + i * size
```

For an array of `long` (8 bytes each) starting at address 1000:

an array is a start address plus arithmetic



the point:

each box is one 8-byte long, with the address of its first byte underneath.
`a[i]` costs a shift and an add before the load can even start.

So `a[2]` is at $1000 + 2 \times 8 = 1016$. In assembly, the multiply by 8 is a shift left by 3. Here is a complete function that sums an array. It was run in an emulator against random inputs:

`sum.s` · risc-v, run and checked in an emulator

```
# long sum(long *a, long n)
# a0 = address of a[0], a1 = n
.globl sum
sum:
    li    t0, 0           # t0 = s = 0
    li    t1, 0           # t1 = i = 0
loop:
    bge   t1, a1, done     # if i >= n, leave the loop
    slli  t2, t1, 3        # t2 = i * 8 (each long is 8 bytes)
    add   t2, a0, t2       # t2 = address of a[i]
    ld    t3, 0(t2)        # t3 = a[i]
    add   t0, t0, t3       # s = s + a[i]
    addi  t1, t1, 1        # i = i + 1
    j     loop
done:
    mv    a0, t0           # the return value goes in a0
    ret
```

The `.globl sum` line tells the assembler to make the label `sum` visible to other files, so C code can call it. Chapter 7 covers these directives.

Read the loop body slowly. The C expression `a[i]` became three instructions: `slli` to turn the index into a byte offset, `add` to make the address, `ld` to fetch. **Indexing is not free.** There is arithmetic hidden in every `[i]`.

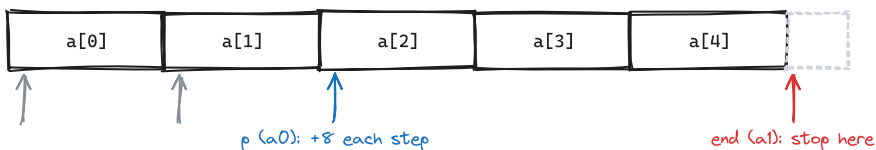
Walking a pointer instead

The multiply-and-add for every element can be avoided. Keep a pointer `p` that starts at `a` and moves forward 8 bytes each step, and stop when it reaches the end address:

`sum_ptr.s` · risc-v, run and checked in an emulator

```
# long sum_ptr(long *a, long n): walk a pointer instead of an index
.globl sum_ptr
sum_ptr:
    li    t0, 0           # s = 0
    slli  a1, a1, 3       # a1 = n * 8 = size of the array in bytes
    add   a1, a0, a1      # a1 = end = address just past the last element
1:
    beq   a0, a1, 2f      # if p == end, done
    ld    t1, 0(a0)       # t1 = *p
    add   t0, t0, t1       # s += *p
    addi  a0, a0, 8       # p = p + 1 element (8 bytes)
    j     1b
2:
    mv    a0, t0
    ret
```

sum_ptr: walk a pointer, stop at the end



The loop body went from 7 instructions to 5, and the index variable disappeared. This is not a trick humans invented for speed. It is exactly what compilers do, as chapter 8 will show with real compiler output. The labels `1:`, `1b` and `2f` are **local labels**: `1b` means “the nearest label 1 going backwards” and `2f` means “the nearest label 2 going forwards”. They save you from inventing names like `loop2_end`.

Bytes and strings

A C string is an array of 1-byte characters ending with a zero byte. Finding its length means loading one byte at a time until the zero appears. Note `lbu` (one byte, unsigned) and `addi 1` (one byte forward, not 8):

strlen.s · risc-v, run and checked in an emulator

```
# long my_strlen(char *s): count bytes until the zero byte
.globl my_strlen
my_strlen:
    mv    t0, a0        # t0 = p = s
1:
    lbu   t1, 0(t0)      # load ONE byte, zero-extended to 64 bits
    beqz  t1, 2f         # zero byte marks the end of a c string
    addi  t0, t0, 1      # p = p + 1 (one byte, not 8)
    j     1b
2:
    sub   a0, t0, a0     # length = p - s
    ret
```

The length is the end pointer minus the start pointer. Pointer subtraction gives a byte count, and here each element is one byte, so it is also the element count.

Structs

A struct is a block of memory where each field sits at a fixed **offset** from the start. The compiler decides the offsets. The program only ever sees `load from (pointer + offset)`.

point.s · risc-v, run and checked in an emulator

```
# struct point { long x; long y; };      x at offset 0, y at offset 8
# long dist2(struct point *p, struct point *q)
# returns (p->x - q->x)^2 + (p->y - q->y)^2
.globl dist2
dist2:
    ld    t0, 0(a0)      # p->x
    ld    t1, 0(a1)      # q->x
    sub   t0, t0, t1      # dx
    mul   t0, t0, t0      # dx * dx
    ld    t1, 8(a0)      # p->y (offset 8)
    ld    t2, 8(a1)      # q->y
    sub   t1, t1, t2      # dy
    mul   t1, t1, t1      # dy * dy
    add   a0, t0, t1
    ret
```

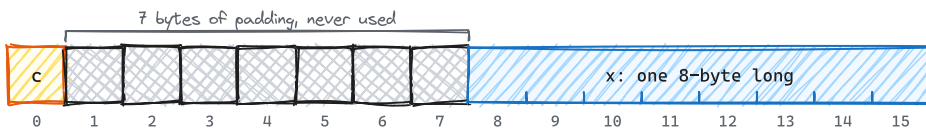
`p->x` is `ld t0, 0(a0)` and `p->y` is `ld t1, 8(a0)`. Field names do not exist in the machine. Only offsets do.

Alignment and padding

A value of size n is **naturally aligned** if its address is a multiple of n . An 8-byte `long` should sit at an address divisible by 8. Some CPUs trap on unaligned access, others handle it slowly. The C compiler adds **padding** bytes inside structs to keep every field aligned:

```
struct s {
    char c;      // offset 0, 1 byte
                // 7 bytes of padding
    long x;      // offset 8 (must be a multiple of 8)
};              // total size 16, not 9
```

padding inside a struct



the point:

struct { char c; long x; } is 16 bytes, not 9,
because x has to start at a multiple of 8.

Reorder fields from largest to smallest and padding usually disappears. This matters on GPUs too: chapter 14 shows a 16-byte load that only works because the data is 16-byte aligned.

Addresses of global data

For data stored at a fixed label in the program, `la t0, label` (load address) puts the label's address into a register. It is a pseudo-instruction that becomes `auipc` (add a 20-bit constant shifted up to pc) plus `addi`. You will see it in chapter 7.

Exercises

1. Write `long max(long *a, long n)` for $n \geq 1$. Start with `best = a[0]` and loop from $i = 1$.
2. Write `void rev(long *a, long n)` that reverses an array in place. Use two pointers, one at each end, that move toward each other. Careful with $n = 0$.
3. Write `long dot(long *a, long *b, long n)`, the dot product of two arrays.

4. `struct node { long val; struct node *next; }`. Write `long list_sum(struct node *p)` that adds up a linked list ending in a null pointer. What are the offsets of `val` and `next`?
5. In `sum_ptr`, what would go wrong if you replaced `beq a0, a1` with `blt a0, a1` and swapped the targets? When would it matter?

Chapter 5

Control flow

The CPU runs instructions in order, one after another, until an instruction changes pc. Every if, else, while, for, break and switch in every language becomes some combination of the instructions in this chapter.

Branches

A **branch** compares two registers and, if the condition is true, sets pc to a target label. If the condition is false, execution continues with the next instruction.

```
beq  rs1, rs2, label    # if rs1 == rs2, go to label
bne  rs1, rs2, label    # if rs1 != rs2
blt  rs1, rs2, label    # if rs1 <  rs2   (signed)
bge  rs1, rs2, label    # if rs1 >= rs2   (signed)
bltu rs1, rs2, label    # if rs1 <  rs2   (unsigned)
bgeu rs1, rs2, label    # if rs1 >= rs2   (unsigned)
```

Those six are the only real branch instructions. Everything else is a pseudo-instruction that swaps the operands or uses `zero` :

You write	Real instruction
<code>bgt a, b, L</code>	<code>blt b, a, L</code>
<code>ble a, b, L</code>	<code>bge b, a, L</code>
<code>beqz a, L</code>	<code>beq a, zero, L</code>
<code>bnez a, L</code>	<code>bne a, zero, L</code>
<code>blez a, L</code>	<code>bge zero, a, L</code>
<code>bgtz a, L</code>	<code>blt zero, a, L</code>

A jump with no condition is `j label`. The target of a branch is stored in the instruction as a distance from the current pc, not as a full address. That is why code can be loaded at any address and still work.

If and else

The pattern for an if-statement is: **test the opposite condition, and jump over the code that should not run.**

```
if (x < lo) return lo;
else if (x > hi) return hi;
else return x;
```

clamp.s · risc-v, run and checked in an emulator

```
# long clamp(long x, long lo, long hi)
# if (x < lo) return lo; else if (x > hi) return hi; else return x;
.globl clamp
clamp:
    bge a0, a1, 1f      # if x >= lo, go check the upper side
    mv a0, a1          # x < lo: return lo
    ret
1:
    bge a2, a0, 2f      # if hi >= x, x is in range
    mv a0, a2          # x > hi: return hi
2:
    ret
```

“if x < lo” became “if x >= lo, skip ahead”. You will see this inversion in nearly every compiled if-statement. Read compiled branches as “the condition under which we skip”.

Loops

A **while** loop is a test at the top and a jump back at the bottom. Here is the max function from the last chapter’s exercise. Notice the inner **if** (skip the update when best is already bigger) sitting inside the loop:

max.s · risc-v, run and checked in an emulator

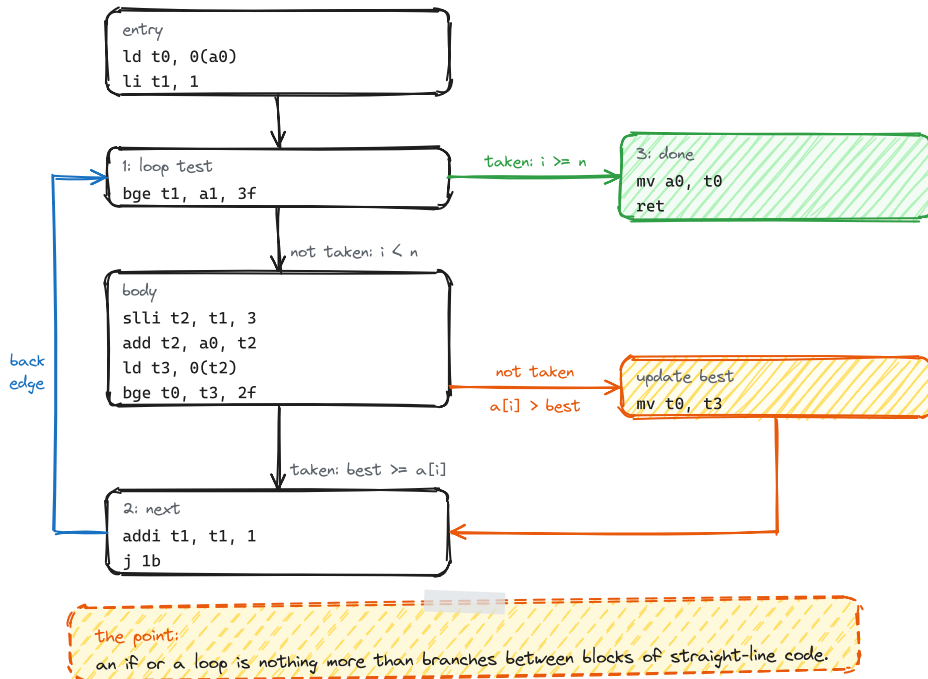
```
# long max(long *a, long n)  assumes n >= 1
.globl max
max:
    ld t0, 0(a0)          # best = a[0]
    li t1, 1              # i = 1
1:
    bge t1, a1, 3f         # i >= n ? done
    slli t2, t1, 3
    add t2, a0, t2
    ld t3, 0(t2)           # t3 = a[i]
    bge t0, t3, 2f         # if best >= a[i], skip the update
    mv t0, t3              # best = a[i]
2:
3:
```

```

addi t1, t1, 1
j 1b
3:
mv a0, t0
ret

```

max() as basic blocks and branches



This loop has **two** branch instructions per iteration that are always executed: the test at the top and the `j` at the bottom. Compilers usually rewrite loops so the test is at the bottom. Then one branch per iteration does both jobs: it tests and jumps back. This is called **loop rotation**. The cost is one extra test before the loop starts, to handle the case where the loop runs zero times. You will see exactly this shape in compiler output in chapter 8:

```

if (n <= 0) skip the whole loop    # test once, before the loop
loop:
  body
  if (not done) go to loop          # one branch per iteration

```

Reversing an array: two pointers

The reversal exercise from chapter 4 uses two pointers walking toward each other. The comparison is `bgeu` because addresses are unsigned numbers:

`rev.s` · risc-v, run and checked in an emulator

```
# void rev(long *a, long n): reverse in place with two pointers
.globl rev
rev:
    beqz a1, 2f          # empty array: nothing to do
    slli t0, a1, 3
    add t0, a0, t0
    addi t0, t0, -8      # t0 = hi = address of the last element
                        # a0 = lo = address of the first element
1:
    bgeu a0, t0, 2f      # stop when lo >= hi (addresses are unsigned)
    ld t1, 0(a0)         # t1 = *lo
    ld t2, 0(t0)         # t2 = *hi
    sd t2, 0(a0)         # *lo = old *hi
    sd t1, 0(t0)         # *hi = old *lo
    addi a0, a0, 8        # lo moves right
    addi t0, t0, -8       # hi moves left
    j 1b
2:
    ret
```

Branchless code

A branch that the CPU cannot predict is expensive, about 15 to 20 cycles each time it guesses wrong (chapter 11 explains why). Sometimes you can compute the same result with arithmetic and no branch at all. Counting how many elements equal `x`, the obvious way:

`ex_count.s` · risc-v, run and checked in an emulator

```
# long count_eq(long *a, long n, long x): how many elements equal x
.globl count_eq
count_eq:
    li t0, 0
    slli a1, a1, 3
    add a1, a0, a1
1:
    beq a0, a1, 3f
    ld t1, 0(a0)
    bne t1, a2, 2f
    addi t0, t0, 1
2:
    addi a0, a0, 8
    j 1b
3:
```

```
mv    a0, t0
ret
```

The branchless way: `xor` gives 0 exactly when the two values are equal, `seqz` turns that into 1 or 0, and you add it unconditionally:

`ex_count_branchless.s` · risc-v, run and checked in an emulator

```
# same as count_eq, but no branch inside the loop body
.globl count_eq2
count_eq2:
    li    t0, 0
    slli  a1, a1, 3
    add   a1, a0, a1
1:
    beq   a0, a1, 2f
    ld    t1, 0(a0)
    xor   t1, t1, a2      # t1 = 0 exactly when a[i] == x
    seqz  t1, t1          # t1 = (t1 == 0) ? 1 : 0
    add   t0, t0, t1
    addi  a0, a0, 8
    j     1b
2:
    mv    a0, t0
    ret
```

The loop still has its loop branch, which is easy to predict (it goes the same way every time except the last). The data-dependent branch is gone. Both versions were tested against the same inputs and give identical results.

Branchless is not always faster. If the branch is predictable (for example the data is mostly one value), the branchy version is fine and does less work. Chapter 11 measures the real costs.

Switch statements and jump tables

A `switch` on a small dense range of values often compiles into a **jump table**: an array of addresses in memory. The code checks the value is in range, loads `table[value]`, and jumps to that address with `jalr` (jump to the address in a register). One load and one indirect jump replace a long chain of comparisons. When you see a load followed by `jr` or `jalr` to a register, think “switch statement” or “function pointer”.

Exercises

1. Write `long gcd(long a, long b)` using euclid's algorithm: while b is not 0, set (a, b) to (b, a % b).
2. Write `long bsearch(long *a, long n, long x)`: binary search in a sorted array, returning the index of x or -1. Use a half-open range [lo, hi).
3. Write `void bubble(long *a, long n)` that sorts an array with bubble sort.
4. Rewrite the `max` loop with loop rotation: one branch at the bottom of the loop that both tests and jumps back.
5. In `clamp`, the test is `bge a0, a1`. What goes wrong if you used `bgeu` and x was -5 with `lo = 0`?

Chapter 6

Functions and the stack

A function call needs three things: jump to the function, pass it arguments, and come back to the right place afterwards with a result. The hardware gives you one instruction for the jump-and-remember part. Everything else is a set of rules that every compiler agrees to follow, called the **calling convention** or **ABI** (application binary interface). If you follow the same rules, your assembly can call C and C can call your assembly.

call and return

```
jal ra, label      # ra = pc + 4 (the next instruction), then jump to label
jalr zero, 0(ra)   # jump to the address in ra, discard the "return address"
```

`jal` is “jump and link”. It saves the address of the instruction after itself into `ra`, then jumps. The function runs and finishes with `ret`, which is the pseudo-instruction for `jalr zero, 0(ra)`: jump to whatever address is in `ra`. So `pc` returns to the instruction right after the call. You will usually write `call label`, a pseudo-instruction that the assembler and linker turn into `jal` or a two-instruction form that can reach further.

The problem with nested calls

There is only one `ra`. If function `f` calls function `g`, the `call g` overwrites `ra` with an address inside `f`. `f`’s own return address, the one that leads back to `f`’s caller, is destroyed. When `f` tries to `ret`, it jumps to the wrong place.

The same problem applies to any register `f` wants to keep: `g` is free to overwrite it. The solution is to save values in memory before the call and load them back after. The memory used for this is the stack.

The stack

The **stack** is a region of memory. The register `sp` holds the address of its top. The stack grows **downward**: to get space, you subtract from `sp`. To give space back, you add.

```
addi sp, sp, -16    # reserve 16 bytes: sp now points at them
sd ra, 8(sp)        # store ra into the upper 8 bytes
...
```

```
ld ra, 8(sp)    # load it back
addi sp, sp, 16 # release the 16 bytes
```

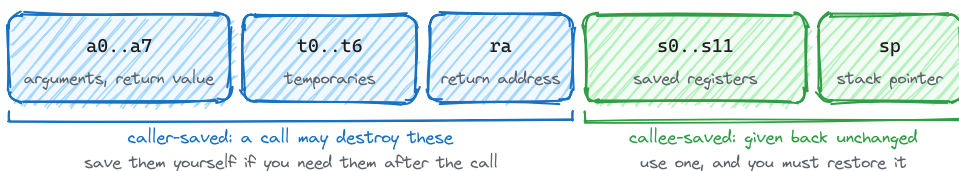
The RISC-V ABI requires `sp` to always be a multiple of 16. So frames are rounded up: a function that needs 3 slots of 8 bytes reserves 32 bytes, not 24.

The stack is just memory. Nothing stops you from corrupting it, and a function that forgets to restore `sp`, or stores past its reserved area, breaks every function above it.

The rules of the ABI

Register	Role	Who must preserve it
<code>a0 .. a7</code>	Arguments 1 to 8; <code>a0</code> (and <code>a1</code>) return the result	Caller (the callee may overwrite them)
<code>t0 .. t6</code>	Temporaries	Caller
<code>ra</code>	Return address	Caller
<code>s0 .. s11</code>	Saved registers	Callee: if you use one, restore it before returning
<code>sp</code>	Stack pointer	Callee: must be back to its original value on return

who saves what across a call (risc-v)



“caller-saved” means: if you are the caller and you need the value after the call, you must save it yourself, because the callee may destroy it. “callee-saved” means: the callee promises the value is unchanged when it returns. If the callee wants to use an `s` register, it saves the old value first and restores it at the end.

This split is a trade-off. A **leaf function** (one that calls nothing) can use all the `t` and `a` registers freely and needs no stack at all. A function that makes calls puts values it needs across those calls into `s` registers, and pays to save and restore those `s` registers once, at entry and exit, instead of around every call.

A leaf function

```
square:
    mul    a0, a0, a0
    ret
```

No stack, no saving. Argument in a0, result in a0. Most small functions look like this after the compiler is done with them.

A recursive function

Factorial: `fact(n) = n <= 1 ? 1 : n * fact(n - 1)`. This function calls itself, so it must save ra. It also needs n after the recursive call returns, so it keeps n in s0, which means it must save the caller's s0 first.

fact.s · risc-v, run and checked in an emulator

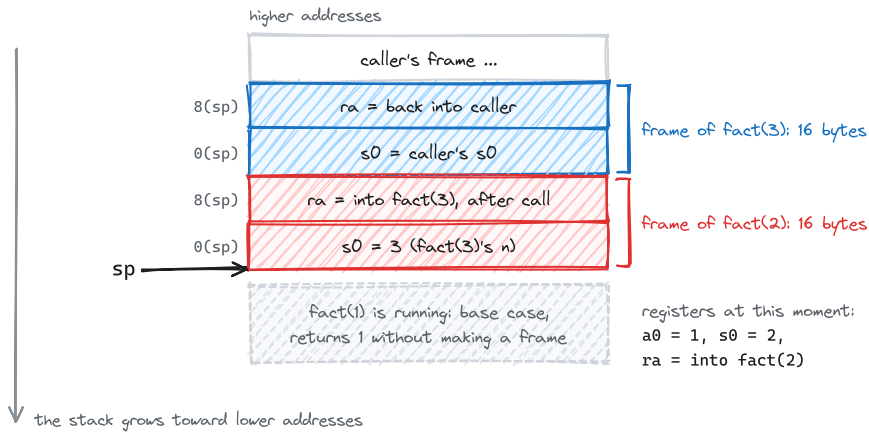
```
# long fact(long n): recursive.  fact(n) = n <= 1 ? 1 : n * fact(n - 1)
.globl fact
fact:
    li    t0, 1
    bgt   a0, t0, 1f      # n > 1: recursive case
    li    a0, 1           # base case: return 1
    ret

1:
    addi  sp, sp, -16      # make room for 2 slots on the stack
    sd    ra, 8(sp)       # save our return address (the call below overwrites ra)
    sd    s0, 0(sp)       # save s0 because we are about to use it
    mv    s0, a0          # s0 = n    (s registers survive calls)
    addi  a0, a0, -1       # argument = n - 1
    call  fact            # a0 = fact(n - 1)
    mul   a0, s0, a0       # a0 = n * fact(n - 1)
    ld    s0, 0(sp)       # restore s0 for our caller
    ld    ra, 8(sp)       # restore our return address
    addi  sp, sp, 16      # give the stack space back
    ret
```

The first part of the function that reserves stack space and saves registers is called the **prologue**. The matching part at the end is the **epilogue**. Every non-leaf function you read will have them.

Here is the stack while `fact(3)` is running deepest, inside `fact(1)`. Each call to fact made its own 16-byte frame. Addresses decrease going down:

the stack while fact(3) is at its deepest



When `fact(2)` stored its `s0`, the value it saved was 3: `fact(3)`'s `n`, which `fact(3)` had put in `s0`. So each frame holds the previous level's `n`. On the way back, each level multiplies by its own `n` (in `s0`), then restores the previous level's `s0`, and returns through its saved `ra`. The emulator checked this function for `n` from 0 to 14.

Two calls: fibonacci

`fib(n) = n < 2 ? n : fib(n - 1) + fib(n - 2)`. Now there are two recursive calls, and the result of the first must survive the second. That needs a second saved register:

`fib.s` · risc-v, run and checked in an emulator

```
# long fib(long n): fib(n) = n < 2 ? n : fib(n-1) + fib(n-2)
.globl fib
fib:
    li    t0, 2
    blt   a0, t0, 1f      # n < 2: return n (already in a0)
    addi   sp, sp, -32     # 3 slots used, keep sp a multiple of 16
    sd     ra, 24(sp)
    sd     s0, 16(sp)
    sd     s1, 8(sp)
    mv     s0, a0          # s0 = n
    addi   a0, s0, -1
    call   fib             # a0 = fib(n - 1)
    mv     s1, a0          # s1 = fib(n - 1), must survive the next call
    addi   a0, s0, -2
    call   fib             # a0 = fib(n - 2)
    add     a0, s1, a0      # result = fib(n-1) + fib(n-2)
    ld     s1, 8(sp)
    ld     s0, 16(sp)
    ld     ra, 24(sp)
```

```

    addi sp, sp, 32
1:
    ret

```

Three 8-byte slots are needed (ra, s0, s1), which is 24 bytes, rounded up to 32 to keep sp a multiple of 16.

A loop that calls a function

This is the most common real pattern: a loop whose body calls something. Every value the loop needs across the call (the pointer, the end address, the running total) goes into an s register, because the call is allowed to destroy all t and a registers:

sumsq.s · risc-v, run and checked in an emulator

```

# long square(long x)                a leaf function: calls nothing
# long sum_squares(long *a, long n)  calls square() in a loop
.globl square
square:
    mul  a0, a0, a0
    ret

.globl sum_squares
sum_squares:
    addi sp, sp, -32
    sd   ra, 24(sp)
    sd   s0, 16(sp)    # s0 = p
    sd   s1, 8(sp)     # s1 = end
    sd   s2, 0(sp)     # s2 = total
    mv   s0, a0
    slli a1, a1, 3
    add  s1, a0, a1
    li   s2, 0

1:
    beq  s0, s1, 2f
    ld   a0, 0(s0)      # argument for square = *p
    call square         # may destroy every a and t register, but not s0..s11
    add  s2, s2, a0
    addi s0, s0, 8
    j    1b

2:
    mv   a0, s2
    ld   s2, 0(sp)
    ld   s1, 8(sp)
    ld   s0, 16(sp)
    ld   ra, 24(sp)
    addi sp, sp, 32
    ret

```

When you read compiler output for a function with calls inside a loop, the first thing to identify is which s register holds which loop variable. After that the loop reads easily.

More than eight arguments, and structs

Arguments 9 and beyond are passed on the stack, at `0(sp)`, `8(sp)`, and so on, at the moment of the call. Small structs (up to 16 bytes on RV64) are passed in one or two a registers. Bigger structs are passed as a pointer to a copy. Return values bigger than two registers are written into memory that the caller provides, with its address passed as a hidden first argument.

The frame pointer

Unoptimized code (`-O0`) sets up `s0` as a **frame pointer**: it points at the top of the current frame, and every local variable is accessed at a fixed negative offset from it, like `-24(s0)`. This makes debugging easy and code slow. Chapter 8 shows a real `-O0` listing. Optimized code usually drops the frame pointer and keeps locals in registers.

Exercises

1. Why does `square` not need to save `ra`, but `sum_squares` does?
2. In `fib`, could you keep `n` in `t0` instead of `s0`? Explain exactly what breaks.
3. Write `long sum_rec(long *a, long n)` recursively: 0 if `n == 0`, else `a[0] + sum_rec(a + 1, n - 1)`. Draw its stack for `n = 2`.
4. A function needs to save `ra`, `s0`, `s1`, `s2` and `s3`. How many bytes does it reserve, and why?
5. Some functions end with a call followed immediately by `ret`. A compiler can replace that pair with a plain jump to the callee (a **tail call**). Why is that correct, and what happens to the stack frame?

Chapter 7

Whole programs: directives, syscalls, linking, encoding

So far every example was a function called from somewhere else. This chapter covers what a complete assembly file contains, how a program talks to the operating system, how files are combined, and how one instruction is encoded as bits.

Directives

Lines starting with `.` are **directives**. They are instructions to the assembler, not to the CPU. They produce no machine instruction themselves.

Directive	Meaning
<code>.text</code>	What follows is code
<code>.data</code>	What follows is writable data
<code>.rodata</code>	Read-only data (string constants, tables)
<code>.bss</code>	Zero-filled data that takes no space in the file
<code>.globl name</code>	Make the label visible to the linker, so other files can use it
<code>.byte 1, 2, .half, .word, .dword</code>	Emit 1, 2, 4 or 8-byte values
<code>.ascii "hi" / .asciz "hi"</code>	Emit the bytes of a string (<code>.asciz</code> adds a zero byte)
<code>.align 3</code>	Pad with zeros until the address is a multiple of $2^3 = 8$
<code>.set name, value</code>	Define a constant for the assembler

A **label** like `msg:` or `loop:` is just a name for the address where the next byte will go. Code labels and data labels are the same kind of thing.

A complete program

This program prints one line and exits. It has no C, no library, nothing. It was run in the emulator, which captured exactly the bytes written:

hello.s · risc-v, run and checked in an emulator

```
# a whole program: print a message, then exit with status 0
# linux on risc-v: syscall number in a7, arguments in a0..a5, then ecall
.section .rodata
msg:
.ascii "hello from risc-v\n"
.set len, . - msg      # len = current address minus start of msg

.text
.globl _start
_start:
li    a0, 1            # fd 1 = standard output
la    a1, msg          # a1 = address of the bytes
li    a2, len          # a2 = number of bytes
li    a7, 64           # syscall 64 = write
ecall
li    a0, 0            # exit status
li    a7, 93           # syscall 93 = exit
ecall
```

`. - msg` means “the current address minus the address of msg”, which is the number of bytes in the string. The assembler computes it.

System calls

A program cannot write to the screen or a file by itself. It asks the operating system with a **system call**. On Linux on RISC-V, the rules are: put the syscall number in `a7`, the arguments in `a0` to `a5`, then run `ecall`. The kernel takes over, does the work, puts the result in `a0`, and returns to the instruction after `ecall`.

Syscall	Number on RISC-V Linux	Arguments
write	64	A0 = file descriptor, a1 = address of bytes, a2 = count
exit	93	A0 = exit status
read	63	A0 = fd, a1 = buffer address, a2 = max count

The numbers are different on x86-64 Linux (write is 1, exit is 60, chapter 9). Every function in the C library that touches the outside world (`printf`, `fopen`, `malloc` for big blocks) ends in system calls like these.

Building and running it yourself

On an Ubuntu x86 machine, after `sudo apt install gcc-riscv64-linux-gnu qemu-user`:

```
riscv64-linux-gnu-gcc -nostdlib -static hello.s -o hello
qemu-riscv64 ./hello
```

`-nostdlib` means “do not link the C library or its startup code”. The program starts at `_start`, the default entry label. `qemu-riscv64` runs a RISC-V Linux program on your x86 machine by translating its instructions.

To call your assembly functions from C (the easiest way to test them):

```
riscv64-linux-gnu-gcc -static test.c sum.s -o test
qemu-riscv64 ./test
```

Where `test.c` declares `long sum(long *a, long n);` and calls it.

Object files, symbols and relocations

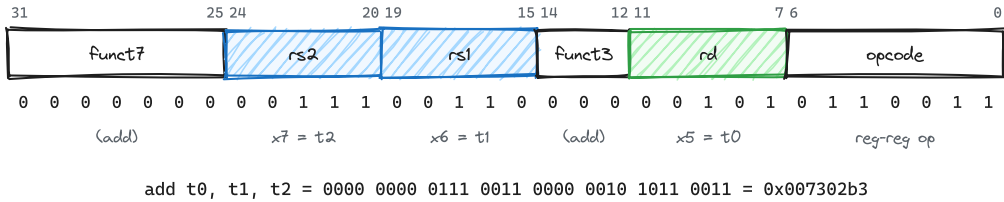
The assembler cannot always finish its job. In `fact.s`, `call fact` jumps to a label in the same file, but a call to a function in another file has an unknown target. So the assembler writes a placeholder instruction and a **relocation**: a note saying “when the address of this symbol is known, patch these bits here”. The linker collects all object files, decides where everything goes in memory, and applies every relocation.

The **symbol table** lists every label marked `.globl`, so the linker can match a call in one file to a definition in another. `nm file.o` prints it. `objdump -d file.o` disassembles the code. `objdump -r file.o` shows the relocations still waiting to be filled in.

How an instruction is encoded

Every RISC-V instruction is 32 bits, split into fields. Arithmetic between registers uses the **r-type** layout:

add t0, t1, t2 as 32 bits



The bits above are `add t0, t1, t2`. Read them left to right and group by 4: `0000 0000 0111 0011 0000 0010 1011 0011` = `0x007302b3`. The assembler printed exactly this (as bytes in memory order, little endian: `b3 02 73 00`).

The `opcode` field says which family the instruction belongs to. `funct3` and `funct7` pick the exact operation inside the family: `sub` is the same as `add` except `funct7` is `0100000`. Each register field is 5 bits, because $2^5 = 32$ registers.

An instruction with an immediate uses the **i-type** layout: the 12 bits that held `funct7` and `rs2` hold the immediate instead. That is where the 12-bit limit on immediates comes from: it is a direct consequence of fitting everything into 32 bits. Loads use i-type (`ld t3, 8(a0)` stores the 8 in those 12 bits). Branches and stores split their immediate across the fields in a scrambled-looking order, chosen so that the hardware can keep register fields in the same position in every format. That keeps the decoding circuit small and fast.

x86 instructions, by contrast, are anywhere from 1 to 15 bytes long, which makes decoding much harder for the chip. Chapter 9 shows examples.

Exercises

1. Change `hello.s` to print two different lines with two `write` calls.
2. Write a program that exits with status 42. Check it with `echo $?` after running it.
3. `sub t0, t1, t2` differs from `add t0, t1, t2` only in `funct7`. What is its 32-bit encoding in hex?
4. What is the largest forward distance a `jal` can jump, given that its immediate is 20 bits and counts in units of 2 bytes?

Chapter 8

Reading compiler output

This is the skill you will use most. Almost nobody writes large programs in assembly. People read what the compiler produced to find out why code is slow, whether an optimization happened, or what a crash means. This chapter teaches you to recognize what the compiler did to your code.

The workflow

1. Put a small function into godbolt.org (or run `clang -O2 -S file.c -o -`).
2. Pick the target and flags you care about. For RISC-V: `--target=riscv64 -march=rv64im -O2`.
3. Find the loop. It is the label that a branch near the bottom jumps back to.
4. For each register in the loop, write down which source variable it holds.
5. Count the instructions in the loop body. That is the work per iteration.

The same function, unoptimized

This is the `sum` function from chapter 4, compiled by clang for RISC-V with `-O0` (no optimization). It is real compiler output:

```
long sum(long *a, long n) {
    long s = 0;
    for (long i = 0; i < n; i++) s += a[i];
    return s;
}
```

compiler output: clang, risc-v rv64im

```
sum:
    addi    sp, sp, -48
    sd      ra, 40(sp)
    sd      s0, 32(sp)
    addi    s0, sp, 48
    sd      a0, -24(s0)
    sd      a1, -32(s0)
    li      a0, 0
    sd      a0, -40(s0)
    sd      a0, -48(s0)
    j       .LBB0_1
.LBB0_1:
```

```

        ld      a0, -48(s0)
        ld      a1, -32(s0)
        bge     a0, a1, .LBB0_4
        j       .LBB0_2
.LBB0_2:
        ld      a0, -24(s0)
        ld      a1, -48(s0)
        slli    a1, a1, 3
        add     a0, a0, a1
        ld      a1, 0(a0)
        ld      a0, -40(s0)
        add     a0, a0, a1
        sd      a0, -40(s0)
        j       .LBB0_3
.LBB0_3:
        ld      a0, -48(s0)
        addi    a0, a0, 1
        sd      a0, -48(s0)
        j       .LBB0_1
.LBB0_4:
        ld      a0, -40(s0)
        ld      ra, 40(sp)
        ld      s0, 32(sp)
        addi    sp, sp, 48
        ret

```

At `-O0` the compiler gives every variable a home on the stack and loads and stores it around every single use. `s0` is the frame pointer. The four variables live at fixed offsets: `a` at `-24(s0)`, `n` at `-32(s0)`, `s` at `-40(s0)`, `i` at `-48(s0)`. One iteration of the loop does 7 loads (only one of them is the array element) and 2 stores, just to update two numbers.

Nobody ships `-O0` code. It exists so a debugger can show you every variable at every line. Never judge performance from it.

The same function, optimized

With `-O2`:

compiler output: clang, risc-v rv64im

```

sum:
        li      a2, 0
        blez    a1, .LBB0_3
        slli    a1, a1, 3
        add     a1, a0, a1
.LBB0_2:
        ld      a3, 0(a0)
        addi    a0, a0, 8
        add     a2, a3, a2
        bne     a0, a1, .LBB0_2

```

```
.LBB0_3:
    mv      a0, a2
    ret
```

8 instructions in total, 4 in the loop. Compare with the hand-written versions in chapter 4: the compiler arrived at the pointer-walking version on its own. Reading this, map each register:

- `a0` = the pointer, moving forward by 8 each iteration.
- `a1` = the end address: `slli` and `add` turned `n` into `a + n*8`.
- `a2` = `s`, the running sum.
- `a3` = the loaded element.

The `blez a1` before the loop is the loop rotation from chapter 5: test once whether `n` is at most 0, and if so skip straight to returning 0. Then the loop tests at the bottom with `bne`.

Optimizations to recognize

These show up in almost every function you read. Learn their shapes.

Name	What it does	What you see
Strength reduction	Replaces expensive ops with cheap ones	<code>i * 8</code> becomes a shift, or a pointer that adds 8 each step
Induction variable elimination	Removes a loop counter that can be derived from another	No <code>i</code> at all; the loop stops when the pointer hits the end
Loop rotation	Moves the loop test to the bottom	One test before the loop, one branch at the end of the body
Constant folding	Computes constant expressions at compile time	<code>60 * 60 * 24</code> appears as <code>86400</code>
Inlining	Copies a small function's body into its caller	A <code>call</code> you expected is missing; its work appears in place
Dead code elimination	Deletes work whose result is never used	A computation you wrote is simply gone
Tail call	<code>call f; ret</code> becomes <code>j f</code>	A function ends in a jump to another function
Unrolling	Repeats the loop body <code>k</code> times per iteration	4 or 8 copies of the body, pointer steps of 32 or 64
Vectorization	Uses SIMD registers to do several elements at once	Wide registers and instructions (chapter 10)

“the compiler deleted my benchmark” is the most common beginner surprise, and dead code elimination is why. If you compute something and never use the result, the loop is removed. Benchmarks in this book write results into a `volatile` variable to prevent that.

Multiplying and dividing by constants

Compilers avoid multiply and divide instructions when the constant is known. This is real x86-64 output (Intel syntax, which chapter 9 explains in full; `rdi` is the argument and `rax` is the result):

```
long mul5add3(long x) { return x * 5 + 3; }
long div8(long x)     { return x / 8; }
unsigned long udiv8(unsigned long x) { return x / 8; }
long div7(long x)     { return x / 7; }
```

compiler output: clang, x86-64, intel syntax

```
mul5add3:
    lea    rax, [rdi + 4*rdi]
    add    rax, 3
    ret

div8:
    lea    rax, [rdi + 7]
    test   rdi, rdi
    cmovns rax, rdi
    sar    rax, 3
    ret

udiv8:
    mov    rax, rdi
    shr    rax, 3
    ret

div7:
    mov    rax, rdi
    movabs rcx, 5270498306774157605
    imul   rcx
    mov    rax, rdx
    shr    rax, 63
    sar    rdx
    add    rax, rdx
    ret
```

Line by line:

- `mul5add3`: `lea rax, [rdi + 4*rdi]` computes $x + 4x = 5x$ using the address-calculation hardware, without touching memory. Then `add 3`. No multiply instruction.
- `udiv8`: unsigned division by 8 is one right shift by 3.
- `div8`: signed division must round toward zero, but a shift rounds toward negative infinity (chapter 1). The fix: if x is negative, add 7 first. `lea rax, [rdi + 7]` computes $x + 7$, `test` and `cmovns` pick x instead when x is not negative, then `sar` shifts. No branch.
- `div7`: division by 7 cannot be done with shifts. Instead the compiler multiplies by a “magic number” close to $2^{65} / 7$ and keeps the top bits of the 128-bit product. `imul rcx` with one operand multiplies `rax` by `rcx` into the 128-bit pair `rdx:rax`, so `rdx` holds the top 64 bits, which is $x \times m / 2^{64}$. `sar rdx` shifts once more (total: divide by 2^{65}). The `shr rax, 63` and `add` add 1 when x is negative, to round toward zero. A division instruction takes 10 to 40+ cycles; this sequence takes a handful.

When you see a giant constant and a multiply where your code had a division, this is what it is.

Bounds checks in Rust

Rust checks every slice index at run time. Here is real rustc output (`-C opt-level=3`, x86-64) for a loop that indexes with values coming from another slice. The panic path's long internal name has been shortened:

```
pub fn get_sum(a: &[i64], idx: &[usize]) -> i64 {
    let mut s = 0;
    for &i in idx { s += a[i]; }
    s
}
```

compiler output: rustc -C opt-level=3, x86-64

```
get_sum:
    test    rcx, rcx
    je      .LBB0_1
    shl     rcx, 3
    xor     eax, eax
    xor     r9d, r9d
.LBB0_3:
    mov     r8, qword ptr [rdx + r9]
    cmp     r8, rsi
    jae     .LBB0_6
    add     rax, qword ptr [rdi + 8*r8]
    add     r9, 8
    cmp     rcx, r9
    jne     .LBB0_3
    ret
.LBB0_1:
    xor     eax, eax
    ret
.LBB0_6:
    push    rax
    lea     rdx, [rip + .Lpanic_location]
    mov     rdi, r8
    call    qword ptr [rip + core::panicking::panic_bounds_check@GOTPCREL]
```

A slice is passed as two registers: the pointer and the length. So `a` arrives as `rdi` (pointer) and `rsi` (length), and `idx` as `rdx` and `rcx`. Inside the loop, `cmp r8, rsi` / `jae .LBB0_6` is the bounds check: if the index is above or equal to the length (unsigned compare), jump to the panic. It costs a compare and a branch per element, and the branch is always predicted correctly, so it is cheap here. The real cost is elsewhere: a loop containing a possible panic usually cannot be vectorized, because the vector version would have to know which exact element panicked.

When the compiler can prove the index is in range (for example when you iterate with `for &x in a`), the check disappears. Writing Rust loops with iterators instead of indices is how you get those checks removed.

Exercises

1. Compile the `max` function from chapter 4 in C with `-O2` for RISC-V on godbolt. Compare it with your hand-written version. Which registers hold which variables?
2. Compile `long f(long x) { return x * 9; }` and `x * 10` for x86-64. What instructions appear instead of `imul`?
3. Compile `unsigned div10(unsigned x) { return x / 10; }`. Find the magic number. Compute `magic × 1000` by hand (or in Python) and check the top bits give 100.
4. Write a C function where the result of a loop is never used. Compile it with `-O2`. What remains?

Part 3

x86-64 and SIMD

Chapter 9

x86-64 for someone who knows RISC-V

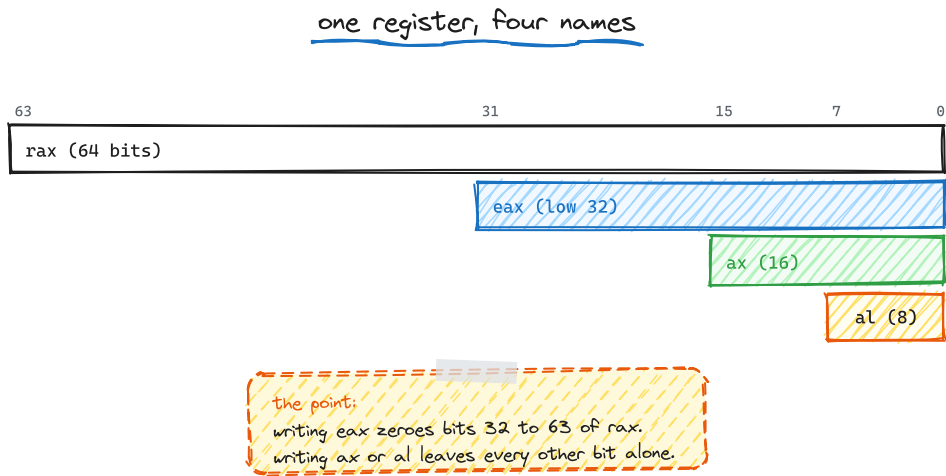
x86-64 is what runs on almost every server, desktop and Windows laptop. Its history goes back to the Intel 8086 from 1978, and it carries that history in its register names, its encoding, and some odd rules. But the ideas are the ones you already know from RISC-V: registers, loads, stores, branches, a stack, a calling convention. This chapter maps each idea across.

The registers

There are 16 general registers of 64 bits. The first eight have historical names, the last eight are just numbered:

64-bit	Low 32 bits	Low 16	Low 8	Historical role
rax	eax	ax	al	Accumulator; return value
rbx	ebx	bx	bl	Base
rcx	ecx	cx	cl	Count; shift counts go in cl
rdx	edx	dx	dl	Data; top half of wide multiply and divide
rsi	esi	si	sil	Source index
rdi	edi	di	dil	Destination index
rbp	ebp	bp	bpl	Base pointer (frame pointer)
rsp	esp	sp	spl	Stack pointer
r8 .. r15	r8d .. r15d	r8w ..	r8b ..	(Added in 2003)

The smaller names are not separate registers. `eax` is the low 32 bits of `rax`. One rule here prevents real bugs: **writing a 32-bit register zeroes the upper 32 bits**, but writing an 8 or 16-bit register leaves the upper bits unchanged. That is why compilers write `xor eax, eax` to zero all of `rax`: it is shorter to encode than `xor rax, rax` and does the same thing.



There is no zero register like RISC-V's `x0`. `pc` is called `rip` (instruction pointer). There is also `rflags`, a register of status bits set by arithmetic, explained below.

Two syntaxes

x86 assembly is written in two different text formats for the same instructions. This is pure notation, the machine code is identical. **Intel syntax** (used by Intel's manuals, Windows tools, and this book) puts the destination first like RISC-V. **At&t syntax** (the default for `gcc`, `objdump` and `gdb` on Linux) puts it last and adds symbols:

	Intel	At&t
Add <code>rbx</code> into <code>rax</code>	<code>add rax, rbx</code>	<code>addq %rbx, %rax</code>
Load from memory	<code>mov rax, qword ptr [rdi + 8*rcx]</code>	<code>movq (%rdi,%rcx,8), %rax</code>
Constant	<code>mov eax, 1</code>	<code>movl \$1, %eax</code>

To get Intel syntax: `clang -S -masm=intel`, `objdump -d -M intel`, and in `gdb` `set disassembly-flavor intel`. On `godbolt` it is the default.

Two operands, not three

Most x86 arithmetic has two operands, and the first is both a source and the destination:

```
add rax, rbx      # rax = rax + rbx      (risc-v would need: add a0, a0, a1)
sub rax, 5        # rax = rax - 5
imul rax, rcx     # rax = rax * rcx
mov rax, rbx      # rax = rbx           (copy)
```

So to compute `c = a + b` without destroying `a`, the compiler first copies: `mov rcx, rax` then `add rcx, rbx`, or uses `lea` (below).

Memory operands

The big difference from RISC-V: most instructions can read or write **one operand directly in memory**. The address can be:

```
[base + index*scale + displacement]      scale is 1, 2, 4 or 8
```

So `add rax, qword ptr [rdi + 8*rcx]` loads `a[i]` (with `rdi = a`, `rcx = i`) and adds it to `rax`, in one instruction. The `qword ptr` says the memory operand is 8 bytes. This addressing form covers array indexing directly, which is why x86 loops often have fewer instructions than RISC-V loops. Inside the chip, the instruction is still split into a load and an add.

Lea: arithmetic with the address hardware

`lea` (load effective address) computes the address expression and puts the address itself in the register. It does not touch memory. Compilers use it as a free three-operand add and small multiply:

```
lea rax, [rdi + 4*rdi]    # rax = 5 * rdi
lea rax, [rdi + rsi]      # rax = rdi + rsi, without destroying either
lea rsi, [rip + msg]      # rsi = address of label msg (position independent)
```

Flags and conditional jumps

RISC-V branches compare two registers directly. x86 splits this into two steps. Arithmetic instructions and `cmp` set bits in `rflags`. A conditional jump then reads those bits.

```

cmp rcx, rsi    # compute rcx - rsi, set flags, throw the result away
jge done       # jump if the result said rcx >= rsi (signed)

```

`test rsi, rsi` computes `rsi & rsi`, sets flags, and discards the result. It is the standard way to check if a register is zero or negative.

Jump	Meaning after <code>cmp a, b</code>	Signed or unsigned
<code>je / jne</code>	$A == b / a != b$	Either
<code>jl / jge</code>	$A < b / a > = b$	Signed (less, greater)
<code>jg / jle</code>	$A > b / a < = b$	Signed
<code>jb / jae</code>	$A < b / a > = b$	Unsigned (below, above)
<code>ja / jbe</code>	$A > b / a < = b$	Unsigned
<code>js / jns</code>	Result negative / not negative	Sign bit

The Rust bounds check in chapter 8 used `jae`: an unsigned compare, so a “negative” index looks huge and also fails the check. One compare covers both cases.

Flags also drive `cmovcc` (conditional move, no branch) and `setcc` (write 0 or 1 into a byte register).

The calling convention (system v, Linux and Mac)

Role	Registers
Arguments 1 to 6	<code>rdi</code> , <code>rsi</code> , <code>rdx</code> , <code>rcx</code> , <code>r8</code> , <code>r9</code>
Float arguments	<code>xmm0</code> to <code>xmm7</code>
Return value	<code>rax</code> (and <code>rdx</code>); floats in <code>xmm0</code>
Callee-saved	<code>rbx</code> , <code>rbp</code> , <code>r12</code> , <code>r13</code> , <code>r14</code> , <code>r15</code>
Caller-saved	Everything else

Windows uses a different convention (`rcx`, `rdx`, `r8`, `r9` for the first four arguments). If your assembly must run on both, you need two versions.

`call f` pushes the return address onto the stack (subtract 8 from `rsp`, store `rip` there) and jumps. `ret` pops it back into `rip`. So unlike RISC-V, the return address lives on the stack, not in a register. `push rbx` stores a register on the stack and moves `rsp` down by 8. `pop rbx` does the reverse.

One more rule: **rsp must be a multiple of 16 at every call**. Since `call` pushes 8 bytes, a function starts with `rsp` ending in 8, and must move `rsp` by an odd multiple of 8 before it calls anything.

The examples again

The array sum, hand-written in Intel syntax and tested natively against C:

sum.s · x86-64, run and checked natively

```
# long sum(long *a, long n)    intel syntax, linux / system v abi
# rdi = a, rsi = n, return value in rax
.intel_syntax noprefix
.globl sum
sum:
    xor    eax, eax            # rax = s = 0 (writing eax also zeroes the top half)
    xor    ecx, ecx            # rcx = i = 0
.Lloop:
    cmp    rcx, rsi            # compare i with n, sets flags
    jge    .Ldone              # jump if i >= n (reads the flags)
    add    rax, qword ptr [rdi + rcx*8] # s += a[i], load and add in one instruction
    inc    rcx                  # i++
    jmp    .Lloop
.Ldone:
    ret
```

And what clang produces with `-O2` (vectorization turned off so the scalar loop is visible):

compiler output: clang, x86-64, intel syntax

```
sum:
    test    rsi, rsi
    jle     .LBB0_1
    xor     ecx, ecx
    xor     eax, eax
.LBB0_4:
    add     rax, qword ptr [rdi + 8*rcx]
    inc     rcx
    cmp     rsi, rcx
    jne     .LBB0_4
    ret
.LBB0_1:
    xor     eax, eax
    ret
```

The loop is 4 instructions: one add-from-memory, one increment, one compare, one branch. Note the compiler compares `rsi` with `rcx` at the bottom (loop rotation again) and checks `test rsi, rsi / jle` before the loop.

Max, using a conditional move instead of a branch:

max.s · x86-64, run and checked natively

```
# long max(long *a, long n), n >= 1, no branch for the update: cmov
.intel_syntax noprefix
.globl max

max:
    mov    rax, qword ptr [rdi]    # best = a[0]
    mov    ecx, 1                  # i = 1
.Lloop:
    cmp    rcx, rsi
    jge    .Ldone
    mov    rdx, qword ptr [rdi + rcx*8]    # rdx = a[i]
    cmp    rdx, rax                # flags = compare a[i] with best
    cmovg  rax, rdx                # if a[i] > best: best = a[i] (no jump)
    inc    rcx
    jmp    .Lloop
.Ldone:
    ret
```

Recursive factorial, using `push` and `pop` to save the callee-saved register `rbx`:

fact.s · x86-64, run and checked natively

```
# long fact(long n), recursive, shows push/pop and the call stack
.intel_syntax noprefix
.globl fact

fact:
    cmp    rdi, 1
    jg     .Lrec
    mov    eax, 1                # base case
    ret
.Lrec:
    push   rbx                  # rbx is callee-saved: save the caller's value
    mov    rbx, rdi              # rbx = n, survives the call
    lea    rdi, [rdi - 1]        # argument = n - 1
    call   fact                  # pushes the return address, jumps
    imul   rax, rbx              # rax = fact(n - 1) * n
    pop    rbx                  # restore the caller's rbx
    ret                          # pops the return address into rip
```

Check the alignment rule here: on entry `rsp` ends in 8. `push rbx` moves it by 8, so it is a multiple of 16 at `call fact`. This one push both saves `rbx` and fixes alignment. You will see compilers push a register they do not even need just to align the stack.

A whole program, with Linux x86-64 syscall numbers (write = 1, exit = 60) and the `syscall` instruction:

hello.s · x86-64, run and checked natively

```
# whole program for linux x86-64: write, then exit
# syscall number in rax, arguments in rdi, rsi, rdx, then the syscall instruction
.intel_syntax noprefix
.section .rodata
msg:
.ascii "hello from x86-64\n"
.set len, . - msg
.text
.globl _start
_start:
mov  eax, 1           # syscall 1 = write
mov  edi, 1           # fd 1 = stdout
lea  rsi, [rip + msg] # address of msg, relative to this instruction
mov  edx, len
syscall
mov  eax, 60          # syscall 60 = exit
xor  edi, edi         # status 0
syscall
```

Variable-length encoding

x86 instructions are 1 to 15 bytes long. Real encodings from the assembler:

ret	c3	1 byte
add rax, rbx	48 01 d8	3 bytes
add rax, qword ptr [rdi + 8*rcx]	48 03 04 cf	4 bytes
mov rax, 0x123456789abcdef0	48 b8 f0 de bc 9a 78 56 34 12	10 bytes
vfmadd231ps ymm7, ymm10, ymm8	c4 c2 2d b8 f8	5 bytes

The **48** byte at the start of several of these is a **prefix** meaning “64-bit operands”. The chip must find where each instruction ends before it can decode the next one, which is real work. Modern x86 cores keep a cache of already-decoded instructions to avoid repeating it for hot loops.

Exercises

1. Translate the RISC-V `rev` function (chapter 5) into x86-64 Intel syntax.
2. What is in `rax` after `mov rax, -1` followed by `mov eax, 5`? And after `mov rax, -1` then `mov al, 5`?
3. Write `long dot(long *a, long *b, long n)` in x86-64.
4. Why does `cmp` followed by `jae` catch both “index too large” and “index negative” when the index is a signed number?

5. A function pushes `rbx` and `r12` and then calls another function. Is the stack aligned at the call? What if it pushed only `rbx` and `r12` and `r13`?

Chapter 10

SIMD: one instruction, many numbers

SIMD means single instruction, multiple data. A SIMD register holds several numbers side by side, called **lanes**, and one instruction does the same operation on every lane at once. If a register holds 8 floats, one add does 8 additions in the time a normal add does one. This is the first big source of speed on a CPU, and the same idea, scaled up, is what a GPU is.

The registers

Name	Width	Floats (f32) per register	Longs (i64) per register	Instruction set
xmm0 .. xmm15	128 bits	4	2	SSE (every x86-64 chip)
ymm0 .. ymm15	256 bits	8	4	AVX, AVX2
zmm0 .. zmm31	512 bits	16	8	AVX-512 (some Intel and AMD chips)

Xmm0 is the low half of ymm0, which is the low half of zmm0. Scalar float math on x86-64 also uses xmm registers, using only the lowest lane.

Arm has NEON (128-bit) and SVE. RISC-V has the “v” extension, where the vector length is decided by the chip and the code asks for it at run time. The ideas in this chapter apply to all of them.

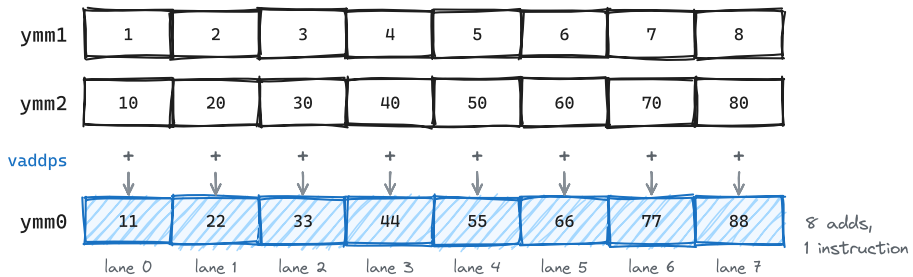
Reading the instruction names

SIMD instruction names are built from parts:

- **v** prefix: the AVX encoding (three operands, like RISC-V).
- The operation: `add`, `mul`, `fmad`, `mov`, `broadcast`, ...
- For floats, the last two letters: `ps` = packed single (all f32 lanes), `pd` = packed double (all f64 lanes), `ss` = scalar single (only lane 0), `sd` = scalar double.

- For integers, a `p` prefix and a size suffix: `vpaddq` = packed add of qwords (64-bit lanes), `vpaddb` = 32-bit lanes.

`vaddps`: one instruction, eight additions



So `vaddps ymm0, ymm1, ymm2` adds 8 pairs of floats. `vaddss xmm0, xmm1, xmm2` adds one pair. When you read compiler output and want to know if a loop was vectorized, look for `ps` and `ymm` instead of `ss` and `xmm`.

FMA: fused multiply-add

`vfmadd231ps ymm7, ymm10, ymm8` computes `ymm7 = ymm10 * ymm8 + ymm7` on 8 lanes, with only one rounding at the end. The digits say which operands are multiplied and which is added: in `231`, operand 2 times operand 3, plus operand 1. FMA is the core instruction of all of linear algebra and ML: a dot product, a matrix multiply and a convolution are long chains of multiply-add. One FMA on 8 lanes is 16 floating point operations (8 multiplies and 8 adds).

A vectorized integer sum

The `sum` function from earlier, compiled by clang with `-O3 -march=x86-64-v3` (allowing AVX2). Unrolling was limited to one vector per iteration so the structure is clear:

compiler output: clang, x86-64, intel syntax

```
sum:
    test    rsi, rsi
    jle     .LBB0_1
    cmp     rsi, 4
    jae     .LBB0_4
    xor     ecx, ecx
    xor     eax, eax
    jmp     .LBB0_7
```

```

.LBB0_1:
    xor     eax, eax
    ret

.LBB0_4:
    movabs  rcx, 9223372036854775804
    and     rcx, rsi
    vpxor   xmm0, xmm0, xmm0
    xor     eax, eax

.LBB0_5:
    vpaddq  ymm0, ymm0, ymmword ptr [rdi + 8*rax]
    add     rax, 4
    cmp     rcx, rax
    jne     .LBB0_5
    vextracti128  xmm1, ymm0, 1
    vpaddq  xmm0, xmm0, xmm1
    vpslufd xmm1, xmm0, 238          # xmm1 = xmm0[2,3,2,3]
    vpaddq  xmm0, xmm0, xmm1
    vmovq   rax, xmm0
    cmp     rcx, rsi
    je      .LBB0_8

.LBB0_7:
    add     rax, qword ptr [rdi + 8*rcx]
    inc     rcx
    cmp     rsi, rcx
    jne     .LBB0_7

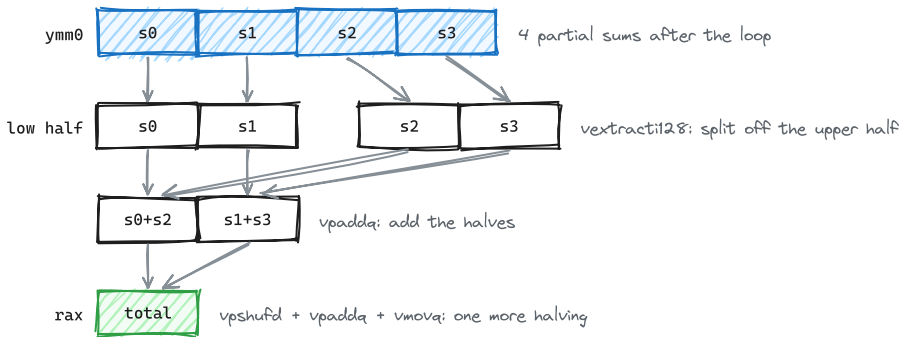
.LBB0_8:
    vzeroupper
    ret

```

There are three parts, and you will see this three-part shape in nearly every vectorized loop:

1. **The main vector loop** (`.LBB0_5`): `vpaddq ymm0, ymm0, ymmword ptr [rdi + 8*rax]` loads 4 longs (32 bytes) and adds them to 4 running sums in `ymm0`. The counter goes up by 4. Before the loop, `movabs rcx, ...` and `and rcx, rsi` round `n` down to a multiple of 4. That constant is `0x7ffffffffffffc`: all ones except the low two bits.
2. **The horizontal reduction**: after the loop, `ymm0` holds 4 partial sums that must be added together. `vextracti128` takes the upper 128 bits, `vpaddq` adds them to the lower half (now 2 sums), `vpslufd` moves the upper 64 bits down, `vpaddq` again (now 1 sum), `vmovq` copies it to `rax`.
3. **The scalar tail** (`.LBB0_7`): the last `n mod 4` elements, one at a time. This is the old scalar loop.

after the loop: fold four partial sums into one



`vzeroupper` at the end is a housekeeping instruction that avoids a slowdown when mixing AVX and older SSE code. It does no math.

Why the compiler can reorder integers but not floats

Look at what the vectorized sum really does: it adds elements 0, 4, 8, ... into lane 0, elements 1, 5, 9, ... into lane 1, and so on, then combines the lanes at the end. That is a different order of additions than the C code. For integers this gives the same result, because integer addition (with wraparound) is associative.

For floats it would not (chapter 1: float addition is not associative). So for a float sum, clang and gcc **will not vectorize the reduction** unless you pass `-ffast-math` (or `-fassociative-math`), or write the multiple accumulators yourself. Chapter 11 measures what that costs.

A vectorized saxpy

Saxpy is `y[i] = a * x[i] + y[i]`. There is no reduction here: each element is independent. So the compiler vectorizes it without any special flags:

```
void saxpy(float *restrict y, const float *restrict x, float a, long n) {
    for (long i = 0; i < n; i++) y[i] = a * x[i] + y[i];
}
```

compiler output: clang, x86-64, intel syntax

```
saxpy:
    test    rdx, rdx
    jle     .LBB0_7
```

```

    cmp     rdx, 8
    jae     .LBB0_3
    xor     eax, eax
    jmp     .LBB0_6
.LBB0_3:
    movabs  rax, 9223372036854775800
    and     rax, rdx
    vbroadcastss ymm1, xmm0
    xor     ecx, ecx
.LBB0_4:
    vmovups ymm2, ymmword ptr [rsi + 4*rcx]
    vfmadd213ps ymm2, ymm1, ymmword ptr [rdi + 4*rcx]
# ymm2 = (ymm1 * ymm2) + mem
    vmovups ymmword ptr [rdi + 4*rcx], ymm2
    add     rcx, 8
    cmp     rax, rcx
    jne     .LBB0_4
    cmp     rax, rdx
    je      .LBB0_7
.LBB0_6:
    vmovss  xmm1, dword ptr [rsi + 4*rax]
    vfmadd213ss xmm1, xmm0, dword ptr [rdi + 4*rax]
# xmm1 = (xmm0 * xmm1) + mem
    vmovss  dword ptr [rdi + 4*rax], xmm1
    inc     rax
    cmp     rdx, rax
    jne     .LBB0_6
.LBB0_7:
    vzeroupper
    ret

```

`vbroadcastss ymm1, xmm0` copies the scalar `a` (passed in `xmm0`) into all 8 lanes of `ymm1`. Then the main loop processes 8 floats per iteration: load 8 of `x`, FMA with 8 of `y` from memory, store 8 back. The tail loop does the same with `ss` instructions, one element at a time.

Restrict and aliasing

The `restrict` keyword in `saxpy` promises the compiler that `x` and `y` do not overlap in memory. Without that promise, the compiler must assume writing `y[i]` might change `x[i+1]`, which would make loading 8 values of `x` ahead of time wrong. It then either refuses to vectorize or adds a run-time check comparing the two pointers and keeps two versions of the loop. Rust slices give the compiler this guarantee automatically (a `&mut` slice cannot overlap anything else), which is one reason Rust loops vectorize well.

What blocks vectorization

When a loop does not vectorize, it is usually one of these:

- A float reduction without permission to reorder.
- Possible aliasing between pointers.
- A loop-carried dependency: iteration i needs the result of iteration $i - 1$ (like a prefix sum).
- A possible early exit: a `break`, a panic, an out-of-bounds check.
- Non-contiguous access: `a[idx[i]]` (a gather) or a large stride.
- A function call inside the loop that was not inlined.

clang tells you why with `-Rpass-missed=loop-vectorize`. gcc has `-fopt-info-vec-missed`.

Exercises

1. Compile saxpy without `restrict` on godbolt. What changes in the output?
2. Compile the float version of `sum (float sum(float *a, long n) )` with `-O3 -march=x86-64-v3`, then add `-ffast-math`. Compare the loops.
3. In the vectorized sum, what happens when $n = 3$? Trace which parts of the code run.
4. How many floating point operations does one `vfmadd231ps` on zmm registers do?

Part 4

Why code is fast or slow

Chapter 11

The cost model

Assembly tells you what work a program asks for. This chapter is about how long that work takes. The short version: on a modern chip, arithmetic is almost free, and the time goes into waiting. Waiting for memory, waiting for a previous result, waiting after a wrong guess. Every number in this chapter was measured on the machine that built this book. They will be different on yours. The ratios will not be far off.

Cycles, latency and throughput

The CPU runs on a clock. One tick is a **cycle**. At 3 GHz a cycle is a third of a nanosecond.

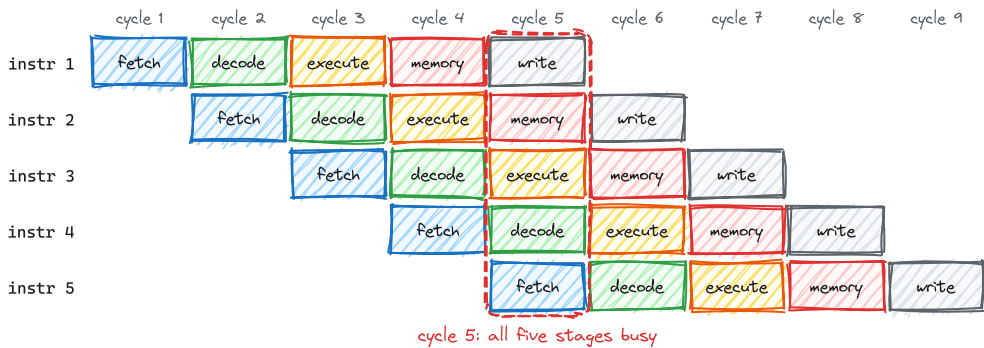
A modern core does not run one instruction at a time. It is **pipelined** (each instruction passes through many stages, and different instructions occupy different stages at the same moment) and **superscalar** (it can start several instructions per cycle). Two numbers describe each kind of instruction:

- **Latency**: cycles from when an instruction starts until its result can be used by the next instruction.
- **Throughput**: how many of that instruction the core can start per cycle, when they do not depend on each other.

Typical numbers for recent x86 cores (they vary by chip; exact tables are at uops.info):

Operation	Latency (cycles)	Can start per cycle
Integer add, logic, shift	1	3 to 4
Integer multiply	3	1
Float add / multiply / FMA	2 to 4	2
Integer divide (64-bit)	About 10 to 40 +	Much less than 1
Load that hits L1 cache	4 to 5	2 to 3
Branch, correctly predicted	(None visible)	1 to 2
Branch, mispredicted	About 15 to 20 lost	

pipelining: several instructions in flight at once



The two columns tell different stories. A float add takes several cycles to finish, but the core can start two new ones every cycle. If each add needs the previous add's result, you get one add per several cycles. If the adds are independent, you get two per cycle. The difference is up to 8x, from the same instruction.

Dependency chains

Here is that effect measured. Three functions sum the same 4096 floats (16 KB, which fits in L1 cache, so memory is not the issue). The first uses one accumulator, so every add waits for the previous one. The others split the work into 4 or 8 independent accumulators. Compiled with `-O2` and no vectorization, so each add is a scalar `addss`:

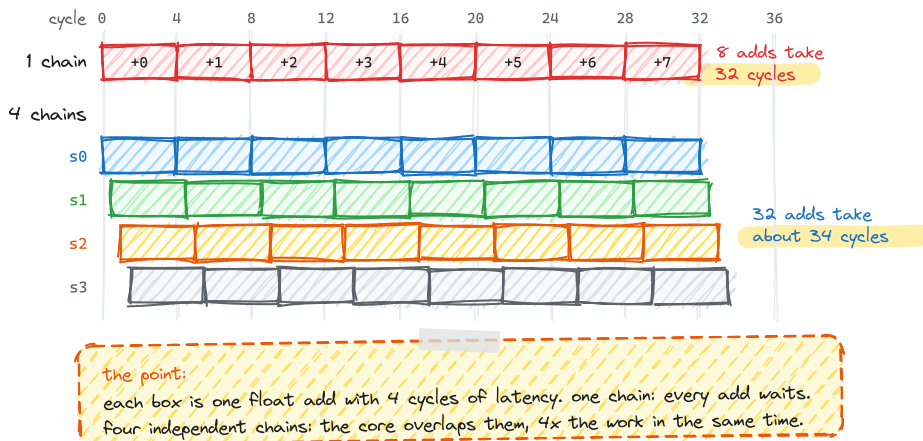
```
float sum1(const float *x) {           // one chain
    float s = 0;
    for (int i = 0; i < N; i++) s += x[i];
    return s;
}
float sum4(const float *x) {           // four independent chains
    float s0 = 0, s1 = 0, s2 = 0, s3 = 0;
    for (int i = 0; i < N; i += 4) { s0 += x[i]; s1 += x[i+1]; s2 += x[i+2]; s3 +=
x[i+3]; }
    return (s0 + s1) + (s2 + s3);
}
```

Measured:

measured output

```
1 chain    0.694 ns per element
4 chains   0.174 ns per element
8 chains   0.164 ns per element
```

latency versus independent work



Four chains are 4x faster than one. The work is identical: the same number of loads and adds. The only change is that four adds can be in flight at once instead of one. Eight chains are no faster than four, because by then the core is already doing about two loads and two adds every cycle, its throughput limit. Once you hit throughput, more independence does not help.

This is the exact reason a compiler cannot speed up `sum1` itself: splitting into four chains changes the order of float additions, which can change the result (chapter 1). You must either write the chains yourself or allow reordering with `-ffast-math`. ML compilers (tinygrad, Triton, TVM) do the same thing when they split a reduction into several accumulators; tinygrad calls it “upcasting”. It is the same trick on a GPU.

Out-of-order execution

A modern core looks ahead in the instruction stream, a window of hundreds of instructions, and starts any instruction whose inputs are ready, regardless of program order. The results are still committed in program order, so the program cannot tell. This is how a core keeps busy while one load waits for memory: it runs independent later instructions in the meantime. It also means **the order instructions appear in a listing is not the order they run**. What limits speed is the chain of dependencies, not the instruction count.

Branch prediction

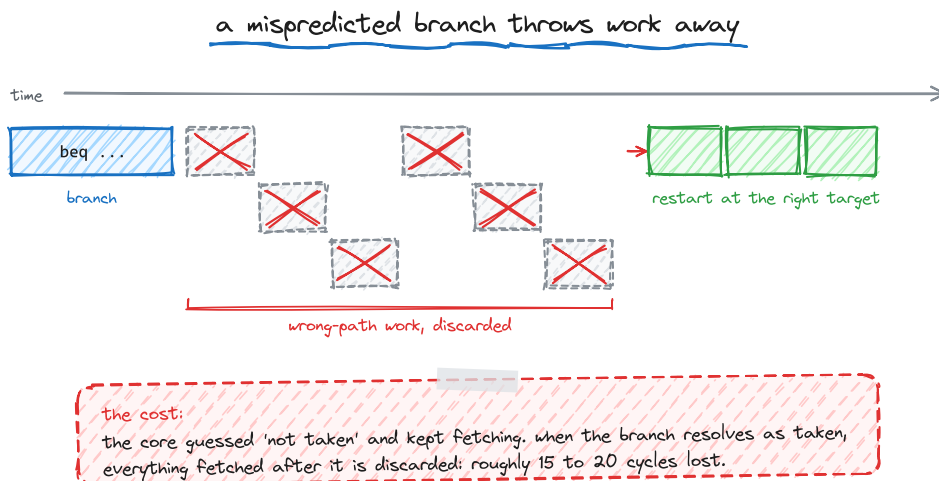
A pipelined core must fetch the instructions after a branch before it knows which way the branch goes. So it guesses, using the history of that branch. A correct guess costs almost nothing. A wrong guess means everything fetched after the branch is thrown away and the core restarts from the right address. That costs roughly 15 to 20 cycles.

Measured: the same loop over the same million values, adding only the values at least 128. On random data the branch goes each way about half the time, so the guess is wrong about half the time. On sorted data the branch goes one way for the first half and the other way for the second half, so the guess is almost always right:

measured output

```
unsorted: 4.37 ns per element
sorted:   0.44 ns per element
```

A 10x difference for identical work. Now the compiler's own version at `-O2`, which replaced the branch with a conditional move (`cmovg`):



measured output

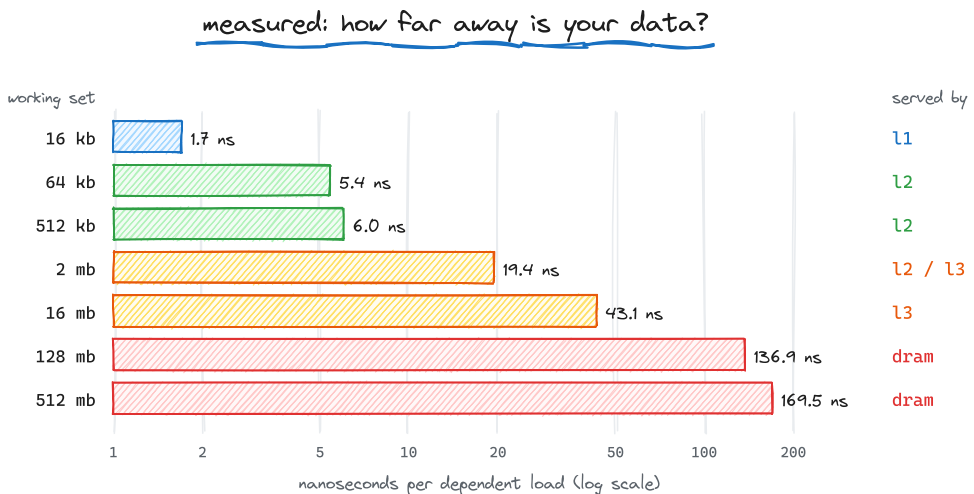
```
unsorted: 0.67 ns per element
sorted:   0.70 ns per element
```

The branchless version does not care about the data order. It is slower than a well-predicted branch (it always does the add) and much faster than a badly-predicted one. That is the rule: **branchless code wins only when the branch is unpredictable.**

Caches

Memory is far away. To hide that, the chip keeps copies of recently used memory in **caches**: small, fast memories close to the core. Data moves between memory and caches in fixed blocks of 64 bytes called **cache lines**. Reading one byte brings its whole 64-byte line into the cache.

There are usually three levels. On this machine: 48 KB of L1 data cache per core, 2 MB of L2 per core, and a large shared L3. To measure the latency of each, the benchmark builds a random chain of pointers through a block of memory of a given size, one pointer per cache line, and follows it. Each load gives the address of the next one, so the core cannot start load $k + 1$ before load k finishes. This measures pure latency:



the point:
from L1 to dram is a 100x jump. that gap is most of performance engineering.

16 KB fits in L1: 1.7 ns. 64 KB and 512 KB are in L2: about 6 ns. 16 MB lives in L3: 43 ns. 128 MB and up go to main memory: 140 to 170 ns. The gap between L1 and main memory is **100x**.

Now the same 512 MB, read in order instead of randomly, still one load per cache line:

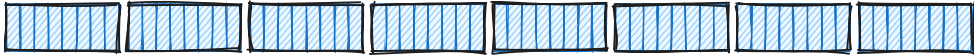
measured output

in-order, one load per cache line: 3.98 ns per line (16.1 GB/s) 0

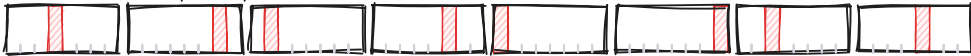
4 ns instead of 170 ns, from the same memory. Two things change. The loads no longer depend on each other, so many can be in flight. And the hardware **prefetcher** notices the sequential pattern and starts fetching lines before the program asks for them. **The order you touch memory in matters as much as how much memory you touch.**

memory moves in 64-byte lines

in order: every byte of every line fetched is used



random: 8 of every 64 bytes fetched are used



the point:

memory always moves whole lines. the random pattern moves 8x more data for the same useful work, and the prefetcher cannot guess what comes next.

The practical rules that follow:

- Use every byte of a cache line you pay for. Walking an array in order uses all 64 bytes. Jumping around uses 8 of them.
- Keep the data a loop works on small enough to stay in cache while the loop needs it. This is what “tiling” or “blocking” means.
- Linked lists and trees of separately allocated nodes are slow because every step is a dependent load to a random address. Arrays are fast.
- Memory is also split into 4 KB pages, and translating each page address is cached in the **TLB**. Random access over a huge range misses the TLB too, which adds to the cost you saw above.

Compute bound or memory bound

For any loop, compare two limits: how fast the core can do its arithmetic, and how fast memory can deliver its bytes. **Arithmetic intensity** is the number of floating point operations done per byte moved from memory.

Saxpy ($y = a * x + y$) loads 8 bytes and stores 4 per element and does 2 FLOPs: $2 / 12$, about 0.17 FLOPs per byte. At 16 GB/s from memory, that is under 3 GFLOP/s, far below what one core’s FMA units can do. Saxpy is **memory bound**: faster arithmetic would not help at all.

A matrix multiply of two $n \times n$ matrices does $2n^3$ FLOPs on $3n^2$ numbers. For $n = 512$ each number gets reused hundreds of times, so if the code is written to reuse data while it is in registers and cache, it can be **compute bound**. The next chapter shows how big that “if” is.



This way of thinking (plot FLOPs per second against FLOPs per byte, and see which limit you hit) is called the **roofline model**. It applies to GPUs without change, and it is the first question to ask about any kernel.

Measuring instead of guessing

- `perf stat ./program` prints cycles, instructions, instructions per cycle (IPC), branch misses and cache misses. IPC tells you a lot: near 4 means the core is busy, below 1 means it is mostly waiting.
- `perf record ./program` then `perf report` shows which functions take the time. `perf annotate` shows the hot instructions in the assembly itself, with the percentage of samples on each line. This is where reading assembly pays off directly.
- `llvm-mca` takes a block of assembly and predicts its cycles per iteration on a chosen chip, showing which resource is the bottleneck. Useful for small hot loops.
- `uops.info` and Agner fog’s instruction tables list latency and throughput per instruction per chip.

One warning: in `perf annotate`, the cost often shows up on the instruction **after** the slow one, because that is where the core was waiting when the sample was taken. A hot `add` right after a load usually means the load is the problem.

Exercises

1. A loop does 1 million dependent float adds with a 4-cycle latency at 3 GHz. How long does it take at minimum? How long if the adds were split into 8 independent chains and the core can start 2 per cycle?
2. Why is the pointer-chasing benchmark a measure of latency, and the in-order benchmark a measure of bandwidth?
3. Compute the arithmetic intensity of a dot product of two float arrays. Is it compute bound or memory bound on this machine?
4. Run `perf stat -e branch-misses` on the branch benchmark yourself (code in appendix C). How many misses per element do you expect for the unsorted case?

Chapter 12

Case study: matrix multiply on a CPU

This chapter puts everything together on the most important operation in ML. Three versions of $C = A \times B$ for 512×512 float matrices, the same math in each, timed on the same machine and compiled with clang `-O3 -march=x86-64-v3`. Then the assembly of the slowest and the fastest, so you can see exactly where the time went. This is exactly the work an ML compiler has to do automatically when it generates fast CPU kernels.

Three versions

```
#define N 512
float A[N][N], B[N][N], C[N][N];

// 1. textbook order
void mm_ijk(void) {
    for (int i = 0; i < N; i++) for (int j = 0; j < N; j++) {
        float acc = 0;
        for (int k = 0; k < N; k++) acc += A[i][k] * B[k][j];
        C[i][j] = acc;
    }
}

// 2. loops swapped: the inner loop runs along rows of B and C
void mm_ikj(void) {
    memset(C, 0, sizeof C);
    for (int i = 0; i < N; i++) for (int k = 0; k < N; k++) {
        float a = A[i][k];
        for (int j = 0; j < N; j++) C[i][j] += a * B[k][j];
    }
}

// 3. register tile: compute a 4 x 16 block of C at a time, kept in registers
void mm_tile(void) {
    for (int i = 0; i < N; i += 4) for (int j = 0; j < N; j += 16) {
        float acc[4][16] = {{0}};
        for (int k = 0; k < N; k++)
            for (int ii = 0; ii < 4; ii++)
                for (int jj = 0; jj < 16; jj++)
                    acc[ii][jj] += A[i+ii][k] * B[k][j+jj];
        for (int ii = 0; ii < 4; ii++) for (int jj = 0; jj < 16; jj++)
            C[i+ii][j+jj] = acc[ii][jj];
    }
}
```

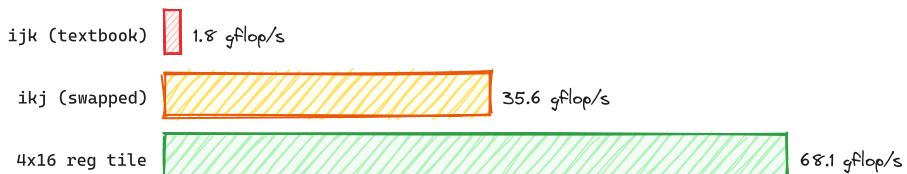
```
    }
}
```

Measured (one core; a 512^3 multiply is $2 \times 512^3 \approx 268$ million FLOPs):

measured output

ijk (textbook)	152.90 ms	1.8 GFLOP/s	ok
ikj (swapped)	7.55 ms	35.6 GFLOP/s	ok
4x16 reg tile	3.94 ms	68.1 GFLOP/s	ok

measured: same math, three loop orders (512 x 512, one core)



the point:
38x from loop order and register reuse alone. not one flop was removed.

The fastest is about **38x** faster than the textbook version. Same FLOPs, same answer (the “ok” column checks two output values against version 1).

Why the textbook version is slow

The inner loop of `mm_ijk`, real compiler output:

compiler output: clang, x86-64, intel syntax

```
mm_ijk:
    lea    rax, [rip + A+28]
    xor    ecx, ecx
    lea    rdx, [rip + B+14336]
    lea    rsi, [rip + C]
.LBB0_1:
    mov    rdi, rcx
    shl    rdi, 11
    add    rdi, rsi
    mov    r8, rdx
    xor    r9d, r9d
.LBB0_2:
    vxorps xmm0, xmm0, xmm0
    mov    r10, r8
```

```

        xor     r11d, r11d
.LBB0_3:
        vmovss  xmm1, dword ptr [rax + 4*r11 - 28]
        vmovss  xmm2, dword ptr [rax + 4*r11 - 24]
        vfmadd132ss    xmm1, xmm0, dword ptr [r10 - 14336] # xmm1 = (xmm1 * mem) +
xmm0
        vfmadd231ss    xmm1, xmm2, dword ptr [r10 - 12288] # xmm1 = (xmm2 * mem) +
xmm1
        vmovss  xmm0, dword ptr [rax + 4*r11 - 20]
        vfmadd132ss    xmm0, xmm1, dword ptr [r10 - 10240] # xmm0 = (xmm0 * mem) +
xmm1
        vmovss  xmm1, dword ptr [rax + 4*r11 - 16]
        vfmadd132ss    xmm1, xmm0, dword ptr [r10 - 8192] # xmm1 = (xmm1 * mem) +
xmm0
        vmovss  xmm0, dword ptr [rax + 4*r11 - 12]
        vfmadd132ss    xmm0, xmm1, dword ptr [r10 - 6144] # xmm0 = (xmm0 * mem) +
xmm1
        vmovss  xmm1, dword ptr [rax + 4*r11 - 8]
        vfmadd132ss    xmm1, xmm0, dword ptr [r10 - 4096] # xmm1 = (xmm1 * mem) +
xmm0
        vmovss  xmm2, dword ptr [rax + 4*r11 - 4]
        vfmadd132ss    xmm2, xmm1, dword ptr [r10 - 2048] # xmm2 = (xmm2 * mem) +
xmm1
        vmovss  xmm0, dword ptr [rax + 4*r11]
        vfmadd132ss    xmm0, xmm2, dword ptr [r10] # xmm0 = (xmm0 * mem) + xmm2
        add     r11, 8
        add     r10, 16384
        cmp     r11, 512
        jne     .LBB0_3
        vmovss  dword ptr [rdi + 4*r9], xmm0
        inc     r9
        add     r8, 4
        cmp     r9, 512
        jne     .LBB0_2
        inc     rcx
        add     rax, 2048
        cmp     rcx, 512
        jne     .LBB0_1
        ret

```

Look at the innermost loop (`.LBB0_3`). Three problems are visible in the assembly:

1. **Scalar:** every instruction ends in `ss` . One float at a time, using 1 of 8 lanes.
2. **One dependency chain:** each `vfmadd` takes the previous one's result (xmm0 feeds xmm1 feeds xmm0 ...). This is `sum1` from chapter 11 again. The compiler may not split it into several accumulators, because that would reorder float additions.
3. **Strided loads:** the B operands are at `[r10 - 14336]` , `[r10 - 12288]` , ... 2048 bytes apart. That is one row of B (512 floats $\times$ 4 bytes). Every B load touches a different cache line and uses 4 of its 64 bytes. Worse, addresses exactly 2048 bytes apart map to the same few sets of the cache and evict each other.

Why swapping the loops helps

In `mm_ikj`, the inner loop is `C[i][j] += a * B[k][j]` for `j = 0..511`. All three of the c-level problems disappear: the `j` loop walks rows of `B` and `C`, so loads are consecutive and use whole cache lines. Each `C[i][j]` is independent of the others, so there is no reduction chain. And the loop is saxpy, which chapter 10 showed vectorizes into 8-wide FMAs. That gives a 20x jump.

But it is still mostly memory bound: for every FMA on 8 floats, it loads 8 floats of `B`, loads 8 floats of `C`, and stores 8 floats of `C`. 3 memory operations per FMA.

The register tile

The tile version computes a 4×16 block of `C`. For each `k` it needs 4 values of `A` (one per row of the tile) and 16 values of `B` (one row segment), and it does $4 \times 16 = 64$ multiply-adds with them. Every `A` value is used 16 times and every `B` value 4 times before it leaves the registers. The accumulators never touch memory until the end. Real compiler output:

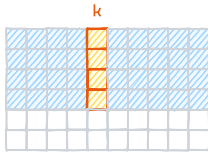
compiler output: clang, x86-64, intel syntax

```
mm_tile:
    push    rbx
    lea     rax, [rip + A+6144]
    xor     ecx, ecx
    lea     rdx, [rip + B+32]
    lea     rsi, [rip + C]
.LBB2_1:
    mov     rdi, rcx
    shl     rdi, 13
    mov     r8, rdx
    xor     r9d, r9d
.LBB2_2:
    mov     r10, r9
    shl     r10, 6
    add     r10, rdi
    vxorps  xmm0, xmm0, xmm0
    mov     r11, -512
    mov     rbx, r8
    vxorps  xmm1, xmm1, xmm1
    vxorps  xmm2, xmm2, xmm2
    vxorps  xmm3, xmm3, xmm3
    vxorps  xmm4, xmm4, xmm4
    vxorps  xmm5, xmm5, xmm5
    vxorps  xmm6, xmm6, xmm6
    vxorps  xmm7, xmm7, xmm7
.LBB2_3:
    vmovups ymm8, ymmword ptr [rbx - 32]
    vmovups ymm9, ymmword ptr [rbx]
    vbroadcastss    ymm10, dword ptr [rax + 4*r11 - 4096]
```

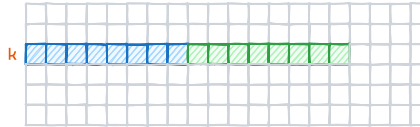
```

vfmadd231ps    ymm7, ymm10, ymm8    # ymm7 = (ymm10 * ymm8) + ymm7
vfmadd231ps    ymm6, ymm9, ymm10    # ymm6 = (ymm9 * ymm10) + ymm6
vbroadcastss   ymm10, dword ptr [rax + 4*r11 - 2048]
vfmadd231ps    ymm5, ymm10, ymm8    # ymm5 = (ymm10 * ymm8) + ymm5
vfmadd231ps    ymm4, ymm9, ymm10    # ymm4 = (ymm9 * ymm10) + ymm4
vbroadcastss   ymm10, dword ptr [rax + 4*r11]
vfmadd231ps    ymm3, ymm10, ymm8    # ymm3 = (ymm10 * ymm8) + ymm3
vfmadd231ps    ymm2, ymm9, ymm10    # ymm2 = (ymm9 * ymm10) + ymm2
vbroadcastss   ymm10, dword ptr [rax + 4*r11 + 2048]
vfmadd231ps    ymm1, ymm10, ymm8    # ymm1 = (ymm10 * ymm8) + ymm1
vfmadd231ps    ymm0, ymm10, ymm9    # ymm0 = (ymm10 * ymm9) + ymm0
add    rbx, 2048
inc    r11
jne    .LBB2_3
vmovups ymmword ptr [r10 + rsi], ymm7
vmovups ymmword ptr [r10 + rsi + 32], ymm6
vmovups ymmword ptr [r10 + rsi + 2048], ymm5
vmovups ymmword ptr [r10 + rsi + 2080], ymm4
vmovups ymmword ptr [r10 + rsi + 4096], ymm3
vmovups ymmword ptr [r10 + rsi + 4128], ymm2
vmovups ymmword ptr [r10 + rsi + 6144], ymm1
vmovups ymmword ptr [r10 + rsi + 6176], ymm0
inc    r9
add    r8, 64
cmp    r9, 32
jne    .LBB2_2
inc    rcx
add    rax, 8192
cmp    rcx, 128
jne    .LBB2_1
pop    rbx
vzeroupper
ret

```

mm_tile's inner loop, drawnA (rows $i:i+3$)

4 values $A[i:i+3][k]$,
each broadcast to 8 lanes

B (row k , columns $j:j+15$)

ymm8 ymm9
16 floats, loaded once per k as two 8-wide registers

broadcast into
every lane of a row



The inner loop `.LBB2_3` is the whole story. Read it line by line:

- `vmovups ymm8, [rbx - 32]` and `vmovups ymm9, [rbx]` : load 16 floats of row k of B (2 registers of 8).
- `vbroadcastss ymm10, [rax + 4*r11 - 4096]` : load one float of A and copy it into all 8 lanes. This happens 4 times, once per row of the tile. The offsets -4096, -2048, 0 and +2048 are 2048 bytes apart, which is exactly one row of A (512 floats $\times$ 4 bytes), measured from a base address the compiler chose.
- 8 `vfmadd231ps` into **ymm0 to ymm7**: the 4×16 tile of C, held in 8 registers of 8 floats.
- `add rbx, 2048` : move to the next row of B.

Per iteration: 6 loads, 8 FMAs = 128 FLOPs. And the 8 FMAs write 8 different accumulators, so there are 8 independent chains: enough to cover the FMA latency. Memory traffic per FLOP dropped from 3 memory ops per 16 FLOPs to 6 per 128.

The C code does not mention registers at all. It only arranged the loops so that a small block of results is reused, and the compiler did the register allocation. **This is what an ML compiler's "upcast" step produces:** the compiler emits C code shaped like `mm_tile`, and checking the assembly for 8 independent `vfmadd231ps` with no loads or stores of the accumulators is how you know it worked.

What is still missing

68 GFLOP/s is good for one core and very simple code, but it is not the end. What fast libraries (OpenBLAS, MKL, oneDNN) add on top:

- **Cache blocking:** split k and j into blocks so the slice of B being reused stays in L1 or L2, instead of streaming all of B from memory for every tile of rows.
- **Packing:** copy blocks of A and B into a contiguous, exactly-ordered buffer first, so the inner kernel reads memory purely sequentially.
- **Bigger tiles:** with AVX-512 there are 32 registers of 16 floats, so tiles like 6×32 or 14×32 .
- **Threads:** split the output across all cores.

Each of these is a loop transformation, not new math, which is why compilers can learn to produce them automatically.

Exercises

1. How many floats of B does `mm_tile` load from memory in total for the whole multiply? Compare with `mm_ikj`.
2. In the tile kernel, why 4×16 and not 16×16 ? (Count registers: there are 16 ymm registers.)
3. Compile `mm_tile` with gcc instead of clang, and with `-march=x86-64` (no AVX). What changes in the inner loop?
4. Change the tile to 6×16 . Does it still fit in registers? Predict the speed, then measure it.

Part 5

GPUs

Chapter 13

How a GPU runs code

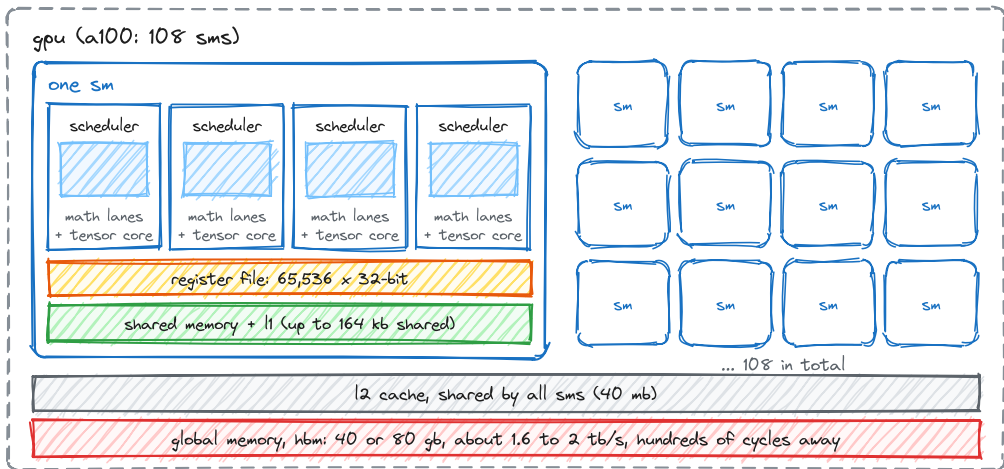
A CPU core is built to finish one stream of instructions as fast as possible: big caches, out-of-order execution, branch prediction. A GPU is built for a different job: run the same small program on millions of data elements, and maximize total throughput even if each element takes a long time. Everything about GPU assembly follows from this. This chapter uses NVIDIA terms, since PTX and SASS are NVIDIA's. AMD's GPUs are organized the same way with different names.

The hardware

A GPU is made of many **streaming multiprocessors** (SMs). An A100 has 108 of them, an H100 (sxm) has 132. Each SM is a processor of its own with:

- A large **register file**: 65,536 registers of 32 bits per SM (256 KB). Far bigger than a CPU core's.
- Several **warp schedulers** (4 per SM on recent chips), each able to issue one instruction per cycle.
- Arithmetic units: many 32-bit float/int lanes, plus **tensor cores** for small matrix multiplies.
- **Shared memory**: fast on-chip memory managed by the program, up to 164 KB per SM on A100 and 228 KB on H100 (it shares space with the L1 cache).

All SMs share an L2 cache (40 MB on A100, 50 MB on H100) and the big off-chip **global memory** (HBM), with bandwidth around 1.6 to 2 TB/s on A100 and about 3.35 TB/s on H100 sxm, but a latency of several hundred cycles per access.

inside a gpu

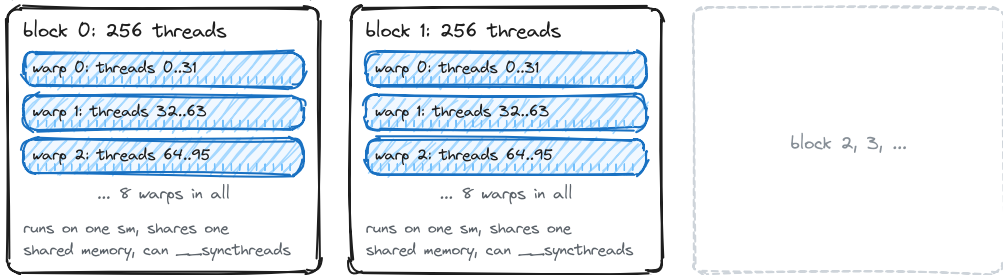
Threads, warps, blocks, grids

You write a **kernel**: a function that one thread runs. You launch it with a grid of threads. The threads are organized in two levels:

- A **block** is a group of up to 1024 threads. A block runs entirely on one SM. Its threads can share data through shared memory and wait for each other with a barrier.
- The **grid** is all the blocks of one launch. Blocks run in any order, on any SM, and cannot wait for each other inside a kernel.

And one level the hardware adds by itself:

- A **warp** is 32 consecutive threads of a block. **The warp is the real unit of execution.** The scheduler issues one instruction for a whole warp, and all 32 threads execute it at once, each on its own data. This is SIMD from chapter 10, with 32 lanes, except that each lane looks like a separate thread to the programmer. NVIDIA calls this SIMT.

grid, blocks, warps, threadsgrid: one launch, blocks numbered by `blockIdx.x`

the point:
 global index $i = \text{blockIdx.x} * \text{blockDim.x} + \text{threadIdx.x}$.
 thread 40 of block 1: $i = 1 * 256 + 40 = 296$.

Each thread knows where it is through built-in values: `threadIdx.x` (its index inside the block), `blockIdx.x` (its block's index in the grid), `blockDim.x` (threads per block). The global index of a thread is almost always computed as:

```
i = blockIdx.x * blockDim.x + threadIdx.x
```

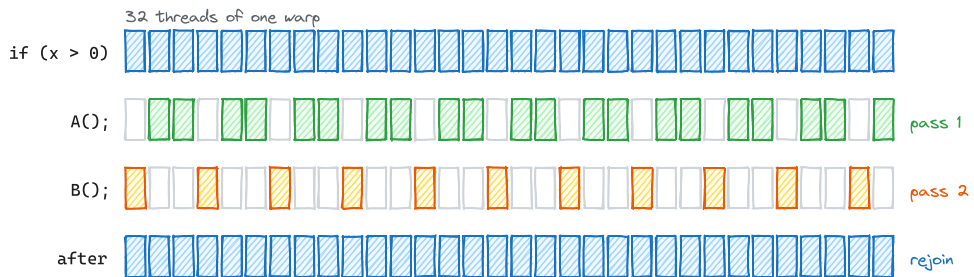
And the simplest kernel uses it to pick which element to work on:

```
__global__ void vadd(const float *a, const float *b, float *c, int n) {
    int i = blockIdx.x * blockDim.x + threadIdx.x;
    if (i < n) c[i] = a[i] + b[i];
}
```

There is no loop. The loop over i is the grid itself: launch enough threads and each one handles one i . The `if (i < n)` handles the last block, which may have more threads than elements left.

Divergence

All 32 threads of a warp execute the same instruction. So what happens at `if (x > 0) A else B` when some threads have $x > 0$ and others do not? The warp runs **both** paths, one after the other, with the threads that should not participate switched off (masked). This is **divergence**. A branch where all 32 threads agree costs nothing extra. A branch where they disagree costs the sum of both sides. In assembly, you will see branches controlled by **predicates**: per-thread true/false registers that switch individual threads on or off.

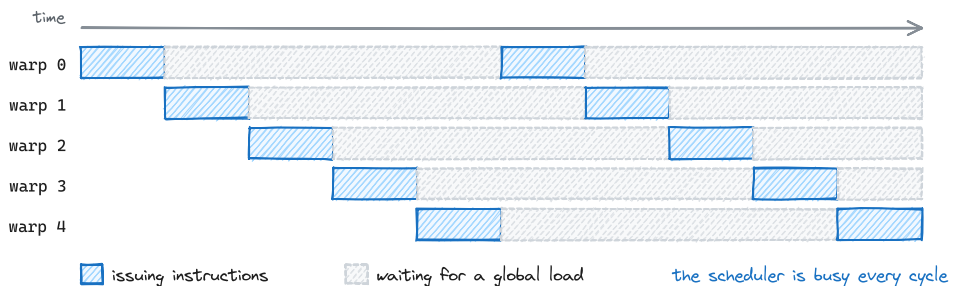
divergence: a warp runs both sides of a branch

the cost:

a branch where all 32 threads agree costs nothing extra. disagreement costs both paths.

Latency hiding: the core idea of GPUs

A global memory load takes hundreds of cycles. A CPU hides this with caches and out-of-order execution. A GPU mostly does not. Instead, each SM keeps **many warps resident at once** (up to 64 warps = 2048 threads on A100) and every cycle each scheduler picks some warp that is ready to run. While warp 3 waits for its load, warps 4, 5 and 6 compute. Switching warps costs nothing, because every warp's registers stay in the register file the whole time.

how a gpu hides memory latency

This gives the main trade-off of GPU programming. **Occupancy** is how many warps an SM holds compared with its maximum. It is limited by resources:

- **Registers:** 65,536 per SM. If a kernel uses 64 registers per thread, an SM can hold $65536 / 64 = 1024$ threads = 32 warps. At 128 registers per thread, only 16 warps. At 255 (the maximum), about 8.

- **Shared memory:** if each block uses 48 KB, at most 3 blocks fit in 164 KB.
- Threads per block and blocks per SM limits.

More warps hide more latency. But more registers per thread allow bigger register tiles (chapter 12), which cut memory traffic. Fast kernels balance these two, and you see the balance in the assembly as the register count.

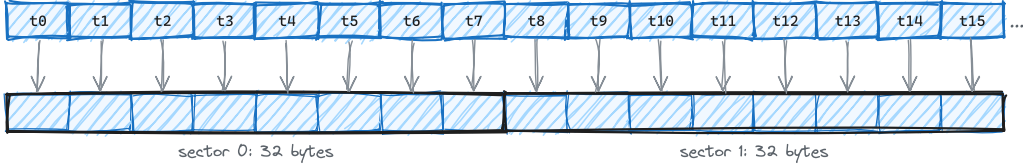
The memory spaces

Space	Where	Who sees it	Speed
Registers	In the SM	One thread	Fastest
Shared memory	In the SM	All threads of one block	About as fast as L1
Local memory	Global memory (cached)	One thread	Slow. Used when registers run out (spills)
Global memory	Off-chip HBM	Everyone	Hundreds of cycles, but huge bandwidth
Constant memory	Global, with a special cache	Everyone, read-only	Fast when all threads read the same address

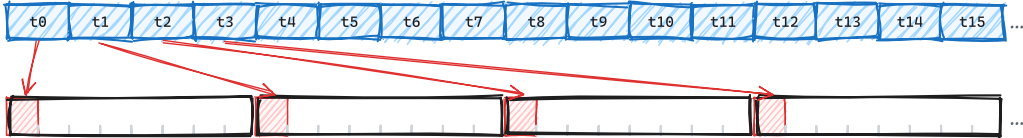
Coalescing

Global memory is read in 32-byte sectors. When the 32 threads of a warp each load a 4-byte float, the hardware merges their requests:

- Thread t reads address $\text{base} + 4t$: all 32 addresses fall in 128 consecutive bytes, so 4 sectors serve the whole warp. This is **coalesced** access, and it is what `vadd` does.
- Thread t reads address $\text{base} + 4 * 512 * t$ (a column of a row-major matrix): 32 different sectors, each used for 4 of its 32 bytes. 8x more memory traffic for the same data.

the same warp, two access patternscoalesced: thread t reads float t 

16 threads need 2 sectors. a full warp of 32 needs 4 sectors, 128 bytes, and uses every byte.

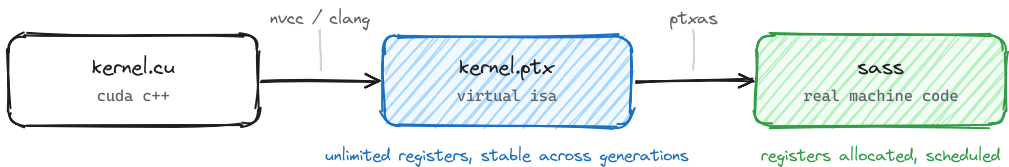
strided: thread t reads float $512 * t$ (one column of a matrix)

each thread needs its own sector and uses 4 of its 32 bytes: the warp needs 32 sectors instead of 4.

On a GPU, consecutive threads should touch consecutive addresses. That rule decides which matrix dimension you map to `threadIdx.x`, and it is the GPU version of “walk arrays in order” from chapter 11.

From CUDA to machine code

```
kernel.cu --(nvcc / clang)--> ptx --(ptxas)--> sass, inside a .cubin file
cuda c++          virtual isa      the real machine code
```

from cuda to gpu machine code

PTX is a virtual instruction set: stable across GPU generations, with unlimited registers.

SASS is the real machine code of one specific GPU generation, produced from PTX by the `ptxas` assembler, which does register allocation and instruction scheduling. A program can ship PTX and let the driver compile it for whatever GPU it lands on. The next two chapters read both.

Exercises

1. A kernel is launched with 4096 blocks of 256 threads. How many threads in total?
How many warps per block?
2. A kernel uses 96 registers per thread. How many warps can one A100 SM hold, considering registers only?
3. A warp executes `if (threadIdx.x % 2 == 0) A(); else B();`. How long does it take compared with only A?
4. A 1024×1024 row-major float matrix is processed by a kernel where `threadIdx.x` picks the row and the thread loops over columns. Is the access coalesced? How would you fix it?

Chapter 14

PTX

PTX is the assembly language that CUDA compiles to first. It looks like a risc instruction set with types attached, and with unlimited registers. You read it to see what the compiler understood from your kernel: which loads it made, which memory space it used, whether it used vector loads, predicates, shuffles or tensor core instructions. Every listing in this chapter is real output of clang's nvptx backend for `sm_80` (A100), from the CUDA source shown above it.

The shape of a PTX instruction

```
opcode.modifier.type    destination, source1, source2;
```

- `add.f32 %f3, %f1, %f2;` adds two f32 registers.
- `ld.global.f32 %f1, [%rd3];` loads an f32 from global memory at the address in %rd3.
- `mad.lo.s32 %r5, %r2, %r3, %r4;` multiplies two signed 32-bit values, keeps the low 32 bits of the product, and adds a third.
- `setp.ge.s32 %p1, %r5, %r1;` sets the predicate %p1 to ($r5 \geq r1$).
- `@%p1 bra $L_BB0_2;` branches only in threads where %p1 is true.

Types: `.s32` / `.u32` / `.s64` / `.u64` (signed and unsigned integers),
`.f16`, `.f32`, `.f64`, `.b32` / `.b64` (raw bits), `.pred` (a true/false predicate).

Registers are virtual and declared in bulk at the top of each function, for example `.reg .f32 %f<4>;` declares %f0 to %f3. The names are a convention of the compiler: `%r` for 32-bit integers, `%rd` for 64-bit, `%f` for floats, `%p` for predicates. PTX does not care how many you use. `ptxas` maps them to real registers later.

Special registers

PTX	CUDA	Meaning
<code>%tid.x</code>	<code>threadIdx.x</code>	Thread index inside the block
<code>%ntid.x</code>	<code>blockDim.x</code>	Threads per block
<code>%ctaid.x</code>	<code>blockIdx.x</code>	Block index (“cta” = cooperative thread array = block)
<code>%nctaid.x</code>	<code>gridDim.x</code>	Number of blocks
<code>%laneid</code>	(None)	0 to 31, position inside the warp

Each has `.y` and `.z` versions too.

State spaces

Every load and store names the memory it touches:

Space	In PTX	Contents
<code>.param</code>	<code>ld.param</code>	The kernel’s arguments
<code>.global</code>	<code>ld.global</code> , <code>st.global</code>	Global memory (HBM)
<code>.shared</code>	<code>ld.shared</code> , <code>st.shared</code>	Per-block on-chip memory
<code>.local</code>	<code>ld.local</code> , <code>st.local</code>	Per-thread memory, used for spills and stack arrays
<code>.const</code>	<code>ld.const</code>	Constant memory

Seeing `ld.local` or `st.local` in a kernel you expected to be fast is a warning sign. It means something did not fit in registers.

Vector add, line by line

```
extern "C" __global__ void vadd(const float *a, const float *b, float *c, int n) {
    int i = blockIdx.x * blockDim.x + threadIdx.x;
    if (i < n) c[i] = a[i] + b[i];
}
```

compiler output: clang, nvptx, sm_80

```
.visible .entry vadd(
    .param .u64 vadd_param_0,
    .param .u64 vadd_param_1,
    .param .u64 vadd_param_2,
    .param .u32 vadd_param_3
)
{
    .reg .pred    %p<2>;
    .reg .b32     %r<6>;
    .reg .f32     %f<4>;
    .reg .b64     %rd<11>;
    ld.param.u32  %r1, [vadd_param_3];
    mov.u32       %r2, %ctaid.x;
    mov.u32       %r3, %ntid.x;
    mov.u32       %r4, %tid.x;
    mad.lo.s32    %r5, %r2, %r3, %r4;
    setp.ge.s32   %p1, %r5, %r1;
    @%p1 bra      $L__BB0_2;
    ld.param.u64  %rd4, [vadd_param_0];
    ld.param.u64  %rd5, [vadd_param_2];
    cvta.to.global.u64 %rd6, %rd5;
    ld.param.u64  %rd7, [vadd_param_1];
    cvta.to.global.u64 %rd8, %rd7;
    cvta.to.global.u64 %rd9, %rd4;
    mul.wide.s32  %rd10, %r5, 4;
    add.s64       %rd1, %rd6, %rd10;
    add.s64       %rd2, %rd8, %rd10;
    add.s64       %rd3, %rd9, %rd10;
    ld.global.f32 %f1, [%rd3];
    ld.global.f32 %f2, [%rd2];
    add.f32       %f3, %f1, %f2;
    st.global.f32 [%rd1], %f3;
$L__BB0_2:
    ret;
}
```

1. The `.entry` header declares a kernel with four `.param` arguments: three 64-bit pointers and one 32-bit int.
2. `ld.param.u32 %r1, [vadd_param_3]` loads `n`.
3. Three `mov` s read `%ctaid.x`, `%ntid.x`, `%tid.x`, and `mad.lo.s32` computes `i = blockIdx × blockDim + threadIdx` in one instruction.
4. `setp.ge.s32` sets `%p1 = (i >= n)`, and `@%p1 bra` sends those threads to the end. This is the `if`.
5. The pointer parameters are loaded and `cvta.to.global` converts each from a generic address to a global-space address.

6. `mul.wide.s32 %rd10, %r5, 4` multiplies the 32-bit `i` by 4 into a 64-bit result: the byte offset. (“wide” means the result is twice the width of the inputs.) then three `add.s64` make the three addresses.
7. Two `ld.global.f32`, one `add.f32`, one `st.global.f32`. The actual work is these four instructions. Everything before is index arithmetic.

That ratio is normal for small kernels: most instructions compute addresses. On a GPU this is cheap because the integer units run in parallel with memory traffic. The kernel’s speed is set by the two loads and one store, and since consecutive threads touch consecutive floats, they are coalesced.

Restrict and the read-only path

Mark the input pointers `const float *__restrict__` and the loads become `ld.global.nc.f32`. `.nc` means “non-coherent”: the compiler has promised the data will not change during the kernel, so the hardware may serve it through the read-only texture/l1 path. You can see this in the reduction and matmul listings below.

Wider loads

A thread can load 16 bytes at once if the address is 16-byte aligned. This kernel processes 4 floats per thread through a 16-byte aligned struct:

```
struct __attribute__((aligned(16))) f4 { float x, y, z, w; };
extern "C" __global__ void vadd4(const f4 *__restrict__ a, const f4 *__restrict__ b,
                                f4 *__restrict__ c, int n4) {
    int i = blockIdx.x * blockDim.x + threadIdx.x;
    if (i < n4) {
        f4 p = a[i], q = b[i];
        c[i] = f4{p.x + q.x, p.y + q.y, p.z + q.z, p.w + q.w};
    }
}
```

The memory instructions of the output:

compiler output: clang, nvptx, sm_80

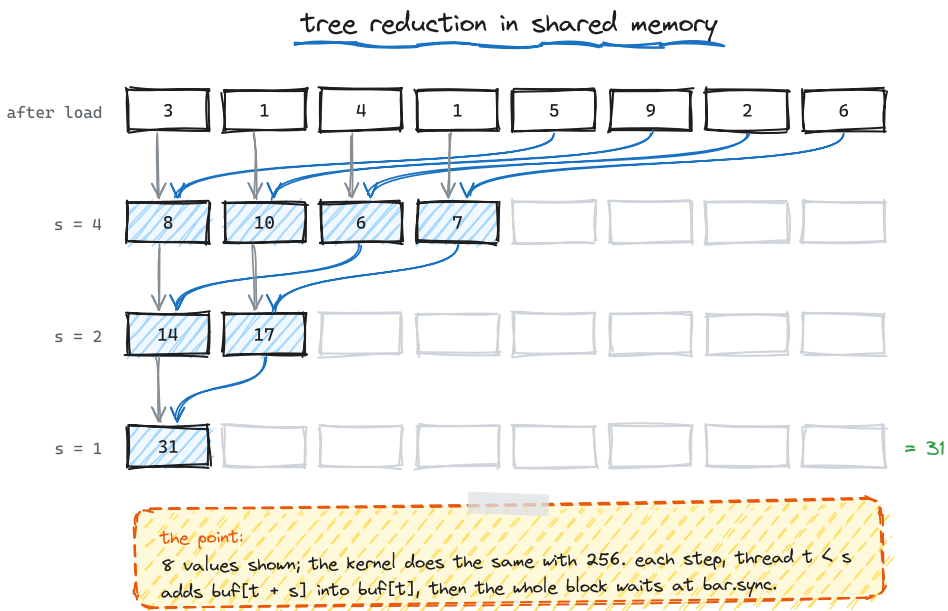
```
ld.global.nc.v4.f32    {%f1, %f2, %f3, %f4}, [%rd8];
ld.global.nc.v4.f32    {%f5, %f6, %f7, %f8}, [%rd9];
add.f32                %f9, %f1, %f5;
add.f32                %f10, %f2, %f6;
add.f32                %f11, %f3, %f7;
add.f32                %f12, %f4, %f8;
st.global.v4.f32       [%rd10], {%f9, %f10, %f11, %f12};
```

`ld.global.nc.v4.f32` loads four floats into four registers with one instruction. Fewer, wider memory instructions mean fewer instructions issued per byte, which matters for memory-bound kernels. In CUDA this is what `float4` does. In a compiler you generate it by grouping 4 consecutive elements per thread and proving alignment, which is the GPU form of upcasting.

A block reduction with shared memory

Summing 256 numbers per block: each thread writes its number into shared memory, then the block repeatedly halves the active threads, each adding a partner's value, with a barrier between steps:

```
extern "C" __global__ void block_sum(const float *__restrict__ x, float *out, int
n) {
    __shared__ float buf[256];
    int t = threadIdx.x;
    int i = blockIdx.x * 256 + t;
    buf[t] = i < n ? x[i] : 0.0f;
    __syncthreads();
    for (int s = 128; s > 0; s >>= 1) {
        if (t < s) buf[t] += buf[t + s];
        __syncthreads();
    }
    if (t == 0) out[blockIdx.x] = buf[0];
}
```



Compiled with loop unrolling off, so the loop is visible:

compiler output: clang, nvptx, sm_80

```
.visible .entry block_sum(
    .param .u64 block_sum_param_0,
    .param .u64 block_sum_param_1,
    .param .u32 block_sum_param_2
)
{
    .reg .pred    %p<5>;
    .reg .b32     %r<10>;
    .reg .f32     %f<9>;
    .reg .b64     %rd<14>;
    // demoted variable
    .shared .align 4 .b8 _ZZ9block_sumE3buf[1024];
    ld.param.u64   %rd7, [block_sum_param_1];
    ld.param.u32   %r5, [block_sum_param_2];
    mov.u32        %r1, %tid.x;
    mov.u32        %r2, %ctaid.x;
    shl.b32        %r6, %r2, 8;
    add.s32        %r7, %r6, %r1;
    setp.ge.s32    %p1, %r7, %r5;
    mov.f32        %f8, 0f00000000;
    @%p1 bra       $L__BB0_2;
    ld.param.u64   %rd6, [block_sum_param_0];
    cvta.to.global.u64 %rd8, %rd6;
    mul.wide.s32   %rd9, %r7, 4;
    add.s64        %rd2, %rd8, %rd9;
    ld.global.nc.f32 %f8, [%rd2];
$L__BB0_2:
    cvta.to.global.u64 %rd1, %rd7;
    mul.wide.s32   %rd10, %r1, 4;
    mov.u64        %rd11, _ZZ9block_sumE3buf;
    add.s64        %rd3, %rd11, %rd10;
    st.shared.f32  [%rd3], %f8;
    bar.sync       0;
    mov.b32        %r9, 128;
    bra.uni        $L__BB0_3;
$L__BB0_5:
    bar.sync       0;
    shr.u32        %r4, %r9, 1;
    setp.lt.u32    %p3, %r9, 2;
    mov.u32        %r9, %r4;
    @%p3 bra       $L__BB0_6;
$L__BB0_3:
    .pragma "nounroll";
    setp.ge.s32    %p2, %r1, %r9;
    @%p2 bra       $L__BB0_5;
    mul.wide.s32   %rd12, %r9, 4;
    add.s64        %rd5, %rd3, %rd12;
    ld.shared.f32  %f4, [%rd5];
    ld.shared.f32  %f5, [%rd3];
    add.f32        %f6, %f4, %f5;
```

```

        st.shared.f32 [%rd3], %f6;
        bra.uni      $L__BB0_5;
$L__BB0_6:
        setp.eq.s32   %p4, %r1, 0;
        @%p4 bra      $L__BB0_7;
        bra.uni      $L__BB0_8;
$L__BB0_7:
        mul.wide.u32  %rd13, %r2, 4;
        add.s64       %rd4, %rd1, %rd13;
        ld.shared.f32 %f7, [_ZZ9block_sumE3buf];
        st.global.f32 [%rd4], %f7;
$L__BB0_8:
        ret;
}

```

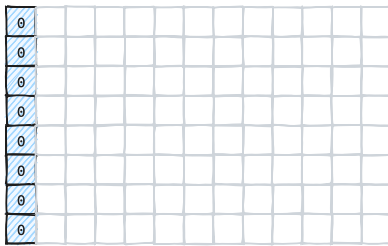
What to notice:

- `.shared .align 4 .b8 _ZZ9block_sumE3buf[1024]` : the 256-float shared array, as 1024 raw bytes. (“demoted variable” means the compiler moved a variable declared inside the function into the shared state space, where it really lives.)
- `blockIdx.x * 256` became `shl.b32 %r6, %r2, 8` : shift left by 8.
- The ternary `i < n ? x[i] : 0.0f` became: put 0 in %f8, and skip the load in threads where $i \geq n$.
- `st.shared.f32` writes the thread’s value, and `bar.sync 0` is `__syncthreads()` : every thread of the block waits there until all have arrived.
- The loop (`$L__BB0_3` and `$L__BB0_5`) : threads with $t \geq s$ skip the add, but all threads reach the `bar.sync` . The `shr.u32` halves `s`. It is a correctness rule that every thread of the block reaches every `bar.sync` . A barrier inside a branch that only some threads take can hang the GPU.
- `bra.uni` is a branch the compiler knows all threads in the warp take the same way: a uniform branch, no divergence.

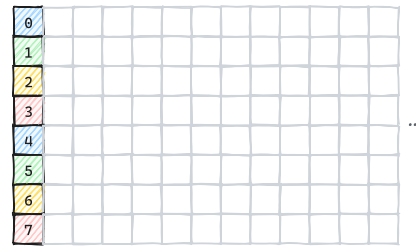
Bank conflicts: shared memory is split into 32 banks, each 4 bytes wide, with consecutive 4-byte words in consecutive banks. If the 32 threads of a warp access 32 different banks, it takes one cycle. If several threads access different addresses in the same bank, those accesses are serialized. Here thread `t` reads `buf[t]` and `buf[t + s]` : consecutive threads hit consecutive banks, so there are no conflicts. A column access in a 2d shared array with 32 columns would put every thread in the same bank, a 32-way conflict. The usual fix is to pad each row by one element.

shared memory bank conflicts

tile[32][32]: column 0 is all bank 0



tile[32][33]: column 0 hits banks 0,1,2,...



the point:

a warp reads one column. left: all 32 addresses sit in bank 0, so the reads go one at a time. right: one unused float per row shifts each row by one bank, so all 32 reads happen at once. numbers are banks: $(row * row_length + col) \bmod 32$.

Warp shuffles

Threads in a warp can read each other's registers directly, without shared memory.

`shfl.sync.down` gives each thread the value from the thread `offset` lanes higher. A warp-level sum:

```
extern "C" __global__ void warp_sum(const float *__restrict__ x, float *out) {
    float v = x[blockIdx.x * 32 + threadIdx.x];
    for (int off = 16; off > 0; off >= 1)
        v += __nvvm_shfl_sync_down_f32(0xffffffff, v, off, 0x1f); //
    __shfl_down_sync_in_cuda
    if (threadIdx.x == 0) out[blockIdx.x] = v;
}
```

compiler output: clang, nvptx, sm_80

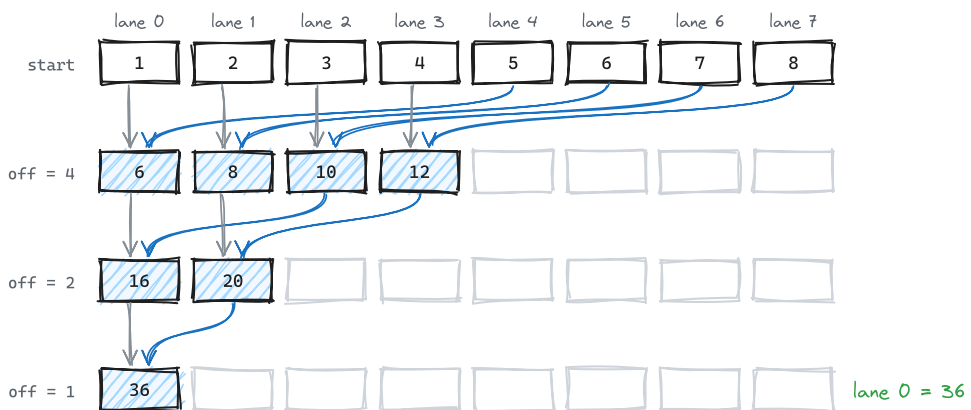
```
.visible .entry warp_sum(
    .param .u64 warp_sum_param_0,
    .param .u64 warp_sum_param_1
)
{
    .reg .pred    %p<2>;
    .reg .b32     %r<5>;
    .reg .f32     %f<12>;
    .reg .b64     %rd<9>;
    ld.param.u64  %rd2, [warp_sum_param_0];
    cvta.to.global.u64 %rd5, %rd2;
    mov.u32       %r1, %ctaid.x;
    shl.b32       %r2, %r1, 5;
    mov.u32       %r3, %tid.x;
```

```

add.s32      %r4, %r2, %r3;
mul.wide.u32 %rd6, %r4, 4;
add.s64      %rd7, %rd5, %rd6;
ld.global.nc.f32 %f2, [%rd7];
shfl.sync.down.b32 %f3, %f2, 16, 31, -1;
add.f32      %f4, %f2, %f3;
shfl.sync.down.b32 %f5, %f4, 8, 31, -1;
add.f32      %f6, %f4, %f5;
shfl.sync.down.b32 %f7, %f6, 4, 31, -1;
add.f32      %f8, %f6, %f7;
shfl.sync.down.b32 %f9, %f8, 2, 31, -1;
add.f32      %f10, %f8, %f9;
shfl.sync.down.b32 %f11, %f10, 1, 31, -1;
setp.ne.s32   %p1, %r3, 0;
@%p1 bra     $L__BB0_2;
ld.param.u64  %rd3, [warp_sum_param_1];
cvta.to.global.u64 %rd4, %rd3;
add.f32      %f1, %f10, %f11;
mul.wide.u32  %rd8, %r1, 4;
add.s64      %rd1, %rd4, %rd8;
st.global.f32 [%rd1], %f1;
$L__BB0_2:
ret;
}

```

warp shuffle reduction



the point:

8 lanes shown; a real warp has 32 lanes and needs 5 steps. grey: the lane keeps its own value. blue: shfl.sync.down brings the value from lane + off.

The compiler fully unrolled the loop into five `shfl.sync.down.b32` and five `add.f32`: offsets 16, 8, 4, 2, 1. After the five steps, lane 0 holds the sum of all 32 values. The operands are: destination, source value, offset, a clamp value (31) and the member mask (-1 = all 32 lanes participate). No shared memory, no barrier. Fast reductions use shuffles inside each warp and shared memory only to combine warps.

Naive matrix multiply, and why it is slow

```
extern "C" __global__ void matmul(const float *__restrict__ A, const float *__restrict__ B,
                                float *__restrict__ C, int M, int N, int K) {
    int row = blockIdx.y * blockDim.y + threadIdx.y;
    int col = blockIdx.x * blockDim.x + threadIdx.x;
    if (row >= M || col >= N) return;
    float acc = 0.0f;
    for (int k = 0; k < K; k++)
        acc += A[row * K + k] * B[k * N + col];
    C[row * N + col] = acc;
}
```

compiler output: clang, nvptx, sm_80

```
.visible .entry matmul(
    .param .u64 matmul_param_0,
    .param .u64 matmul_param_1,
    .param .u64 matmul_param_2,
    .param .u32 matmul_param_3,
    .param .u32 matmul_param_4,
    .param .u32 matmul_param_5
)
{
    .reg .pred    %p<6>;
    .reg .b32     %r<20>;
    .reg .f32     %f<10>;
    .reg .b64     %rd<16>;
    ld.param.u32  %r7, [matmul_param_4];
    ld.param.u32  %r9, [matmul_param_3];
    mov.u32       %r10, %ctaid.y;
    mov.u32       %r11, %ntid.y;
    mov.u32       %r12, %tid.y;
    mad.lo.s32    %r1, %r10, %r11, %r12;
    mov.u32       %r13, %ctaid.x;
    mov.u32       %r14, %ntid.x;
    mov.u32       %r15, %tid.x;
    mad.lo.s32    %r2, %r13, %r14, %r15;
    setp.ge.s32   %p1, %r1, %r9;
    setp.ge.s32   %p2, %r2, %r7;
    or.pred       %p3, %p1, %p2;
    @%p3 bra      $L__BB0_5;
    ld.param.u32  %r19, [matmul_param_5];
```

```

ld.param.u64    %rd8, [matmul_param_2];
cvta.to.global.u64    %rd1, %rd8;
setp.lt.s32     %p4, %r19, 1;
mov.f32        %f9, 0f00000000;
@%p4 bra       $L__BB0_4;
ld.param.u64    %rd7, [matmul_param_0];
ld.param.u64    %rd9, [matmul_param_1];
cvta.to.global.u64    %rd2, %rd9;
cvta.to.global.u64    %rd3, %rd7;
mul.lo.s32     %r16, %r1, %r19;
mul.wide.s32   %rd10, %r16, 4;
add.s64        %rd15, %rd3, %rd10;
mov.f32        %f9, 0f00000000;
mov.u32        %r18, %r2;
$L__BB0_3:
.pragma "nounroll";
ld.global.nc.f32    %f6, [%rd15];
mul.wide.s32     %rd11, %r18, 4;
add.s64         %rd12, %rd2, %rd11;
ld.global.nc.f32    %f7, [%rd12];
fma.rn.f32       %f9, %f6, %f7, %f9;
add.s32         %r19, %r19, -1;
add.s32         %r18, %r18, %r7;
add.s64         %rd15, %rd15, 4;
setp.eq.s32      %p5, %r19, 0;
@%p5 bra       $L__BB0_4;
bra.uni         $L__BB0_3;
$L__BB0_4:
mad.lo.s32      %r17, %r1, %r7, %r2;
mul.wide.s32    %rd13, %r17, 4;
add.s64        %rd14, %rd1, %rd13;
st.global.f32   [%rd14], %f9;
$L__BB0_5:
ret;
}

```

The inner loop is `$L__BB0_3`. Per iteration: two `ld.global.nc.f32`, one `fma.rn.f32` (fused multiply-add, round to nearest), and loop bookkeeping. **Two global loads for every 2 FLOPs**. This is the GPU version of the textbook matmul from chapter 12, and it is memory bound for the same reason.

The access pattern is not all bad. `col` comes from `threadIdx.x`, so the 32 threads of a warp read 32 consecutive floats of B: coalesced. All 32 threads read the same A element: one sector, broadcast to all. The real problem is reuse: each value is loaded from global memory once per thread that needs it, with no sharing.

The standard fix has two levels, exactly mirroring the CPU:

1. **Shared memory tiling**: a block loads a tile of A and a tile of B into shared memory once (`ld.global` then `st.shared`, then `bar.sync`), and all its threads read the tiles from there many times (`ld.shared`).

2. **Register tiling:** each thread computes a small block of C (say 8×8) in registers, so each value it reads from shared memory is used 8 times.

In the PTX of a well-tiled kernel, the inner loop is mostly `fma.rn.f32` with a few `ld.shared` (often `.v4`) between them, and `ld.global` only in the outer loop.

Tensor cores

Tensor cores multiply small matrices in one instruction per warp. The PTX is:

```
mma.sync.aligned.m16n8k16.row.col.f32.f16.f16.f32 {d0,d1,d2,d3}, {a0,a1,a2,a3},
{b0,b1}, {c0,c1,c2,c3};
```

Reading the name: a warp-wide (`sync`, `aligned`) matrix multiply-accumulate with shape `m16 n8 k16`: a 16×16 tile of A times a 16×8 tile of B, added to a 16×8 tile of C. A is row-major, B column-major. The output and accumulator are `f32`, the inputs `f16`. Each of the 32 threads holds a few fragments of each matrix in its registers, in a layout fixed by the PTX documentation. `ldmatrix` loads those fragments from shared memory in the right layout. On Hopper (H100), the newer `wgmma` instructions work on a group of 4 warps and read operands straight from shared memory.

When you check whether a kernel uses tensor cores, you look for `mma` in the PTX or `HMMMA` in the SASS. If they are not there, the kernel is running on the normal FMA units at a fraction of the GPU's possible throughput.

Writing PTX by hand: inline assembly

You rarely write whole kernels in PTX, but you do drop single instructions into CUDA when there is no C++ way to express them. The syntax is the same as gcc inline assembly:

```
extern "C" __global__ void lane_ids(unsigned *out) {
    unsigned lane;
    asm volatile("mov.u32 %0, %%laneid;" : "=r"(lane));
    out[blockIdx.x * blockDim.x + threadIdx.x] = lane;
}
```

`%0` refers to the first operand, `"=r"(lane)` says it is an output in a 32-bit register bound to `lane`, and `%%` writes a literal `%`. The instruction lands verbatim in the output:

compiler output: clang, nvptx, sm_80

```
{
    .reg .b32      %r<6>;
    .reg .b64      %rd<5>;
```

```

ld.param.u64    %rd1, [lane_ids_param_0];
cvta.to.global.u64    %rd2, %rd1;
mov.u32 %r1, %laneid;
mov.u32    %r2, %ctaid.x;
mov.u32    %r3, %ntid.x;
mov.u32    %r4, %tid.x;
mad.lo.s32    %r5, %r2, %r3, %r4;
mul.wide.u32    %rd3, %r5, 4;
add.s64    %rd4, %rd2, %rd3;
st.global.u32    [%rd4], %r1;
ret;
}

```

The same mechanism is used for `cp.async` (asynchronous copies from global to shared memory), special cache hints, and `mma` in hand-tuned kernels.

Exercises

1. In the `vadd` PTX, which instructions would disappear if `n` were known to be a multiple of the block size and the `if` were removed?
2. Why does `block_sum` need a `bar.sync` inside the loop, and not just one at the end?
3. A 32×32 float array in shared memory is read column-wise by a warp (thread `t` reads `tile[t][c]`). Which bank does each thread hit? What does declaring it as `tile[32][33]` change?
4. In the naive `matmul`, how many bytes of global memory traffic does one thread cause for $K = 1024$? How many FLOPs does it do?
5. Compile `vadd` yourself with `nvcc -ptx` or on godbolt with `nvcc`. Compare with the clang output here. What differs?

Chapter 15

SASS

SASS is the machine code the GPU actually executes. `ptxas` produces it from PTX: it assigns real registers, picks real instructions, and schedules them. SASS changes with every GPU generation (Volta is `sm_70`, Ampere A100 is `sm_80`, Hopper H100 is `sm_90`), and NVIDIA does not publish a complete specification. You read it anyway, because it is the only place you can see the final register count, spills, the real memory instructions, and whether tensor cores are used.

Getting SASS

```
nvcc -cubin -arch=sm_80 kernel.cu -o kernel.cubin
cuobjdump -sass kernel.cubin
```

`nvdiasm` gives a lower-level view of the same file. Nsight compute’s “source” page shows SASS side by side with your CUDA code and with per-instruction stall statistics, which is the most useful view of all. The CUDA toolkit installs and runs on a machine without a GPU, so you can study SASS on a laptop.

The vector add in SASS

A note on this listing: the environment where this book was built had no NVIDIA toolchain, so unlike every other listing in this book, this one was not printed by a tool during the build. It shows the standard shape of what `nvcc` (CUDA 12) produces for the `vadd` kernel on `sm_80`. Register numbers and the order of independent instructions may differ in your output. The instructions and their meaning will not.

```
MOV R1, c[0x0][0x28]           // set up the stack pointer (unused
here)
S2R R6, SR_CTAID.X             // R6 = blockIdx.x
S2R R3, SR_TID.X               // R3 = threadIdx.x
IMAD R6, R6, c[0x0][0x0], R3    // R6 = blockIdx.x * blockDim.x +
threadIdx.x = i
ISETP.GE.AND P0, PT, R6, c[0x0][0x178], PT // P0 = (i >= n)
@P0 EXIT                      // threads with i >= n stop here
HFMA2.MMA R7, -RZ, RZ, 0, 2.384185791015625e-07 // R7 = 4 (see below)
ULDC.64 UR4, c[0x0][0x118]     // memory descriptor into a uniform
register
IMAD.WIDE R4, R6, R7, c[0x0][0x168] // R4:R5 = b + i * 4
IMAD.WIDE R2, R6, R7, c[0x0][0x160] // R2:R3 = a + i * 4
```

```

LDG.E R4, desc[UR4][R4.64]           // R4 = b[i]
LDG.E R3, desc[UR4][R2.64]           // R3 = a[i]
IMAD.WIDE R6, R6, R7, c[0x0][0x170]  // R6:R7 = c + i * 4
FADD R9, R4, R3                       // R9 = a[i] + b[i]
STG.E desc[UR4][R6.64], R9           // c[i] = R9
EXIT
BRA 0x...                             // padding: a jump to itself, never
reached

```

Reading it

Registers. R0 to R254 are 32-bit per-thread registers. RZ always reads as zero (like RISC-V's x0). A 64-bit value uses a pair: R2.64 means R2 (low half) with R3 (high half). P0 to P6 are predicate registers, PT always reads as true. UR registers are **uniform registers** (Turing and later): one value for the whole warp, used for things every thread agrees on, like the memory descriptor here.

Constant bank 0. c[0x0][offset] reads from constant memory bank 0, where the driver puts launch information. c[0x0][0x0] is blockDim.x. On sm_70 and sm_80 the kernel parameters start at offset 0x160: so a is at 0x160, b at 0x168, c at 0x170 (each 8 bytes) and n at 0x178. Reading a parameter costs nothing extra, because most instructions can take a constant-bank operand directly. On sm_90 the parameters start at a different offset (0x210).

Special registers. S2R (“special to register”) reads SR_TID.X, SR_CTAID.X and similar. It has a long latency, so the compiler issues these first.

Predication. ISETP.GE.AND P0, PT, R6, c[0x0][0x178], PT is an integer compare that writes P0. The two PT operands are an extra predicate combined with AND, unused here. @P0 EXIT runs EXIT only in threads where P0 is true. Any instruction can be predicated this way, which is how short ifs compile without branches.

IMAD.WIDE. A 32×32 bit multiply-add producing a 64-bit result in a register pair.

IMAD.WIDE R2, R6, R7, c[0x0][0x160] computes $a + i * 4$ as a 64-bit address in one instruction. Note there is no separate “add the base pointer”: the base comes straight from the constant bank.

The 4 in R7. HFMA2.MMA R7, -RZ, RZ, 0, 2.384185791015625e-07 is a trick. HFMA2 does two half-precision FMAs on the two 16-bit halves of a register. Here it computes $(-0 \times 0) + k$, which just writes the constant k. The constant's bit pattern is 0x00000004: in the low half, 0x0004 read as an f16 is $2^{-24} \times 4 = 2.38e-07$, and the high half is 0. So R7 gets the integer 4. The compiler uses this instead of a MOV because the .MMA form runs on a different execution unit, leaving the integer units free. You will see this odd-looking instruction all the time. It just loads a constant.

Memory. `LDG.E` is a load from global memory, `.E` meaning a 64-bit address. `desc[UR4]` attaches a memory descriptor (CUDA 11 and earlier print the same instruction without it). `STG.E` is the store. Other memory instructions you will meet:

SASS	Meaning
<code>LDG.E.128</code> , <code>LDG.E.64</code>	16 or 8-byte global load (from <code>ld.global.v4.f32</code> and similar)
<code>LDG.E.CONSTANT</code>	Global load through the read-only path (<code>ld.global.nc</code>)
<code>LDS</code> , <code>STS</code>	Shared memory load and store
<code>LDSM</code>	<code>ldmatrix</code> : load tensor core fragments from shared memory
<code>LDGSTS</code>	Asynchronous copy global to shared (<code>cp.async</code> , Ampere)
<code>LDL</code> , <code>STL</code>	Local memory: register spills
<code>BAR.SYNC</code>	<code>__syncthreads()</code>
<code>SHFL.DOWN</code> , <code>SHFL.BFLY</code>	Warp shuffles
<code>RED</code> , <code>ATOM</code>	Atomics

Arithmetic. `FADD` , `FMUL` , `FFMA` are f32 add, multiply, FMA. `HFMA2` does 2 f16 FMAs at once. `IADD3` adds three integers. `LOP3` computes any logic function of three inputs, selected by an 8-bit lookup table. `HMMA.16816.F32` is the tensor core instruction behind `mma.m16n8k16` with f32 accumulation, and on Hopper `HGMMMA` is behind `wgmma` .

What to look for

When you open the SASS of a kernel you care about, check these in order:

1. **The register count** (printed by `nvcc -Xptxas -v` or shown in Nsight compute). It sets the occupancy (chapter 13).
2. **LDL and STL** . Any spills in the hot loop are a serious problem. The fix is fewer live values (a smaller register tile) or an explicit register limit with `__launch_bounds__` .
3. **The width of memory instructions.** `LDG.E.128` and `LDS.128` where you expected vectorized access. Plain `LDG.E` in a loop over many consecutive floats means the vector loads did not happen, usually because alignment could not be proven.
4. **The inner loop's ratio of math to memory.** A good matmul inner loop is a long run of `FFMA` or `HMMA` with a few `LDS` between them. A loop that is half `LDG` is memory bound.

5. **Tensor cores.** For f16/bf16 matmuls, `HMMMA` must be there. If it is not, the kernel uses a small fraction of the GPU's possible throughput.
6. `BAR.SYNC count` in the main loop. Each one makes the whole block wait.

Scheduling is in the instructions

On a CPU, the hardware decides at run time which instruction can go next (out-of-order execution). NVIDIA GPUs mostly do not: `ptxas` decides, and encodes **control information** inside each instruction: how many cycles to wait before issuing the next instruction, and which scoreboard barriers a slow operation (like a load) sets and which instructions wait on them. `cuobjdump` does not print these bits by default, but they are why the order of instructions in SASS matters and why `ptxas` spends so long on big kernels. It is also why the same PTX can produce kernels of quite different speed with different `ptxas` versions.

Exercises

1. In the SASS above, which instructions correspond to the PTX `mad.lo.s32` and `setp.ge.s32 + @%p1 bra ?`
2. Why does the vector add have only one `EXIT` for the “ $i \geq n$ ” threads and no branch?
3. A kernel's SASS shows `STL [R1+0x10], R12` in its inner loop. What does it mean, and what would you try first?
4. Compile `vadd4` from chapter 14 for `sm_80` and find the 128-bit loads in its SASS.

Chapter 16

Assembly and ML compilers

ML compilers like tinygrad, Triton and TVM take a graph of tensor operations and generate kernels from it. Many of them never emit assembly themselves: they generate C, CUDA or LLVM IR and let clang, nvcc or LLVM do the last step. This chapter uses a tinygrad-style design as its example, where the graph is made of small ops called uops and the output is C or CUDA. If you build or work on such a compiler, assembly is how you check that the generated code does what you meant, and how you find out why it is slow.

How uops become instructions

Every uop ends up as a recognizable pattern in the final machine code. Once you know the mapping, you can read a kernel's assembly and point at the uop each instruction came from.

Uop	C / CUDA your compiler emits	What it becomes in assembly
CONST	A literal	An immediate operand, or a load from a constant pool
ADD, MUL, MAX	<code>a + b</code> , <code>a * b</code> , <code>fmaxf(a, b)</code>	One arithmetic instruction each; a <code>MUL</code> feeding an <code>ADD</code> usually fuses into one FMA
CMPLT, WHERE	<code>a < b ? x : y</code>	A compare plus a select: <code>cmov</code> / <code>vblendvps</code> on x86, a predicate plus <code>SEL</code> on GPU, rarely a branch
CAST	<code>(float)i</code>	A conversion instruction (<code>cvt</code> in PTX, <code>vcvtqdq2ps</code> on x86)
EXP2, LOG2	<code>exp2f(x)</code>	On GPU a single fast special-function instruction (<code>ex2.approx</code> in PTX, <code>MUFU.EX2</code> in SASS); on CPU a call or an inlined polynomial
INDEX	<code>buf + idx</code>	Address arithmetic: shifts, <code>lea</code> , <code>IMAD.WIDE</code> . Often folded into the load's addressing mode
LOAD, STORE	<code>buf[idx]</code>	<code>ld</code> / <code>mov</code> / <code>vmovups</code> , <code>ld.global</code> / <code>LDG</code> , <code>ld.shared</code> / <code>LDS</code>
RANGE (CPU)	A <code>for</code> loop	A counter register, an increment, a compare, a branch back
RANGE (GPU, mapped to an axis)	<code>threadIdx.x</code> , <code>blockIdx.x</code>	<code>S2R</code> reads of special registers, and no loop at all
REDUCE	An accumulator updated in a loop	A register updated by a chain of adds or FMAs inside the loop
Upcast reduce	Several accumulators	Several independent registers: the 8 <code>ymm</code> accumulators of chapter 12
Upcast load	4 consecutive elements per thread	<code>vmovups ymm</code> on CPU, <code>ld.global.v4.f32</code> / <code>LDG.E.128</code> on GPU

Uop	C / CUDA your compiler emits	What it becomes in assembly
A staged buffer / shared tile	A <code>__shared__</code> array	<code>st.shared</code> + <code>bar.sync</code> + <code>ld.shared</code>

The most important line is `RANGE`. On a CPU a range is a loop and costs instructions every iteration. On a GPU, a range mapped to a thread axis costs nothing at run time: it becomes the thread's own index. Deciding which ranges become loops and which become thread indices is most of what a GPU code generator does.

What to check as the compiler grows

Stage 1: correctness and simplification. Speed is not the goal yet, but assembly is still useful: compile your generated C with `-O2` and look at it. If your symbolic simplifier left `(x * 4 + 2) / 4` in the index math, the C compiler may or may not clean it up. If you see a division or a `div7`-style magic multiply inside a hot loop, your index simplification missed a case. Clean index math in the C source should become a pointer that adds a constant each iteration.

Stage 2: fast CPU kernels. This is where reading assembly becomes the main tool. For a matmul or reduce kernel your compiler generated, check:

1. The inner loop uses `ps` and `ymm` (or `zmm`) instructions, not `ss` and `xmm`. If not, it did not vectorize; find out why with `-Rpass-missed=loop-vectorize`.
2. Reductions are split across several independent accumulator registers. One accumulator means one dependency chain, which caps the loop at the FMA latency (chapter 11).
3. Accumulators stay in registers for the whole k loop: no stores and reloads of them inside the loop.
4. The ratio of loads to FMAs is low. The tile kernel of chapter 12 had 6 loads per 8 FMAs. A loop with more loads than FMAs is memory bound.
5. There are no spills. On x86 a spill is a store to `[rsp + ...]` inside the hot loop followed by a reload.

Stage 3: GPU kernels. For each generated kernel, compile to PTX and SASS and check:

1. Consecutive threads touch consecutive addresses (coalescing). Look at how `%tid.x` enters the address computation. It should be multiplied by the element size only, not by a row stride.
2. Loads are as wide as they can be: `ld.global.v4.f32`, `LDG.E.128`.
3. Shared memory tiles exist when data is reused: `st.shared`, `bar.sync`, then many `ld.shared` in the inner loop.

4. No `ld.local` / `st.local` / `LDL` / `STL`.
5. For matmuls in f16 or bf16: `mma` in PTX, `HMMA` in SASS.
6. The register count leaves enough occupancy (chapter 13).

A checklist you can print

When someone hands you a slow kernel, go through this in order, and stop at the first thing that is wrong:

1. Is it computing what you think? (Read the inner loop and map every instruction to the source.)
2. Is it memory bound or compute bound? (Count bytes and FLOPs; compare with the roofline.)
3. If memory bound: is the access in order and coalesced, are loads wide, is data reused from registers or shared memory instead of reloaded?
4. If compute bound: is it vectorized or on tensor cores, are there enough independent accumulators, is anything a division or a branch that could be removed?
5. Is anything spilling?
6. Only then: measure with perf or Nsight compute, change one thing, measure again.

Most of the time, the problem is found at step 3.

Where to go next

- Read the assembly of real kernels: the SASS of a CUTLASS or flash-attention kernel in Nsight compute, and the generated C and PTX of tinygrad (`DEBUG=4` prints generated code).
- The RISC-V specification volume 1 is short and readable. The Intel software developer's manual volume 2 is the x86 reference. The PTX ISA document from NVIDIA is the PTX reference and is very good.
- uops.info for x86 instruction costs. Agner fog's optimization manuals for how x86 cores work inside.
- "computer systems: a programmer's perspective" (Bryant and o'hallaron) for the full systems picture. "programming massively parallel processors" (Hwu, Kirk, El Hajj) for GPUs.

Appendices

Appendix A

Exercise solutions

Every code solution below was assembled and run: RISC-V in the emulator against a Python reference, x86-64 natively against a C test program. Where an exercise asks you to try something on godbolt, the answer says what you should find.

Chapter 1

1. `0xb7` = `1011 0111` = 183 unsigned. Signed 8-bit: $-128 + 32 + 16 + 4 + 2 + 1 = -73$.
2. -1 is `0xffffffff`. -2 is `0xffffffe`.
3. `lb: 0xfffffffffffff80` (the sign bit is copied up, the value is -128). `lbu: 0x0000000000000080` (128).
4. The bytes from address `0x1000` are `ef be ad de`, so `0x1001` holds `0xbe`.
5. `-9 >> 3` is -2: the shift rounds toward negative infinity, so -1.125 becomes -2. `-9 / 8` is -1, because C division rounds toward zero.

Chapter 3

1. No multiply:

`ex_lin.s` · risc-v, run and checked in an emulator

```
# a0 = 3 * a0 + 7, no mul
.globl lin
lin:
    slli t0, a0, 1      # t0 = 2x
    add  a0, a0, t0     # a0 = x + 2x = 3x
    addi a0, a0, 7      # a0 = 3x + 7
    ret
```

2. Two answers, both tested:

`ex_sign.s` · risc-v, run and checked in an emulator

```
# a0 = 1 if a0 < 0 else 0, two ways
.globl is_neg1
is_neg1:
    slt  a0, a0, zero   # compare with zero (signed)
    ret
.globl is_neg2
```

```
is_neg2:
    srli a0, a0, 63    # move the sign bit down to bit 0
    ret
```

- 2048 does not fit in a 12-bit signed immediate (the maximum is 2047), so it needs two instructions. LLVM emits `li t0, 1` then `slli t0, t0, 11`. Another correct form is `lui t0, 1` (4096) then `addiw t0, t0, -2048`.
- 1 is all ones. Shifting right logically by 60 leaves 4 ones: `t0 = 15`.

Chapter 4

- Max: the listing in chapter 5 (`max.s`).
- Rev: the listing in chapter 5 (`rev.s`). The `beqz a1` at the top handles `n = 0`, where “the last element” would be one element before the array.
- Dot product:

`ex_dot.s` · risc-v, run and checked in an emulator

```
# long dot(long *a, long *b, long n)
.globl dot
dot:
    li    t0, 0        # total
    slli  a2, a2, 3
    add   a2, a0, a2    # end of a
1:
    beq   a0, a2, 2f
    ld    t1, 0(a0)
    ld    t2, 0(a1)
    mul   t1, t1, t2
    add   t0, t0, t1
    addi  a0, a0, 8
    addi  a1, a1, 8
    j     1b
2:
    mv    a0, t0
    ret
```

- `val` is at offset 0, `next` at offset 8:

`ex_list.s` · risc-v, run and checked in an emulator

```
# struct node { long val; struct node *next; };    val at 0, next at 8
# long list_sum(struct node *p)
.globl list_sum
list_sum:
    li    t0, 0
1:
    beqz  a0, 2f        # null pointer ends the list
```

```

    ld    t1, 0(a0)      # p->val
    add   t0, t0, t1
    ld    a0, 8(a0)      # p = p->next
    j     1b
2:
    mv    a0, t0
    ret

```

5. `blt` compares addresses as signed numbers. On Linux user space, addresses are below 2^{47} , so they never look negative and it would work in practice. It is still wrong in principle: addresses are unsigned, so `bltu` is the correct comparison. `beq` / `bne` avoids the question entirely, which is why compilers use it.

Chapter 5

1. Gcd:

`ex_gcd.s` · risc-v, run and checked in an emulator

```

# long gcd(long a, long b): euclid, inputs >= 0
.globl gcd
gcd:
1:
    beqz a1, 2f          # while (b != 0)
    rem  t0, a0, a1      # t = a % b
    mv   a0, a1          # a = b
    mv   a1, t0          # b = t
    j    1b
2:
    ret

```

2. Binary search over [lo, hi):

`ex_bsearch.s` · risc-v, run and checked in an emulator

```

# long bsearch(long *a, long n, long x): index of x in sorted a, or -1
.globl bsearch
bsearch:
    li    t0, 0          # lo = 0
    mv    t1, a1          # hi = n    (search range is [lo, hi))
1:
    bge   t0, t1, 4f      # empty range: not found
    add   t2, t0, t1
    srli  t2, t2, 1        # mid = (lo + hi) / 2
    slli  t3, t2, 3
    add   t3, a0, t3
    ld    t3, 0(t3)        # t3 = a[mid]
    beq   t3, a2, 3f       # found
    blt   t3, a2, 2f       # a[mid] < x: go right

```

```

    mv    t1, t2        # else go left: hi = mid
    j     1b
2:
    addi  t0, t2, 1      # lo = mid + 1
    j     1b
3:
    mv    a0, t2
    ret
4:
    li    a0, -1
    ret

```

3. Bubble sort:

ex_bubble.s · risc-v, run and checked in an emulator

```

# void bubble(long *a, long n): sort ascending
.globl bubble
bubble:
    addi  a1, a1, -1     # outer: passes = n - 1
1:
    blez  a1, 4f         # no passes left
    mv    t0, a0         # p = a
    slli  t1, a1, 3
    add   t1, a0, t1     # last = a + passes   (compare p[0] with p[1] while p <
last)
2:
    bgeu  t0, t1, 3f
    ld    t2, 0(t0)
    ld    t3, 8(t0)
    ble   t2, t3, 5f     # already in order
    sd    t3, 0(t0)     # swap
    sd    t2, 8(t0)
5:
    addi  t0, t0, 8
    j     2b
3:
    addi  a1, a1, -1     # the biggest value is now at the end; one fewer pass
    j     1b
4:
    ret

```

4. Max with a rotated loop:

ex_max_rot.s · risc-v, run and checked in an emulator

```

# long max(long *a, long n), n >= 1, rotated loop: one branch at the bottom
.globl max_rot
max_rot:
    ld    t0, 0(a0)      # best = a[0]
    slli  a1, a1, 3
    add   a1, a0, a1     # end

```

```

    addi a0, a0, 8      # p = &a[1]
    beq  a0, a1, 3f     # n == 1: nothing else to look at
1:
    ld   t1, 0(a0)
    bge  t0, t1, 2f
    mv   t0, t1
2:
    addi a0, a0, 8
    bne  a0, a1, 1b     # test and jump back in one instruction
3:
    mv   a0, t0
    ret

```

5. With `bgeu`, `x = -5` is read as an enormous unsigned number, so “`x >= lo`” is true and the function goes on to compare with `hi`. `-5` compared unsigned with `hi` is again “greater”, so `clamp` returns `hi` instead of `lo`. A wrong answer from a single wrong letter.

Chapter 6

1. `square` calls nothing, so nothing overwrites `ra`. `sum_squares` executes `call square`, which writes a new value into `ra`.
2. `T0` is caller-saved. The recursive `call fib` is allowed to overwrite `t0` (and does, since `fib` itself uses `t0`). After the call, `n` would be garbage.
3. Recursive sum:

`ex_sumrec.s` · risc-v, run and checked in an emulator

```

# long sum_rec(long *a, long n) = n == 0 ? 0 : a[0] + sum_rec(a + 1, n - 1)
.globl sum_rec
sum_rec:
    bnez a1, 1f
    li   a0, 0          # base case
    ret
1:
    addi sp, sp, -16
    sd   ra, 8(sp)
    sd   s0, 0(sp)
    ld   s0, 0(a0)      # s0 = a[0], needed after the call
    addi a0, a0, 8      # a + 1
    addi a1, a1, -1     # n - 1
    call sum_rec
    add  a0, s0, a0      # a[0] + sum_rec(...)
    ld   s0, 0(sp)
    ld   ra, 8(sp)
    addi sp, sp, 16
    ret

```

For $n = 2$ there are two frames on the stack at the deepest point (from the calls with $n = 2$ and $n = 1$), each holding a saved `ra` and a saved `s0`. The $n = 0$ call returns without a frame.

4. $5 \text{ registers} \times 8 \text{ bytes} = 40 \text{ bytes}$, rounded up to 48 to keep `sp` a multiple of 16.
5. After the callee returns, the caller would immediately return too, using the same `ra`. So the caller can restore its saved registers, free its frame, and jump straight to the callee, which will return directly to the caller's caller. The caller's frame is gone before the callee starts, so deep chains of tail calls use no extra stack.

Chapter 7

1. Two writes (note that `a0` must be set again, because the first `ecall` put its return value there):

`ex_hello2.s` · risc-v, run and checked in an emulator

```
.section .rodata
m1: .ascii "first line\n"
.set n1, . - m1
m2: .ascii "second line\n"
.set n2, . - m2
.text
.globl _start
_start:
    li    a0, 1
    la    a1, m1
    li    a2, n1
    li    a7, 64
    ecall
    li    a0, 1           # a0 was overwritten by the first write's return value
    la    a1, m2
    li    a2, n2
    li    a7, 64
    ecall
    li    a0, 0
    li    a7, 93
    ecall
```

2. Exit with status 42:

`ex_exit42.s` · risc-v, run and checked in an emulator

```
.text
.globl _start
_start:
    li    a0, 42           # exit status
```

```
li    a7, 93          # exit
ecall
```

3. Funct7 for `sub` is `0100000`, which sets bit 30: `0x407302b3`. The assembler agrees: bytes `b3 02 73 40`.
4. The immediate is a signed 20-bit count of 2-byte units: up to $(2^{19} - 1) \times 2 = 1,048,574$ bytes forward, about 1 MB. That is why `call` sometimes becomes `auipc` + `jalr`, which can reach ± 2 GB.

Chapter 8

1. clang walks a pointer up to an end address, like `ex_max_rot.s` above. RV64IM has no conditional move instruction, so the update stays a branch (`blt`).
2. `x * 9` becomes `lea rax, [rdi + 8*rdi]`. `x * 10` becomes `add rdi, rdi (2x)` followed by `lea rax, [rdi + 4*rdi]` ($5 \times 2x$). No `imul` in either.
3. The magic number is 3435973837 (`0xcccccccd`), followed by a shift right of 35: `x / 10 == (x * 0xcccccccd) >> 35`. For `x = 1000`: $3,435,973,837,000 / 2^{35} = 100.000000005$, which truncates to 100.
4. The whole loop is gone. A function returning nothing compiles to a bare `ret`.

Chapter 9

1. Rev in x86-64:

`ex_rev.s` · x86-64, run and checked natively

```
# void rev(long *a, long n)
.intel_syntax noprefix
.globl rev

rev:
    test rsi, rsi
    jz    .Ldone
    lea   rax, [rdi + 8*rsi - 8]
    .Lloop:
        cmp    rdi, rax
        jae    .Ldone
        mov    rcx, qword ptr [rdi]
        mov    rdx, qword ptr [rax]
        mov    qword ptr [rdi], rdx
        mov    qword ptr [rax], rcx
        add    rdi, 8
        sub    rax, 8
        jmp    .Lloop
```

```
.Ldone:
    ret
```

2. `mov eax, 5` zeroes the upper 32 bits: `rax = 5`. `mov al, 5` changes only the low byte: `rax = 0xffffffffffff05`. Both tested on real hardware.

3. Dot product:

`ex_dot.s` · x86-64, run and checked natively

```
# long dot(long *a, long *b, long n)    rdi = a, rsi = b, rdx = n
    .intel_syntax noprefix
    .globl dot
dot:
    xor    eax, eax                    # total
    xor    ecx, ecx                    # i
.Lloop:
    cmp    rcx, rdx
    jge    .Ldone
    mov    r8, qword ptr [rdi + 8*rcx]
    imul   r8, qword ptr [rsi + 8*rcx]  # r8 = a[i] * b[i]
    add    rax, r8
    inc    rcx
    jmp    .Lloop
.Ldone:
    ret
```

4. A negative signed index, read as unsigned, is a number at least 2^{63} , far larger than any length. So the single unsigned test “index < length” rejects both negative and too-large indices.
5. On entry `rsp` ends in 8. Two pushes move it by 16, so it still ends in 8: misaligned at the call. The function must also subtract 8 (or push a third register). With three pushes it ends in 0: aligned.

Chapter 10

1. Without `restrict`, clang adds a run-time check that compares the two pointer ranges and keeps both a vector loop and a scalar loop, choosing at run time.
2. Without `-ffast-math`: only scalar `vaddss`, one chain. With it: `vaddps` into four ymm accumulators (32 floats per iteration), combined at the end.
3. `N = 3` is below 4, so `cmp rsi, 4 / jae` is not taken: `rcx` and `rax` are zeroed and execution jumps straight to the scalar tail, which runs 3 times. The vector loop never runs.
4. 16 lanes, each a multiply and an add: 32 FLOPs.

Chapter 11

- 1 million $\times$ 4 cycles = 4 million cycles = 1.33 ms at 3 GHz. With 8 chains and 2 adds started per cycle: 500,000 cycles = 0.17 ms.
- In pointer chasing, each load's address comes from the previous load, so only one load is ever in flight: total time = count $\times$ latency. In-order access has independent addresses and a prefetcher, so many loads overlap and the limit becomes bytes per second.
- A dot product loads 8 bytes per element and does 2 FLOPs: 0.25 FLOPs per byte. At 16 GB/s that is 4 GFLOP/s: memory bound.
- About half the branches go each way at random, so roughly 0.5 mispredictions per element. perf will report close to that.

Chapter 12

- Each 4×16 tile reads 16 floats of B for each of 512 values of k. There are $128 \times 32 = 4096$ tiles: $4096 \times 512 \times 16 = 33.5$ million floats. `mm_ikj` reads all of B (262,144 floats) for each of 512 rows of A: 134 million floats. The tile version reads 4x less B.
- 16×16 would need 32 accumulator registers of 8 floats, plus registers for the loaded B values and the broadcast A. There are only 16 ymm registers. 4×16 needs 8 accumulators + 2 + 1 = 11.
- gcc may not fully unroll the inner `ii / jj` loops, leaving accumulators in memory. Without AVX, only xmm registers (4 floats) and no FMA are available.
- 6×16 needs 12 accumulators + 2 + 1 = 15 registers: it fits. It reuses each B load 6 times instead of 4, so it should be somewhat faster. Measure it.

Chapter 13

- $4096 \times 256 = 1,048,576$ threads. $256 / 32 = 8$ warps per block.
- $96 \text{ registers} \times 32 \text{ threads} = 3072$ registers per warp. $65536 / 3072 = 21$ warps.
- Both paths run one after the other with half the threads masked: about the time of A plus B.
- No. At each step, the 32 threads of a warp read 32 different rows, 4096 bytes apart: 32 sectors per load. Map the column index to `threadIdx.x` instead, so a warp reads 32 consecutive floats of one row.

Chapter 14

- The `setp.ge.s32` and the `@%p1 bra`, and the load of n.

2. In each step, threads read values that other threads wrote in the previous step. Without a barrier, a thread could read a partner's slot before the partner has finished writing it.
3. Word `tile[t][c]` is at index $32*t + c$, so its bank is $(32*t + c) \% 32 = c$: every thread hits the same bank, a 32-way conflict. With 33 columns the index is $33*t + c$ and the bank is $(t + c) \% 32$: all different banks.
4. Per k: 2 loads of 4 bytes = 8 bytes. For K = 1024: 8 KB of loads, plus 4 bytes stored. 2 FLOPs per k: 2048 FLOPs. 0.25 FLOPs per byte.
5. nvcc's PTX uses different register names and usually computes the addresses in a slightly different order. The loads, the add and the store are the same.

Chapter 15

1. `IMAD R6, R6, c[0x0][0x0], R3` is the `mad.lo.s32`. `ISETP.GE.AND` plus `@P0 EXIT` replace `setp.ge.s32` plus the predicated branch.
2. After the bounds test there is nothing left for those threads to do, so the compiler makes them exit with a predicated `EXIT` instead of jumping to the end.
3. A store to local memory at offset 0x10 from the per-thread stack pointer R1: a register spill. Reduce the number of live values in the loop (a smaller register tile, less unrolling) and check the register count with `-Xptxas -v`.
4. Look for `LDG.E.128` (and `STG.E.128`).

Appendix B

Cheat sheets

RISC-V RV64IM

Group	Instructions
Arithmetic	add sub addi ; 32-bit: addw subw addiw
Logic	and or xor andi ori xori
Shifts	sll srl sra slli srli srai
Compare	slt sltu slti sltiu
Multiply / divide	mul mulh mulhu div divu rem remu
Loads	lb lbu lh lhu lw lwu ld
Stores	sb sh sw sd
Branches	beq bne blt bge bltu bgeu
Jumps	jal jalr
Constants / addresses	lui auipc
System	ecall
Pseudo	li la mv not neg nop seqz snez j ret call beqz bnez bgt ble

Register	ABI name	Role	Saved by
X0	Zero	Always 0	
X1	Ra	Return address	Caller
X2	Sp	Stack pointer	Callee
X5-7, x28-31	T0-t6	Temporaries	Caller
X8-9, x18-27	S0-s11	Saved	Callee
X10-17	A0-a7	Arguments, return values	Caller

Stack: grows down, 16-byte aligned. Linux syscalls: number in a7 (write 64, exit 93, read 63), arguments in a0..a5, `ecall`.

x86-64 (system v: Linux, Mac)

Role	Registers
Integer arguments	Rdi, rsi, rdx, rcx, r8, r9
Float arguments	Xmm0 to xmm7
Return	Rax (rdx), xmm0
Callee-saved	Rbx, rbp, r12, r13, r14, r15
Stack	Rsp 16-byte aligned at every call

Jump after <code>cmp a, b</code>	Signed	Unsigned
<code>A < b</code>	<code>jl</code>	<code>jb</code>
<code>A <= b</code>	<code>jle</code>	<code>jbe</code>
<code>A > b</code>	<code>jg</code>	<code>ja</code>
<code>A >= b</code>	<code>jge</code>	<code>jae</code>

Linux syscalls: number in rax (write 1, exit 60, read 0), arguments in rdi, rsi, rdx, r10, r8, r9, `syscall`.

SIMD suffixes: `ps` packed f32, `pd` packed f64, `ss` scalar f32, `sd` scalar f64; integer `vpaddd{b,w,d,q}` for 8/16/32/64-bit lanes. Xmm 128, ymm 256, zmm 512 bits.

PTX

Item	Syntax
Instruction	<code>op.space.type dst, src1, src2;</code>
Predicated	<code>@%p1 bra L; , @!%p1 ...</code>
Compare	<code>setp.lt.s32 %p1, %r1, %r2;</code>
Thread ids	<code>%tid.x %ntid.x %ctaid.x %nctaid.x %laneid</code>
Loads	<code>ld.param, ld.global[.nc][.v2/.v4], ld.shared, ld.local</code>
Barrier	<code>bar.sync 0;</code>
Shuffle	<code>shfl.sync.down.b32 d, a, off, 31, -1;</code>
FMA	<code>fma.rn.f32 d, a, b, c;</code>
Tensor core	<code>mma.sync.aligned.m16n8k16.row.col.f32.f16.f16.f32</code>

SASS (sm_80)

SASS	Meaning
<code>S2R</code>	Read special register (tid, ctaid)
<code>IMAD, IMAD.WIDE</code>	Integer multiply-add, 64-bit result
<code>ISETP, FSETP</code>	Compare into predicate
<code>LDG, STG</code>	Global load / store
<code>LDS, STS</code>	Shared load / store
<code>LDL, STL</code>	Local memory: spills
<code>FFMA, FADD, FMUL</code>	f32 math
<code>HMMA</code>	Tensor core mma
<code>BAR.SYNC</code>	Block barrier
<code>SHFL</code>	Warp shuffle
<code>c[0x0][0x160]</code>	First kernel parameter (sm_70, sm_80)

Numbers to remember

Thing	Rough value
L1 hit	~1 to 2 ns (4 to 5 cycles)
L2 hit	~4 to 6 ns
L3 hit	~15 to 45 ns
Main memory, random	~80 to 170 ns
Branch mispredict	~15 to 20 cycles
Float FMA latency	4 cycles, 2 per cycle
Cache line	64 bytes (GPU sector: 32 bytes)
Page	4 KB
Warp	32 threads
A100 / H100 SMs	108 / 132
Registers per SM	$65,536 \times 32$ bits

Appendix C

Tools and commands

Compiling to assembly

```
clang -O2 -S file.c -o -           # x86-64, at&t syntax
clang -O2 -S -masm=intel file.c -o - # x86-64, intel syntax
clang -O3 -march=x86-64-v3 -S -masm=intel ... # allow avx2 and fma
clang --target=riscv64 -march=rv64im -O2 -S file.c -o -
rustc -C opt-level=3 --emit=asm --crate-type=lib file.rs
cargo rustc --release -- --emit=asm # for a cargo project
```

RISC-V on an x86 machine

```
sudo apt install gcc-riscv64-linux-gnu qemu-user
riscv64-linux-gnu-gcc -nostdlib -static prog.s -o prog && qemu-riscv64 ./prog
riscv64-linux-gnu-gcc -static test.c func.s -o test && qemu-riscv64 ./test
```

x86-64

```
gcc -nostdlib -static prog.s -o prog && ./prog # a whole program with _start
gcc -O1 test.c func.s -o test && ./test      # functions called from c
```

Inspecting binaries

```
objdump -d -M intel prog # disassemble
objdump -r file.o        # relocations
nm prog                 # symbols
llvm-mc -show-encoding f.s # print the bytes of each instruction
```

Measuring

```
perf stat ./prog # cycles, instructions, ipc
perf stat -e branch-misses,cache-misses ./prog
perf record ./prog && perf report # where time goes
perf annotate # hot instructions in the assembly
llvm-mca -mcpu=skylake loop.s # static analysis of a loop
```

GPU

```
nvcc -ptx -arch=sm_80 k.cu           # ptx
nvcc -cubin -arch=sm_80 k.cu -o k.cubin # machine code
cuobjdump -sass k.cubin              # sass
nvcc -Xptxas -v -arch=sm_80 -c k.cu   # register count, spills, shared memory
ncu --set full ./prog                 # nsight compute profile
```

clang can also emit PTX without the CUDA toolkit, which is how this book's PTX was made:

```
clang++ -x cuda --cuda-gpu-arch=sm_80 --cuda-device-only -nocudainc -nocudalib -O2 -S k.cu
```

(You then need small definitions of `__global__`, `__shared__` and `threadIdx` yourself, since the CUDA headers are not included.)

The benchmarks in this book

```
gcc -O2 -fno-tree-vectorize bench_ilp.c -o bench_ilp           # dependency chains
gcc -O2 bench_mem.c -o bench_mem                             # cache latency
gcc -O2 bench_seq.c -o bench_seq                             # in-order bandwidth
gcc -O1 -fno-if-conversion -fno-if-conversion2 -fno-tree-vectorize bench_branch.c -o bench_branch
clang -O3 -march=x86-64-v3 bench_mm.c -o bench_mm            # matmul
```

The dependency-chain benchmark:

bench_ilp.c · c source

```
// how much do independent accumulators matter? float sum, no -ffast-math,
// so the compiler must keep the order we wrote.
#include <stdio.h>
#include <time.h>
#define N 4096           // 16 KB of floats: fits in the L1 cache
#define REPS 200000
static float a[N];
static double now(void){ struct timespec t; clock_gettime(CLOCK_MONOTONIC,&t); return t.tv_sec+t.tv_nsec*1e-9; }

__attribute__((noinline)) float sum1(const float *x){
    float s = 0;
    for (int i = 0; i < N; i++) s += x[i];           // every add waits for the
previous add
    return s;
}
__attribute__((noinline)) float sum4(const float *x){
    float s0=0, s1=0, s2=0, s3=0;                  // 4 independent chains
```

```

    for (int i = 0; i < N; i += 4){ s0+=x[i]; s1+=x[i+1]; s2+=x[i+2]; s3+=x[i+3]; }
    return (s0+s1)+(s2+s3);
}
__attribute__((noinline)) float sum8(const float *x){
    float s0=0,s1=0,s2=0,s3=0,s4=0,s5=0,s6=0,s7=0; // 8 independent chains
    for (int i = 0; i < N; i += 8){ s0+=x[i]; s1+=x[i+1]; s2+=x[i+2]; s3+=x[i+3];
        s4+=x[i+4]; s5+=x[i+5]; s6+=x[i+6];
    }
    s7+=x[i+7]; }
    return ((s0+s1)+(s2+s3))+((s4+s5)+(s6+s7));
}
int main(void){
    for (int i = 0; i < N; i++) a[i] = (float)(i % 7) * 0.5f;
    float (*f[3])(const float*) = {sum1, sum4, sum8}; const char *name[3] = {"1
chain ", "4 chains", "8 chains"};
    volatile float sink = 0;
    for (int k = 0; k < 3; k++){
        double t0 = now(); for (int r = 0; r < REPS; r++) sink += f[k](a); double t
= now() - t0;
        printf("%s %3f ns per element\n", name[k], t * 1e9 / ((double)N * REPS));
    }
    return 0;
}

```

The cache latency benchmark:

bench_mem.c · c source

```

// pointer chasing: each load gives the address of the next load,
// so the cpu cannot start load k+1 before load k finishes. this measures latency.
#include <stdio.h>
#include <stdlib.h>
#include <time.h>
static double now(void){ struct timespec t; clock_gettime(CLOCK_MONOTONIC,&t); retur
n t.tv_sec+t.tv_nsec*1e-9; }
int main(void){
    size_t sizes_kb[] = {16, 64, 512, 2048, 16384, 131072, 524288};
    for (int s = 0; s < 7; s++){
        size_t n = sizes_kb[s] * 1024 /
64; // one node per 64-byte cache line
        char *mem = aligned_alloc(64, n * 64);
        size_t *order = malloc(n * sizeof *order);
        for (size_t i = 0; i < n; i++) order[i] = i;
        for (size_t i = n - 1; i > 0; i--){ size_t j = ((size_t)rand() * RAND_MAX +
rand()) % (i + 1);
            size_t t = order[i]; order[i] = order[j]; order[j] = t; }
        for (size_t i = 0; i < n; i++) // random cycle through all
lines
            *(char**)(mem + order[i] * 64) = mem + order[(i + 1) % n] * 64;
        char *p = mem + order[0] * 64;
        size_t steps = 20000000;
        for (size_t i = 0; i < n; i++) p = *(char**)p; // warm up
        double t0 = now();
        for (size_t i = 0; i < steps; i++) p = *(char**)p;
    }
}

```

```

        double t = now() - t0;
        printf("%8zu KB    %6.1f ns per load    %s\n", sizes_kb[s], t * 1e9 / steps,
p ? "" : "x");
        free(mem); free(order);
    }
}

```

The branch benchmark:

bench_branch.c · c source

```

// the same loop over the same values, unsorted versus sorted.
// the branch inside is unpredictable on random data and almost free on sorted data.
#include <stdio.h>
#include <stdlib.h>
#include <time.h>
#define N (1 << 20)
static int a[N];
static double now(void){ struct timespec t; clock_gettime(CLOCK_MONOTONIC,&t); return
n t.tv_sec+t.tv_nsec*1e-9; }
static int cmp(const void *x, const void *y){ return *(const int*)x - *(const
int*)y; }
__attribute__((noinline)) long sum_big(const int *x, long n){
    long s = 0;
    for (long i = 0; i < n; i++){
        if (x[i] >= 128) s += x[i];        // taken about half the time
    }
    return s;
}
int main(void){
    srand(1); for (int i = 0; i < N; i++) a[i] = rand() % 256;
    volatile long sink;
    for (int pass = 0; pass < 2; pass++){
        if (pass == 1) qsort(a, N, sizeof a[0], cmp);
        sink = sum_big(a, N);
        double t0 = now(); for (int r = 0; r < 50; r++) sink = sum_big(a, N);
    double t = now() - t0;
        printf("%s %.2f ns per element\n", pass ? "sorted: " : "unsorted:", t *
1e9 / (50.0 * N));
    }
    return 0;
}

```